NIPT 检测时点优化与女胎异常判定

——基于 GLMM-DP/PSO 与 LightGBM 的研究

摘要

近年来，我国人口发展已迈入“质量优先”的关键阶段。无创产前检测 NIPT 致力于尽早、准确识别 13/18/21 等染色体异常，但孕周与 BMI 的共同作用使个体化检测时点与受控阈值的判定策略成为必需。本文以 GLMM 捕捉人群异质性，以 DP/PSO 生成分组化最优检测时点，并以 LightGBM 实现女胎异常监督判定，形成端到端可复现流程。该流程有助于降低复检与误检风险，提升资源利用率与孕妇就医体验。

针对问题一，为了构建 Y 染色体浓度与 BMI、孕周的相关性模型，本文首先拟合 Y 染色体浓度与孕周、BMI 等因素的线性、非线性关系，计算 **Spearman 相关性系数（p<0.05）**。之后，本文采用**自然样条**刻画 BMI 与 Y 染色体浓度的非线性关系，同时将其与孕周的交互项纳入**广义线性混合模型**，并对 GLMM 拟合结果从固定、随机效应及误差项角度分别分析。结果表明，GLMM 的边际 R 方为 0.7118，残差无系统结构。最后，本文通过对孕周分段，重新拟合 GLMM 并比对拟合结果，证明了模型的稳健性。

针对问题二，为了实现组合择时风险最小化，本文首先基于 GLMM 方差边际分布和 Y 染色体浓度 Z 值显著性，使用**逻辑回归**的方法计算一次成功检验概率为 0.87。接着设定孕周检测时点相应的风险系数，定义**一次检验期望风险函数**及组内**风险成本函数**。划分 BMI、定义孕周网格后，本文使用**动态规划**求得最优解为：BMI 分为(20.7, 30.0]、(30.0, 31.2]、(31.2, 32.7]、(32.7, 34.2]、(34.2, 45.7]五组，对应时点分别为 11、12、17、16.2、13.7 周。最后采用 **Bootstrap 统计重采样**方法算得综合稳定性分数为 0.8243，验证了模型的稳定性。

针对问题三，为了综合考虑多种个体因素及检测显著性、误差，首先采用**主成分分析**、中心化等多种方法对多元数据进行预处理，并将其加入 GLMM 模型的固定效应项或异方差项。拟合后计算一次检测成功概率，接着将其纳入**多阶段粒子群优化模型**中的期望损失函数。在施加**可行性惩罚约束**下，以**最小化适应度**为目标构建函数。求得结果为：BMI 组 1[20.7, 29.9)对应时点为 18.2、23.2、26.7 周，BMI 组 2[29.9, 33.6)对应时点为 17.5、18.6、21.7 周，BMI 组 3[33.6, 36.1)对应时点为 15.4、17.2、24.0 周，BMI 组 4[36.1, 46.9)对应时点为 16.5、19.0、22.1 周。最后计算多分组下的**变异系数(CV)**均小于 0.05，验证了模型的鲁棒性。

针对问题四，由分析可知，本文所提出的女胎异常判定决策是一个类不平衡的监督判定问题。首先，本文对异质的非线性数据进行预处理及特征工程。接着分别构建顶层二分类与细分类 OvR 的目标函数。之后，本文采用 **LightGBM 树模型**对数据多次进行 **GroupKFold 交叉验证**，并不断调试在 Recall 最大目标下的**可靠性门控与阈值**。结果表明，超过 95%的正常样本被正确归类在 0.08-0.12 的低风险区间，证明了模型的可靠性。

关键词： NIPT 检验 GLMM 动态规划 多阶段粒子群算法 LightGBM

一、 问题重述

1.1 问题背景

从全球医疗健康发展来看，产前检测技术是保障母婴健康、降低新生儿出生缺陷率的关键环节。随着分子生物学技术的发展，无创产前检测（NIPT）凭借其无创伤性、高准确性的独特优势，成为了全球产前筛查领域的核心技术之一。

我国人口发展已进入“质量优先”阶段，《“健康中国 2030”规划纲要》明确提出要“加强出生缺陷综合防治”，NIPT 作为产前筛查的核心技术被纳入多地妇幼健康服务体系。但该技术在实际应用中还存在一些问题：一方面，随着生活方式变化，孕妇群体的年龄、BMI（身体质量指数）、孕情等个体差异显著，而临床实践表明，这些因素可能会影响胎儿游离 DNA 在母体血液中的浓度，进而影响 NIPT 准确性；另一方面，部分医疗机构对 NIPT 检测时点的选择仍依赖传统经验分组，缺乏针对不同生理特征孕妇的个性化方案，导致部分孕妇因检测时点不当面临接收错误信息的风险，或因检测过晚错过最佳治疗窗口期。

因此，基于真实临床数据构建 NIPT 时点选择与胎儿异常判定的数学模型，既是提升我国产前检测精准度的现实需求，也是响应全球出生缺陷防治战略的重要举措。

1.2 问题提出

问题一：分析男胎 Y 染色体浓度与孕妇孕周数、BMI 等指标的相关特性，构建量化关系模型，并通过统计方法检验模型显著性。

问题二：以男胎孕妇为研究对象，基于 BMI 对其进行合理分组，确定每组的 BMI 区间与最佳 NIPT 时点，核心目标是最小化孕妇潜在风险（即尽可能早地在 12 周以内检测，且保证 Y 染色体浓度达标）。同时需分析检测误差对分组结果与最佳时点的影响，验证方案的准确性。

问题三：综合考虑身高、体重（可推导 BMI）、年龄等多因素对男胎 Y 染色体浓度达标时间的影响，结合检测误差与 Y 染色体浓度达标比例（浓度≥4%的样本占比），对男胎孕妇进行 BMI 分组并确定每组最佳 NIPT 时点，实现孕妇潜在风险最小化。

问题四：针对女胎无 Y 染色体的特性，以 13 号、18 号、21 号染色体非整倍体（AB 列判定结果）为核心目标，综合 X 染色体及上述三条染色体的 Z 值、GC 含量、读段数及相关比例、孕妇 BMI 等因素，构建女胎染色体异常的判定方法。

二、 问题分析

问题一：问题一的核心是明确 Y 染色体浓度（NIPT 男胎检测关键指标）与孕周、BMI 的关联并验证模型。需先通过相关性分析验证关联显著性，再用自然样条处理孕周非线性，最终构建 GLMM（含个体随机效应、误差项），结合拟合优度与分组检验确保模型可靠。

问题二：问题二是问题一的临床落地，目标是最小化时间风险和检测风险（浓度达标且显著）。解决问题需要平衡分组粗细来避免无差异或操作复杂，并通过 GLMM 方差拟合成功概率。再通过定义风险函数（含组数惩罚），用动态规划遍历 BMI 切点与孕周网格求最优分组，最后经过 Bootstrap 重采样验证结果稳定性。

问题三：问题三在问题二基础上加入年龄、IVF、数据质量等因素（均间接影响 Y 浓度达标）。这使得模型变得冗余（如数据质量指标共线性），多因素多时点风险使得目标更复杂。本文先通过 PCA 降维、变量中心化等方法处理数据，再通过纳入新变量与误差项异方差性改进 GLMM，最后用多阶段 PSO 算法求解最小总体风险，验证分组边界稳定性。

问题四：女胎因为不存在 Y 染色体，是否异常需要聚焦 13、18、21 号染色体非整倍体，需要解决类不平衡和标签噪声。为了判定异常并定位染色体，采用 LightGBM 模型，通过顶层二分类-细分层来判定，最后结合 GroupKFold 交叉验证及其与可靠性相关的核心指标确保泛化性。

三、 模型假设与符号说明

3.1 模型基本假设

(1) 假设实验平台与流程在研究期内保持稳定、无系统性批次效应影响模型结论。

(2) 假设问题一中仅考虑核心变量 BMI、孕周对 Y 染色体浓度的影响；误差项来源于其他弱相关变量。

(3) 假设信息质量、年龄信息不对成功概率产生影响。

(4) 假设数据口径一致，本文仅针对首检决策与评估，训练阶段涉及流程变量仅作控制，多次测验成功概率等于首检成功概率。

3.2 符号说明

表 1 符号说明

符号	含义	备注
i	第 i 位孕妇	孕妇编号
j	第 j 次检测	次数编号
V	男胎 Y 染色体浓度	单位：%
J	检测孕周	单位：周
K	孕妇 BMI 指标	
Age	孕妇年龄	单位：岁
R	单样本一次检验的期望风险	
G	BMI 总组数	
L	多阶段期望损失总和	
Recall	召回率	

四、 数据预处理

为确保数据质量及后续分析的准确性，本研究对原始数据进行系统预处理，主要包括以下方面：

4.1 数据初步探索

(1) NIPT 检测可靠性分析

以女胎孕妇的 21 号、18 号和 13 号染色体非整倍体检测结果（AB 列）为依据，与胎儿出生后的实际健康状况（AE 列）进行逐一比对，生成 NIPT 检测准确性指标

列。其中，当检测结果与胎儿实际健康状况一致时，指标值记为 1 否则指标值记为 0。该指标列的生成为后续根据数据判定女胎是否异常的探究奠定基础。

(2) **孕周数据规范化处理**

将孕周 J 连续化，$t_i = weeks_i + \dfrac{days_i}{7}$

(3) **GC 含量指标筛查与验证**

对 13、18、21 及总 GC 含量绘制分布曲线，发现数据均在 40 左右范围波动。

(4) **数据质量筛查**

对读段数及相关比例绘制箱线图，发现原始读段数与唯一比对的读段数有明显异常值，记各读段数的上四分位点、中位数、下四分位点为Q_3、Q_2和Q_1。记四分位距$IQR = Q_3 - Q_1$。获取数据质量超出正常分布区间的异常值，其中正常分布区间为：$[Q_1 - 1.5 \times IQR，\ Q_3 + 1.5 \times IQR]$，然后求得去除异常值之后日销量的平均值。并用该平均值替换异常值，从而获得异常值处理之后的整体数据。

(5) **身高、体重数据**

由于身高和体重与 BMI 具有一定的关联，因此不采用身高、体重数据，在后续建模过程中仅使用 BMI 作为变量。

(6) **IVF 妊娠、产次、孕次、检测次数数据**

查阅相关文献得知[1]，IVF 妊娠方式、产次、孕次及检测次数会产生一定的个体差异并较弱影响 Y 染色体浓度，不构成与 Y 染色体浓度的直接关联。故在后续分析中作为协变量或者影响因子加入模型。

(7) **末次月经、检测日期与孕周的关联**

查阅相关资料知[2]，得末次月经时间到检测日期时间间隔大致为孕周时长。检验数据后进一步确认了数据的准确性。

(8) **13、18、21 号染色体与 X、Y 染色体 Z 值**

根据 Z-score 方法，可知 Z 值反映数据对于正常数值的偏离。由此，我们绘制了各个染色体的 Z 值群体分布，发现部分个体 Y 染色体浓度大于 4 但是 Z 值严重偏离正常数据。因此我们将 Z 值大小加入后续模型，反映 NIPT 检测的准确性与显著性。

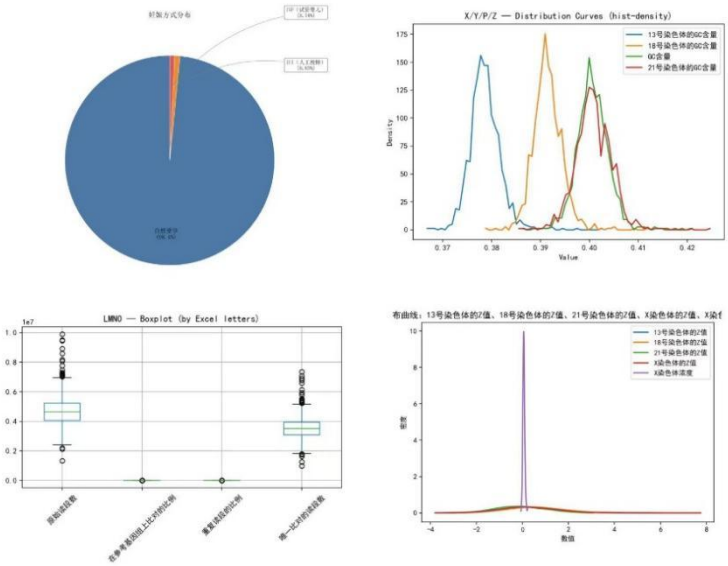

图 1 数据初步探索结果图集

4.2 核心变量描述性统计分析

表 2 描述性统计分析表

	年龄	检测孕周	孕妇 BMI	GC 含量	Y 染色体浓度
均值	28.94	16.86	32.29	0.4	0.08
标准差	3.66	4.07	2.97	0	0.03
最小值	21	11	20.7	0.39	0.01
第 25 百分位数	27	13	30.21	0.4	0.05
中位数	29	16	31.81	0.4	0.08
第 75 百分位数	31	20	33.93	0.4	0.1
最大值	43	29	46.88	0.42	0.23

注：总共样本量为 1082

表 3 检测结果准确性统计表

	检测结果准确性
均值（%）	86
标准差（按 0-1 计算）	0.35
检测正确占比(%)	85.58

以 1082 例全量样本为分析对象，针对与 NIPT 检测结果及孕妇基础特征相关的核心变量，开展描述性统计分析，明确各变量的分布特征，为后续变量相关性挖掘与异常影响因素定位提供基础依据。

4.3 数据可视化

4.3.1 男胎 X/Y 染色体 Z 值分级筛选与异常识别

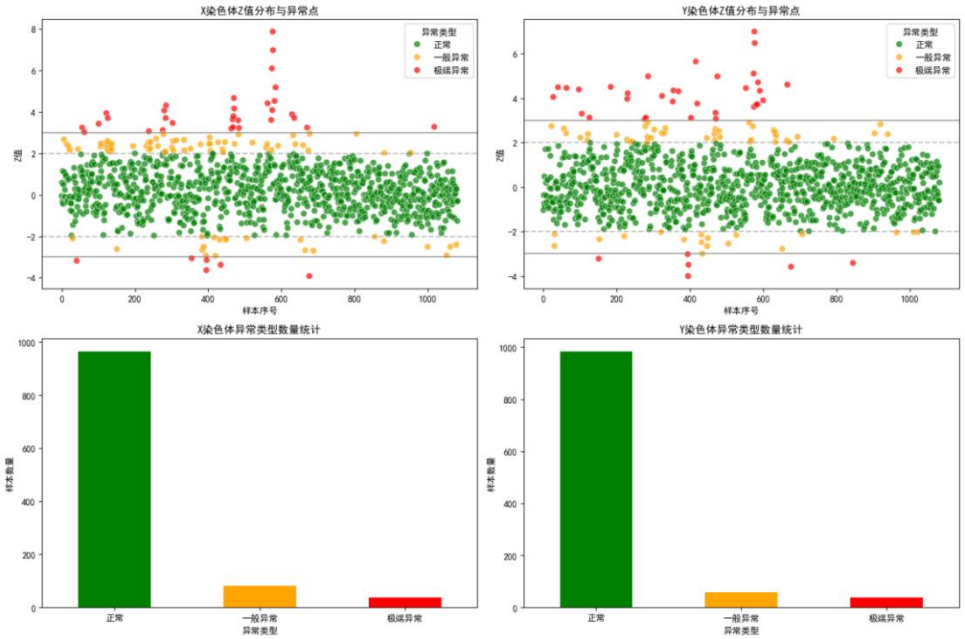

图 2 男胎 X/Y 染色体 Z 值分级筛选与异常识别图集

针对男胎样本，聚焦 X 染色体 Z 值（数据集 T 列）与 Y 染色体 Z 值（数据集 U 列）开展分级筛选，依据 NIPT 中染色体非整倍体的 Z 值统计判定逻辑，制定分级标准：若样本$2<|z|\leq 3$，则判定为一般异常，若样本$|z|>3$，则判定为极端异常。进行可视化

分析，结果显示绝大多数样本的 X、Y 染色体 Z 值处于合理区间内，符合正常对照群体的统计规律，一般异常与极端异常样本的累计占比极低，无需对整体数据进行大规模异常剔除。

4.3.2 检测孕周分布合规性校验

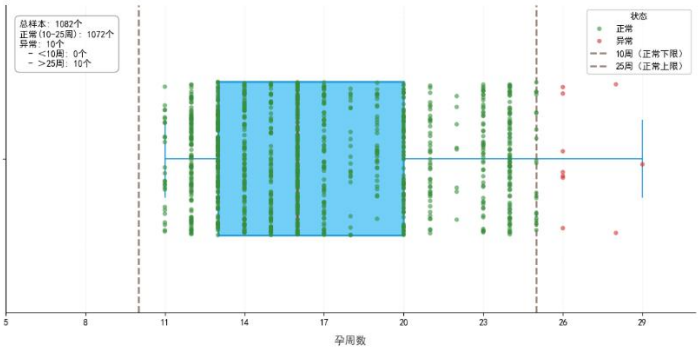

图 3 检测孕周分布箱图

对检测孕周的分布统计结果表明：规定 10-25 周为正常范围，仅发现 10 个样本的检测孕周超过 25 周，异常率为 0.92%。该比例较小，对整体数据的代表性未构成显著影响。无需对孕周数据进行额外的校正或样本剔除处理，可直接用于后续相关性分析与模型构建。

五、 问题一模型建立与求解

5.1 问题一求解思路

问题一的核心是量化男胎 Y 染色体浓度与孕妇孕周、BMI 的相关特性并检验模型显著性。由于数据的非线性性并存在一定的组内相关性，本文采用广义线性混合模型。它能解决传统线性模型无法处理的数据非正态、存在组内相关性等复杂问题。并在最后对孕周进行分组回归验证模型的稳健性。

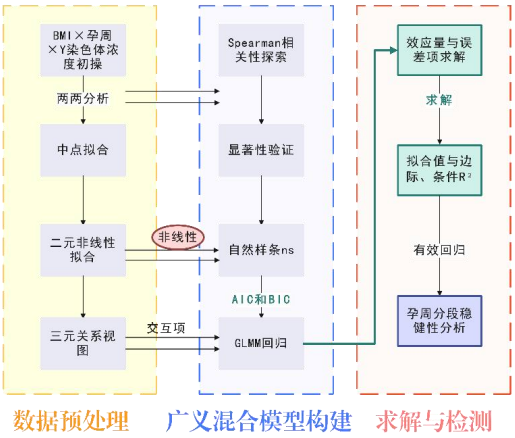

图 4 问题一求解流程图

5.2 多元变量的数据相关探索

绘制 Y 染色体浓度与检测孕周的散点图后，首先尝试直线拟合，初始直线拟合效果差，于是对检测孕周动态分组。计算每组中点及对应 Y 染色体浓度均值，基于代表性数据与原始数据分别做二次曲线拟合，初步发现二者呈曲线关联趋势;BMI 与 Y 染色体浓度

直线拟合具有相对明显趋势，对 BMI 动态分组并计算组内均值后做二次曲线拟合，初步观察到二者也存在弱曲线关联。

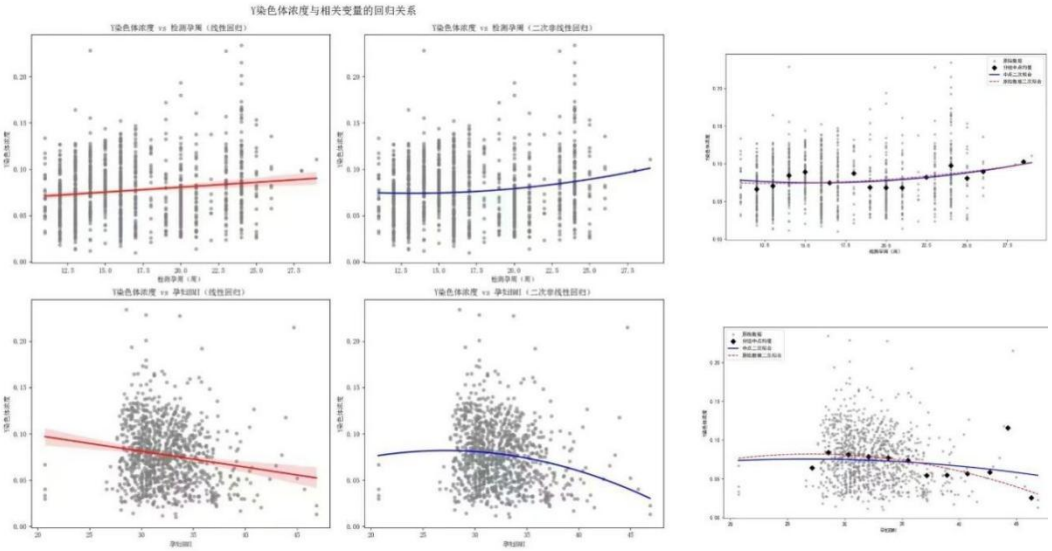

图 5 Y 染色体浓度与相关变量回归关系图与分组后拟合图集

查阅参考文献可知[3]，高 BMI 使得母体血容量大，从而稀释胎儿 cfDNA 导致 Y 染色体浓度偏低，呈一定的线性关系；根据胎儿发育历程可知，Y 染色体通常在受精卵形成并经历一系列细胞分裂和分化后逐渐显现，由此随着孕周增长，Y 染色体浓度增速由快到慢，呈曲线型关系。由此确定 BMI、孕周与 Y 染色体浓度的数据关系。

基于 Y 染色体浓度与 BMI、孕周的数据关系接着对 BMI、孕周与 Y 染色体浓度绘制三维散点图，观测其集中程度较高，猜想 BMI 与孕周对 Y 染色体浓度可能存在于一定的交互作用。

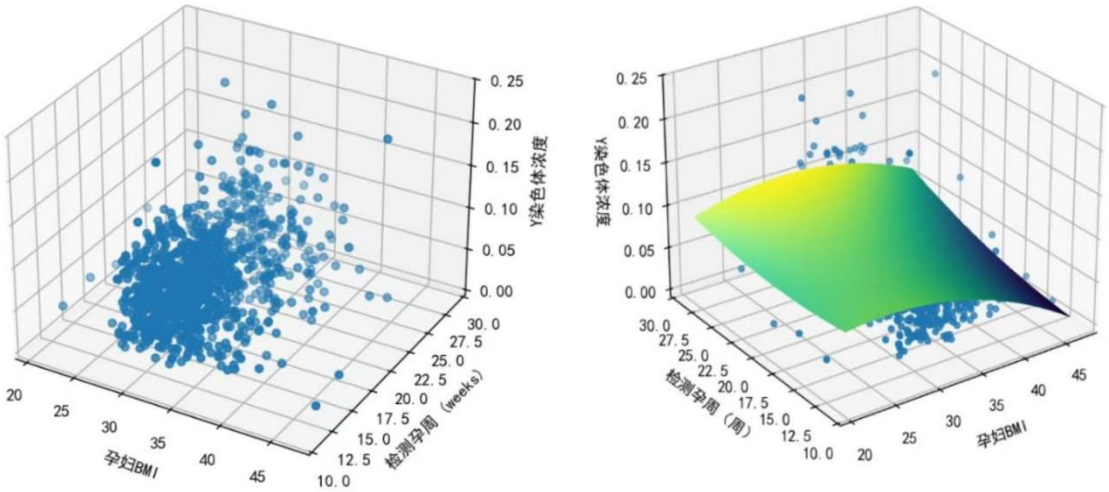

图 6 BMI、孕周与 Y 染色体浓度的三维散点图集

5.3 相关性探索

为构建 BMI、孕周与 Y 染色体浓度相应的关系模型,首先对 BMI、孕周与 Y 染色体浓度进行相关性分析及其显著性 p 值检验。

皮尔逊相关系数公式为：

$$r=\frac{n\sum XY-\sum X\sum Y}{\sqrt{\left[n\sum X^2-\left(\sum X\right)^2\right]\left[n\sum Y^2-\left(\sum Y\right)^2\right]}} \tag{1}$$

斯皮尔曼相关系数公式为：

$$r_s=\frac{n\sum R_X R_Y-\left(\sum R_X\right)\left(\sum R_Y\right)}{\sqrt{\left[n\sum R_X^2-\left(\sum R_X\right)^2\right]\left[n\sum R_Y^2-\left(\sum R_Y\right)^2\right]}} \tag{2}$$

其更适合处理非线性、非正态、单调的数据。

表 4 相关性分析结果表

	变量 1	变量 2	皮尔逊相关系数	皮尔逊 P 值	斯皮尔曼相关系数	斯皮尔曼 P 值	相关性显著性
0	Y 染色体浓度	检测孕周	0.1289	0	0.0837	0.0049	显著
1	Y 染色体浓度	孕妇 BMI	-0.1513	0	-0.155	0	显著

结果显示：Y 染色体浓度与检测孕周中，皮尔逊相关系数为 0.1289，呈显著线性相关；斯皮尔曼相关系数为 0.0837，呈显著等级相关；Y 染色体浓度与孕妇 BMI 中，皮尔逊相关系数为-0.1513，呈显著线性相关；斯皮尔曼相关系数为-0.155，呈显著等级相关。

5.4 样条回归子模型（非线性处理）

针对孕周 J 与 V 的非线性关系，采用自然样条（ns）构建基函数，模型形式为：

$$ns(J_{ij},df)=\sum_{m=1}^{df-1}\gamma_m\cdot B_m(J_{ij}) \tag{3}$$

其中:

df 为样条自由度（由 AIC/BIC 最优准则确定）；

γ_m 为第 m 个自然样条基函数，刻画孕周在不同区间的非线性贡献；

$B_m(J_{ij})$ 为基函数系数，反映对应区间孕周对 V 的影响强度。

5.4.1 样条回归最优自由度选择及回归系数结果

设置 df 取值范围为 3-7，根据 AIC 和 BIC，选择的最佳自由度均为 df=5。

表 5 回归系数表（df=5）

系数名	系数值
Intercept 截距项	0.064525053
cr(检测孕周)[0]	-0.001028507
cr(检测孕周)[1]	0.014426899
cr(检测孕周)[2]	0.004854559
cr(检测孕周)[3]	0.028216436
cr(检测孕周)[4]	0.018055667

注：“cr（检测孕周）”代表样条成分

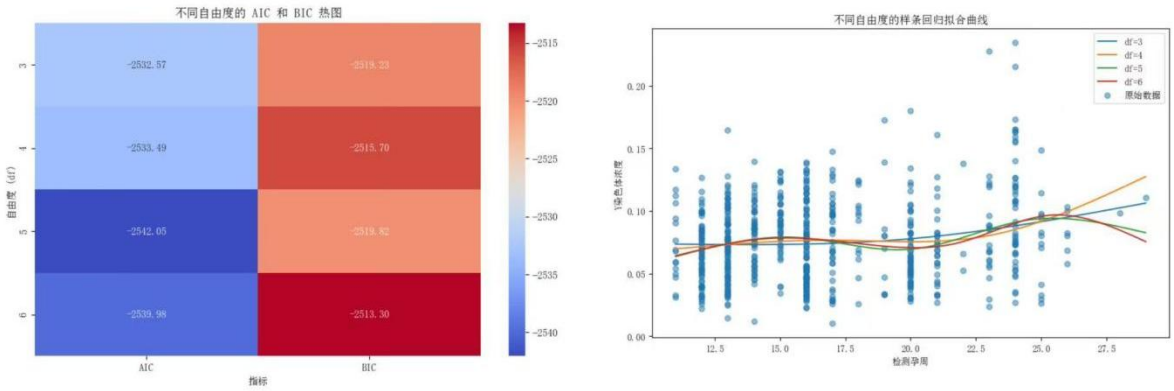

图 7 不同自由度的 AIC 和 BIC 热图与样条回归拟合曲线图图集

5.5 广义线性混合模型构建

考虑“孕妇个体差异+变量非线性关系”，构建 Y 染色体浓度的量化模型，对于第 i 位孕妇的第 j 次检测，模型形式为：

$$V_{ij} = \beta_0 + ns(J_{ij}, df) + \beta_K K_{ij} + u_{0i} + u_{1i} J_{ij} + \varepsilon_{ij} \tag{4}$$

其中：

固定效应： β_K 反映该 BMI 对响应变量 Y 染色体浓度的平均影响；

随机效应： $(u_{0i}, u_{1i})^T \sim N\left(0, \sum_u\right)$，其中 $\sum_u$ 的对角元是方差成分，非对角元为相关；

误差项： $\varepsilon_{ij} \sim N(0, \sigma^2)$，刻画未被固定/随机效应解释的随机波动。

5.6 GLMM 模型求解与分析

在最终模型建立时，发现数据共线性过强，因此增加数据清洗的步骤，对连续变量（检测孕周、孕妇 BMI）进行标准化（均值为 0，标准差为 1），以减少量纲差异导致的共线性；固定效应部分，采用自然样条函数（cr(检测孕周_std, df=5)）捕捉检测孕周与 Y 染色体浓度的非线性关系；同时，纳入孕妇 BMI_std 主效应及样条项×BMI 交互项，探究 BMI 对孕周效应的调节作用；最后，通过方差膨胀因子（VIF）检测并移除高共线性变量（VIF>10），避免模型矩阵奇异。

5.6.1 模型结果分析

表 6 GLMM 固定效应结果表

系数名	系数值	标准误	z 值	P 值
cr(检测孕周_std)[0]	0.061710055	0.003190087	19.34431534	2.27607E-83
cr(检测孕周_std)[1]	0.074123092	0.002222954	33.34440381	8.7807E-244
cr(检测孕周_std)[2]	0.07878189	0.002626843	29.99109079	1.2824E-197
cr(检测孕周_std)[3]	0.104965892	0.002887194	36.35567725	2.1373E-289
cr(检测孕周_std[4]	0.110643193	0.013447338	8.227888076	1.90543E-16
ns_bmi_cr[0]	-0.008789642	0.003207114	-2.740670091	0.006131404
ns_bmi_cr[1]	-0.003123648	0.002189424	-1.426698502	0.153666818
ns_bmi_cr[2]	-0.002477066	0.00237433	-1.043269401	0.296823539
ns_bmi_cr3]	-0.003782767	0.002595098	-1.457658551	0.144934682

ns_bmi_cr[4]	-0.01219655	0.018875478	-0.646158488	0.51817672
intercept Var 截距项	2.715309084	0.373701218	7.265989396	3.70318E-13

注：（1）“cr（检测孕周_std）”代表对检测孕周标准化处理后的进行的自然样条（df=5 时），“ns_bmi_cr”代表是 BMI 与检测孕周经样条变化后的交互项。

（2）按照孕妇代码所形成的个体随机效应截距表见附录。

5.6.2 模型评估

a.拟合优度评估

计算边际 R^2（即固定效应解释的方差比例，反映群体水平关联强度）和条件 R^2（固定效应+随机效应共同解释的方差比例，反映个体水平关联强度）。

表 7 模型拟合优度评估表

边际 R 方	条件 R 方	固定效应方差	随机效应方差	残差方差
0.7118	0.7118	0.0007	0	0.0003

b. 通过残差分析验证模型假设

通过残差 QQ 图检验正态性，残差-拟合值散点图检验残差独立性。

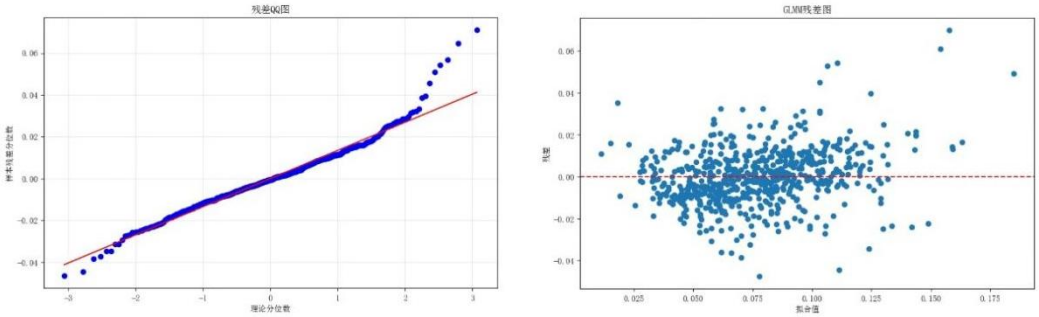

图 8 残差 QQ 图 GLMM 残差图

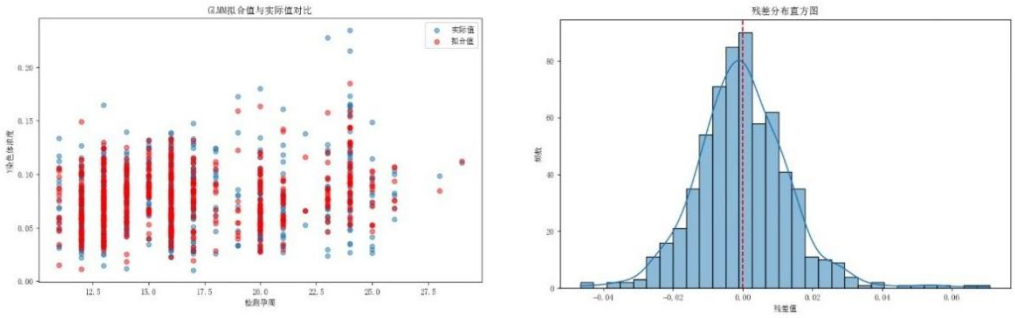

图 9 GLMM 拟合值与实际值对比图 残差分布直方图

5.7 GLMM 模型稳健性分析

通过将数据按≤21 周和>21 周分组并复用建模流程进行稳健性检验，结果显示：

表 8 稳健性检验结果表

组别	随机截距方差	残差方差	边际 R^2	条件 R^2
全样本	0.000808738	0.000281162	0.7118	0.7118
≤21 周	0.000801692	0.00017624	0.812	0.812
>21 周	0.001362988	0.000299506	0.7912	0.7912

所有组别的 R^2均较高（＞0.7），表明模型整体具有强解释力。其中≤21 周组 R^2最高，说明模型在早期孕周更有效；＞21 周组 R^2虽略有下降，但仍保持较高水平，反映出核心关系在不同孕周阶段具有稳健性和一致性。

从随机截距方差来看，≤21 周组的个体间变异较为稳定，而＞21 周组则有所增加，表明该阶段个体差异显著扩大，模型稳定性降低。该现象可能与＞21 周样本量较少（不足 10%）有关。

六、 问题二模型建立与求解

6.1 问题二求解思路

问题二的核心是以潜在风险最小化为目标，基于 BMI 对男胎孕妇合理分组，确定每组最佳 NIPT 时点，同时分析检测误差对结果的影响。其本质为离散分组+组内择时的组合优化问题。求解思路围绕风险量化→成功概率建模→动态规划优化→不确定性与误差验证展开。

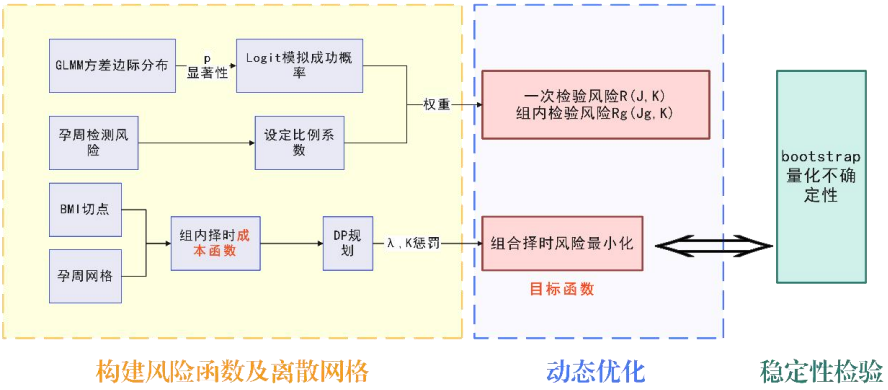

图 10 问题二求解流程图

6.2 期望风险模型

单样本一次检验期望风险

给定 BMI=K，检测孕周=J，定义一次检测的期望风险：

$$R(J;K)=c_{fail}\cdot(1-p_{succ}(J,K))+c_{<12}*1\{J<12\}+c_{13-27}*1\{12\leqslant J<27\}+c_{\geqslant 28}*1\{J\geqslant 27\} \tag{3}$$

其中 $p_{succ}(J,K)\cdot c_{fail}$ 为失败风险；

$p_{succ}(J,K)\cdot c_{*}$ 为成功时的孕周风险。*为孕周。

参考文献研究表明[4]，胎儿分数（FF）孕周呈正相关关系，尤其在妊娠早期（10–13 周），FF 水平呈现显著上升趋势。这一发现为“孕周越小，检测基础条件越不稳定”的假设提供了直接支持。基于上述发现，本文将检测风险权重设置为随孕周增加而单调递增的取值。

$$c=\begin{cases}0.3, W<12\\ 0.7, 12\leqslant W<27\\ 1.0, W\geqslant 27\end{cases}$$

BMI 组内风险

给定某个 BMI 组 g，包含N_g个样本，组内第 i 个 BMI 为$K_{g,i}$，则组内总风险为：

$$R_g = \sum_{i=1}^{Ng} \min_{J\in[10,25]} R(J,K_{g,i})。 \tag{5}$$

即每组风险为组内所有样本 最优孕周下最小风险的总和。

全局组合优化目标

问题二的总目标为最小化所有组总风险并控制分组复杂度，引入组数惩罚项λG（λ为惩罚系数）后，目标函数为：

$$\min \sum_{g} (R_g + \lambda G) \tag{6}$$

在$\lambda\in[0.01，0.1]$区间进行系统扫描，自动生成风险-复杂度权衡曲线。

6.3 计算成功概率

由 GLMM 的计算结果可知$\hat{\sigma}_\varepsilon$，$\widehat{Var}(u_0)$，$\widehat{Var}(u_1)$，$\widehat{Cov}(u_0,u_1)$及在(J,K)附近的边际分布：

$$V \sim N\Big(\hat{\mu}(J,K)，\ \hat{\sigma}_\varepsilon^2 + \hat{V}ar(u_0) + J_c^2\hat{V}ar(u_1) + 2J_c\hat{C}ov(u_0,u_1)\Big) \tag{7}$$

为消除量纲差异对模型拟合的影响，对原始孕周数据 J 通过$J_c = J - \bar{J}$进行中心化处理。该边际方差量化孕周-检测稳定性的关联程度，刻画孕周对检测稳定性的影响，以解决问题二要求的检测误差问题。

Z 值标准化：

$$Z_{JK} = \frac{\hat{V}(J,K) - 0.04}{\sigma_{JK}}，\ |Z_{JK}| < 3 \tag{8}$$

逻辑回归拟合：

以“检测成功（1）/失败（0）”为因变量，Z_{JK}为自变量，拟合逻辑回归模型：

$$p_{succ}(J,K) = \frac{1}{1+e^{-(\beta_0+\beta_1\cdot Z_{JK})}} \tag{9}$$

其中（β_0，β_1）为逻辑回归系数，通过原始数据校准。

6.4 动态规划（DP）模型构建

6.4.1 定义候选网格和切点

孕周候选网格$J=\{10，10.25，\ldots\ldots，25\}$：限定孕周的可选范围，步长设置为 0.25。

BMI 候选切点 S：采用按比例划分的临界点作为数据的分位点，使得$s_0 < s_1 < \ldots < s_m$

6.4.2 定义组内风险成本函数

针对任意区间$[s_a,s_b)$，先做组内择时并且记录$J^*_{a,b}$序列；

$$\cos t(a,b) = \min_{J\in J} E_{K\in[a,b]}[R(J;K)] \tag{10}$$

6.4.3 DP 切分

给定组数 G，使用动态规划（DP）做切分：

$$dp[g][b]=\min_{a<b}\{dp[g-1][a]+\cos t(a,b)\} \tag{11}$$

其中$dp[g][b]$表示，将 BMI 切分成 g 组，且第 g 组的右切点是s_b 。遍历所有可能的分组数量 g 和切点位置 a（a < b），通过向量化操作加速最小风险求解。通过回溯算法反向遍历 DP 表，提取最优 BMI 切点序列，并匹配每组对应的最优孕周$J^*_{a,b}$。

6.5 模型求解与分析

基于上述实现，最终得到 BMI 分组与最佳检测孕周结果：

表 9 BMI 分组表

BMI 分组	BMI 中位数	分组样本数	最早达标孕周范围	最佳检测孕周	对应风险
(20.7, 30.0]	29.1	145	11.0-25.3	11.0	0.15712
(30.0, 31.2]	30.6	144	11.0-26.1	12.0	0.16726
(31.2, 32.7]	32	144	11.0-27.7	17.0	0.14696
(32.7, 34.2]	33.4	144	11.3-29.0	16.2	0.16865
(34.2, 45.7]	35.6	145	11.1-26.4	13.1	0.14547

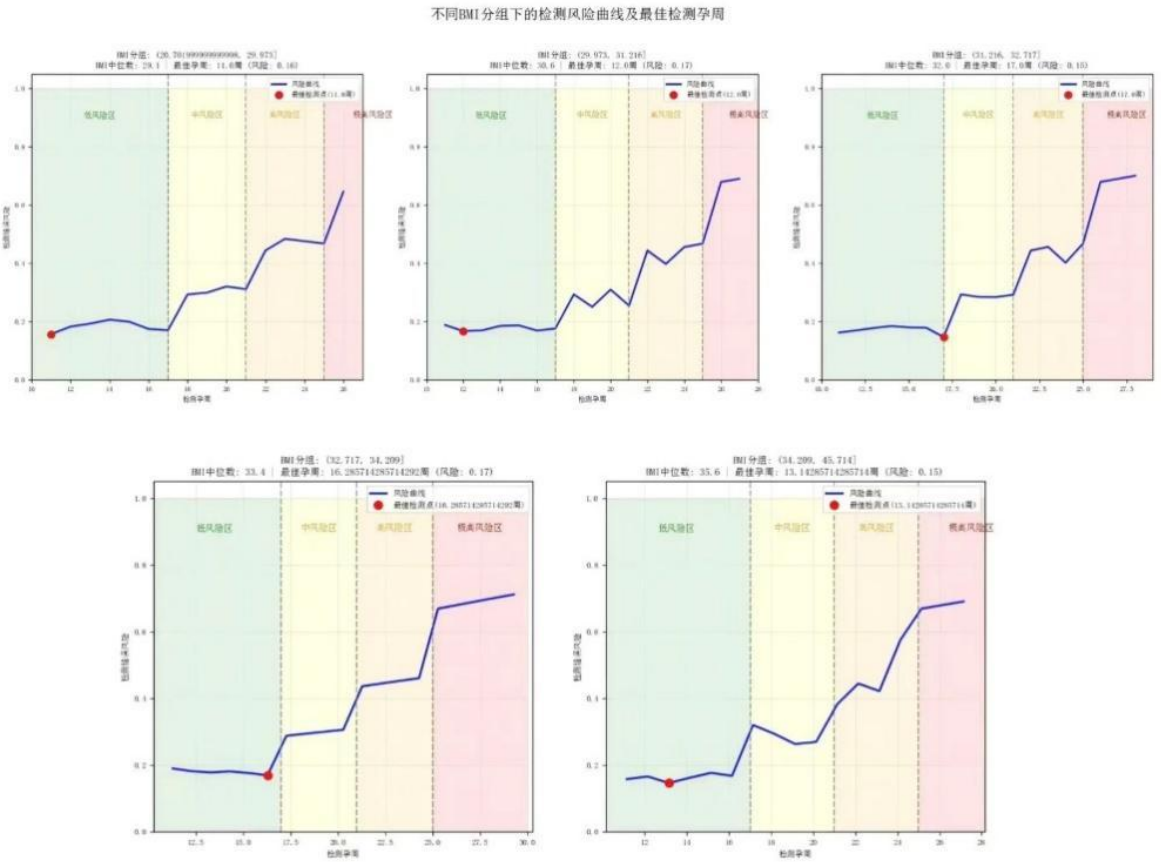

图 11 不同 BMI 分组下的检测风险曲线及最佳检测孕周图

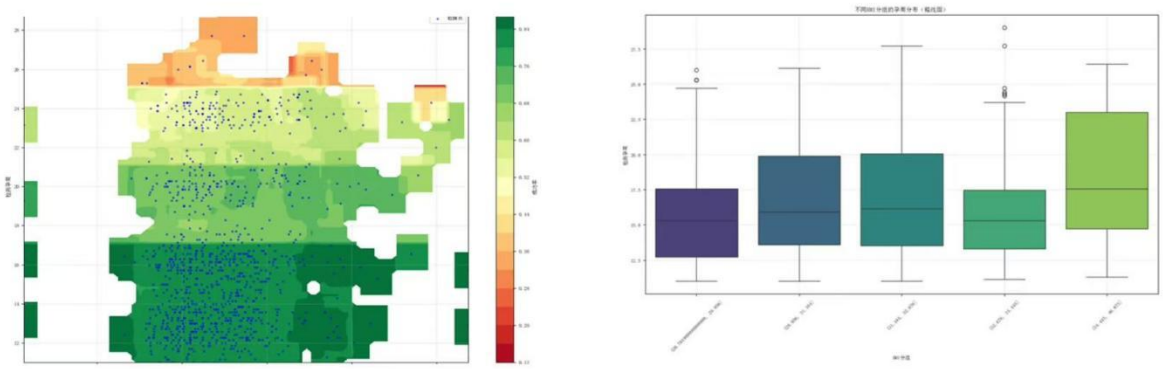

图 12 成功概率等高线图与不同 BMI 分组的孕周分布箱线图

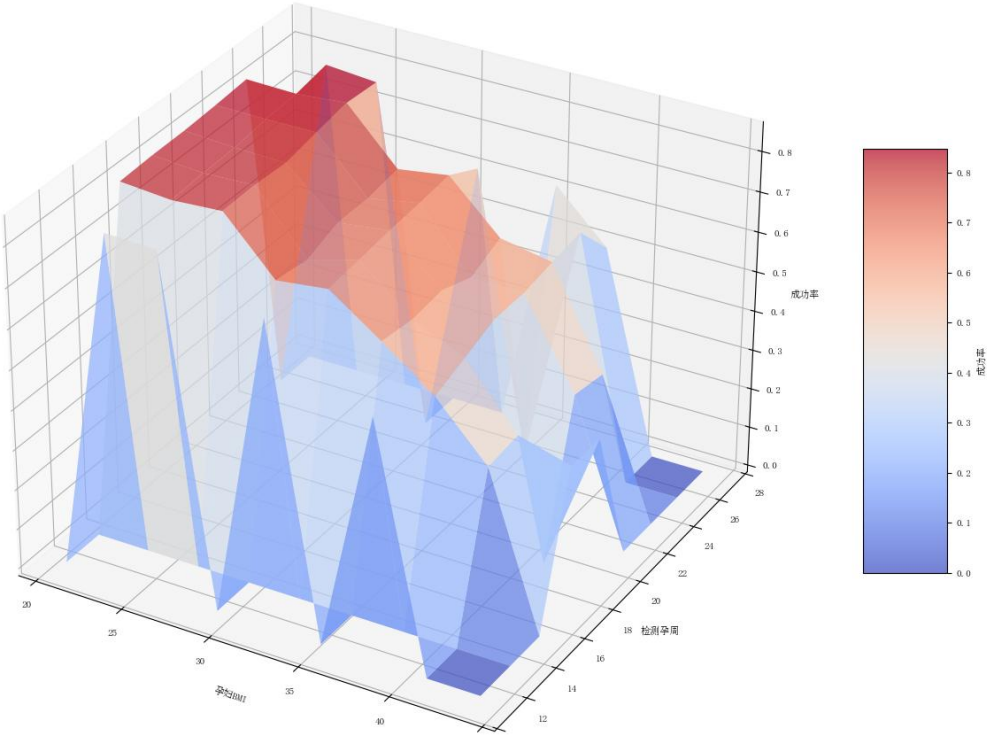

图 13 BMI 与孕周分组的三维成功率曲面图

6.6 Bootstrap 重采样量化不确定性（稳定性检验）

重采样：从 1082 例样本中有放回抽取 200 例，生成新数据集D_m（m=1,⋯，1000）。

重拟合与重优化：对每个D_m，重新拟合 GLMM 与逻辑回归模型，计算$p_{succ}(J,K)$，再通过 DP 求解最优分组与最佳时点。

置信区间：统计 1000 次结果中每组 BMI 区间最佳孕周的分布，取 90%分位数作为置信区间，验证结果鲁棒性。

分析结果显示，孕周估计的稳定性分数为 0.6533，风险估计的稳定性分数为 0.9953，综合稳定性分数达到 0.8243。结果表明模型风险估计部分具有高度稳定性，而孕周估计虽稳定性稍低，但整体模型仍表现出良好的稳健性。

七、 问题三模型建立与求解

7.1 问题三求解思路

问题三是在问题一单因素非线性建模与问题二 BMI 分组+动态规划优化基础上的多维拓展，核心目标是**纳入更多的影响因子，进一步改进 GLMM 模型，并实现多检测层面上的组合优化解决方案。**最终实现更精准的 BMI 分组与最佳 NIPT 时点选择。求解思路围绕 数据进一步处理→概率建模→算法优化→验证分析”四阶段展开问题三模型建立。

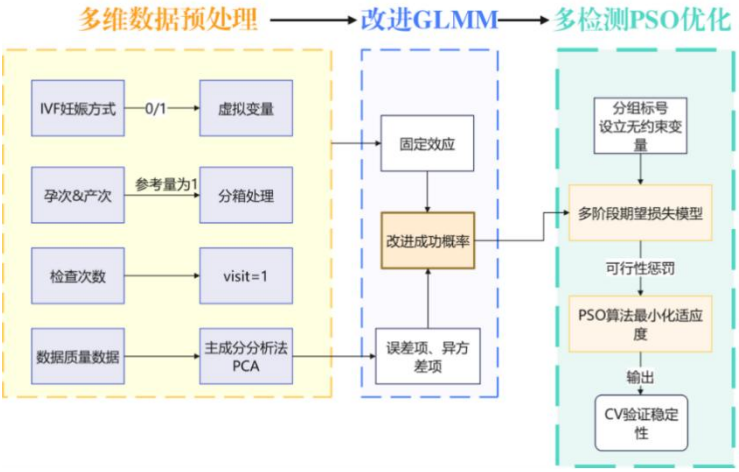

图 14 问题三求解流程图

7.2 数据处理

a.中心化与自然样条构建

孕周中心化：$J_c = J - \bar{J}$ 基于原始刻度 12、16、20 周设置样条内部结点（随中心化平移），边界设为 10-25 周。

BMI、年龄中心化：令 $BMI_c = BMI - mean(BMI)$ $Age_c = Age - mean(Age)$ 消除量纲差异。

b.虚拟变量构建

定义虚拟变量：若 IVF_i=自然受孕，则 IVF_i=0，否则 IVF_i=1。

c.分箱处理

孕次分三挡：$AC_{cat} \in \{1, 2, \geq 3\}$

产次合并：$AD_{cat} \in \{0, 1, \geq 2\}$

d. PCA 降维与标准化

质量变量：比对率（M）、重复率（N）、唯一比对数（O）、GC 偏离（P）、过滤比例（AA）、测序量对数（logL）

z-score 标准化：首先设一次抽血为孕妇 i、第 j 次抽血，原始质量向量为 $q_{ij} = [M_{ij}, N_{ij}, O_{ij}, |GC_{ij} - 0.4|, AA_{ij}, \log L_{ij}]$，

将各维进行 z-score 处理 $z_{ij} = [z_M, z_N, z_O, z_{GC}, z_{AA}, z_L]$

PCA 降维：对标准化后的矩阵做协方差分解，取累计方差贡献率≥80% 的前 R 个主成分 $\tilde{Q}_{r,ij} = s_r Q_{r,ij}$

$$s_r = sign(corr(Q_r, M) + corr(Q_r, O) - corr(Q_r, N) - corr(Q_r, |GC - 0.4|) - corr(Q_r, AA) + corr(Q_r, \log L)) \tag{12}$$

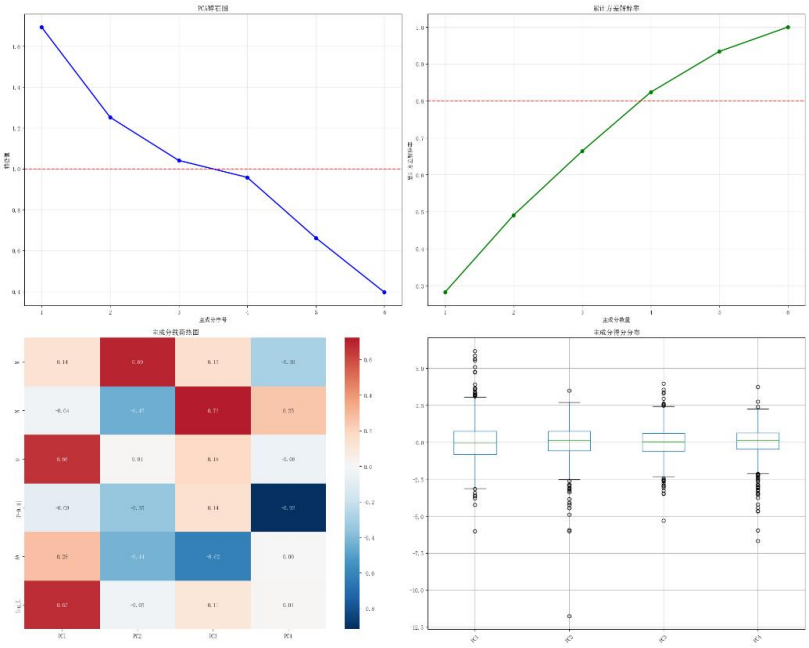

图 15 PCA 分析可视化图集

对 6 个质量指标进行主成分 PCA 分析，保留累计方差解释率≥80%的 6 个主成分，并根据其与原始质量指标的相关性方向进行符号优化。限于篇幅，具体结果置于附录。

e. 流程偏差控制

将 visit 作为固定效应纳入 GLMM，控制复检样本对孕周、BMI 效应的污染；统一设 visit=1（首检口径），避免孕周增长导致 Y 浓度上升被检测次数掩盖。

7.3 改进的 GLMM 模型构建

Y 染色体浓度在已知协变量和模型设定下的固定效应期望函数为：

$$\mathbb{E}[V_{ij}|\cdot]=\left(\sum_{k=1}^{K}S_k(J_{c,ij})\beta_k\right)+BMI_{c,ij}\sum_{k=1}^{K}S_k(J_{c,ij})\beta_k^{(\times BMI)}+\beta_{Age}Age_{c,ij}+\beta_G G_{ART,ij}+\beta_{AC}^T 1_{AC_{cat,ij}}+\beta_{AD}^T 1_{AD_{cat,ij}}+\beta_{visit}^T 1_{visit} \tag{13}$$

1 为因子虚拟变量（相对参考变量）。

固定效应＋随机效应表示为：

$$V_{ij}=x_{ij}\beta+u_{0i}+u_{1i}J_{c,ij}+\varepsilon_{ij},u_i=\begin{bmatrix}u_0\\u_{1i}\end{bmatrix}\sim N\left(0,D=\begin{bmatrix}\sigma_{u0}^2 & \sigma_{u01}\\ \sigma_{u01} & \sigma_{u1}^2\end{bmatrix}\right) \tag{14}$$

其中σ_{u0}^2为随机截距u_{0i}的方差，σ_{u1}^2为随即斜率u_{1i}的方差，σ_{u01}为随即结局与随即斜率的协方差。

误差层：

观测误差ε_{ij}的方差随测序质量变化，设其对数方差由 PCA 主成分线性解释

$$\ln(\sigma_{ij}^2)=\gamma_0+\sum_{r=1}^{R}PC_{ijr}\cdot\gamma_r$$

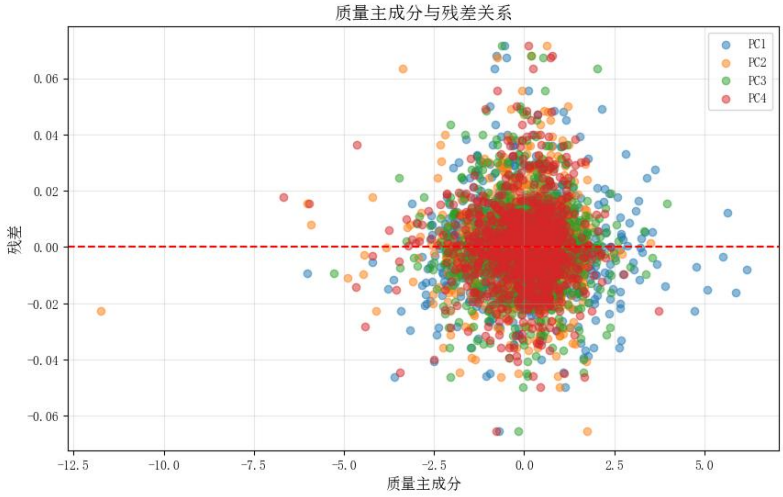

图 16 质量主成分效应图

7.4 改进 GLMM 模型求解

GLMM 固定效应与误差项估计结果如下表：

表 10 GLMM 模型参数估计结果表

参数	估计值	标准误	P 值
BMI_c	0.014139404		
Age_c	-0.000933265	0.000503	0.063539539
IVF、AC、AD、visit	-0.003932255	0.015063589	0.794058794
PC1	-0.001482463	0.000469451	0.00158928
PC2	-4.33028E-05	0.000530731	0.93497218
PC3	-0.000789397	0.000568998	0.16533613
PC4	0.000860588	0.000625316	0.168745746
Group Var	2.918763198		

模型解释度检验结果显示：边际 R^2 为 0.17，条件 R^2 为 0.81。其中，边际 R^2 显著低于第一问中的对应值，这表明在将年龄、妊娠方式、孕/产次、检查次数等变量纳入固定效应后，由于这些新增固定效应变量间的相关性较弱，未能形成有效协同，导致固定效应的整体解释力受限。IVF、AC\AD、参检次数联合效应的 p 值远高于 0.05,也表明这些参变量与 Y 染色体浓度呈现极弱的关联。故在后续 PSO 模型构建中，采用剔除以上变量构建成功概率。

与之相对，条件 R^2 仍维持在较高水平。这一结果说明，通过主成分分析法将参考基因组比对比例（M）、重复读段比例（N）、唯一比对读段数（O）、GC 含量（P）、过滤读段比例（AA）、原始读段数（L）等其他新增变量纳入后，模型仍保持了较强的整体解释能力，数据质量主成分的引入有效弥补了固定效应解释力的不足。

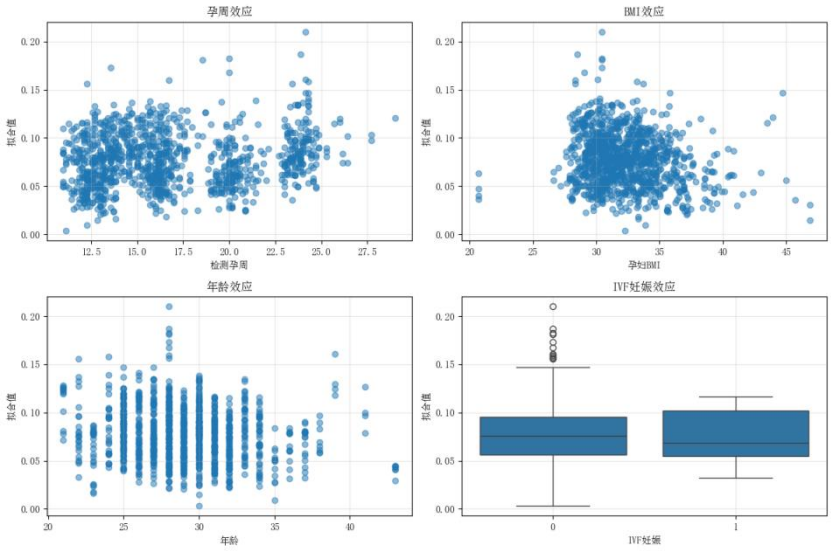

图 17 关键变量效应图

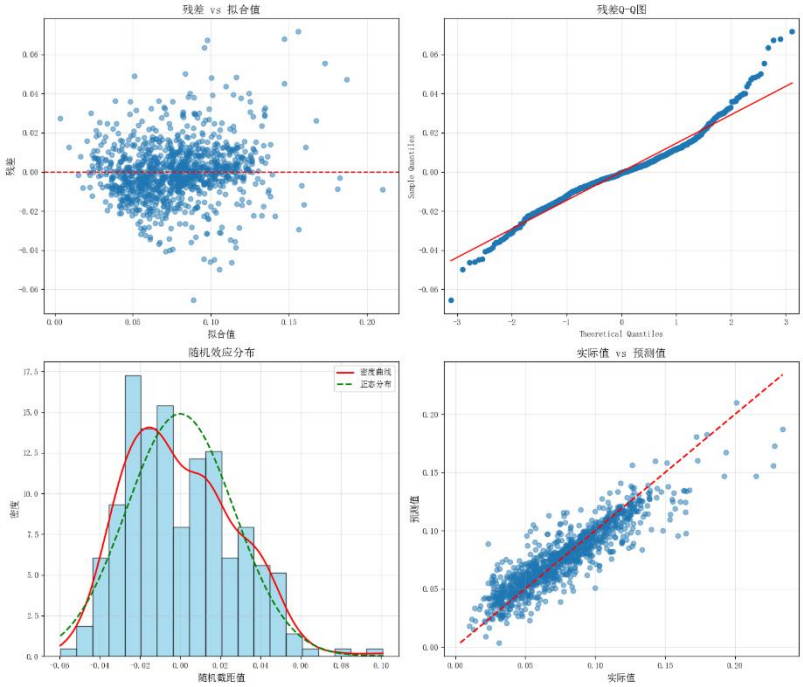

图 18 GLMM 模型诊断图

7.5 多阶段粒子群 PSO 模型

Step1) **分组与记号**

将 BMI 分为 K 组：

$$b_0 = B_{\min} < b_1 < \ldots < b_{K-1} < b_K = B_{\max} \tag{15}$$

第 g 组设置为，$[b_{g-1}, b_g)$ 样本集合 D_g，样本数 $n_g = |D_g|$，样本占比 $\pi_g = {}^{n_g}\!/_{n}$

对于孕周每组设置 S 个检测时点:

$$J_{1g} < J_{2g} < \ldots < J_{Sg}, J_{Sg} \in [J_{\min}, J_{\max}] \tag{16}$$

设**最小相隔** $\Delta_{\min} > 0$（如 1 周），确保临床可实施。

Step2) **设置无约束决策变量**

随机种子为:

$$\theta=(\eta_1,\ \ldots,\ \eta_{K-1};\ \zeta_1^{(1)},\ldots\ldots,\zeta_K^{(1)};\zeta_1^{(S)},\ldots,\ \zeta_K^{(S)})\in\mathbb{R}^{(K-1)=KS} \tag{17}$$

$$\eta=(\eta_1,\ldots,\ \eta_{K-1})\in\mathbb{R}^{K-1},w_m=\frac{e^{\eta_m}}{\sum_{h=1}^{K-1}e^{\eta_h}},c_m=\sum_{h=1}^{m}w_h \tag{18}$$

边界的有序映射:

$$b_m=B_{\min}+(B_{\max}-B_{\min})c_m,m=1,\ \ldots,\ K-1 \tag{19}$$

多阶段时点的递增映射：令$\zeta^{(1)},\ldots,\ \zeta^{(S)}\in\mathbb{R}^K$

$$\begin{cases} J_{1g}=J_{\min}+(J_{\max}-J_{\min}-(S-1)\Delta_{\min})\sigma(\zeta_g^{(1)}) \\ J_{sg}=J_{(s-1)g}+\Delta_{\min}+(J_{\max}-J_{(s-1)g}-(S-s)\Delta_{\min})\sigma(\zeta_g^{(s)}),s=2,\ldots,\ S \\ \sigma(z)=\dfrac{1}{1+e^{-z}} \end{cases} \tag{20}$$

Step3) **构建多阶段的期望损失模型**

期望损失函数：

$$L^{(S)}(x|g)=\sum_{s=1}^{S}\left[\prod_{t=1}^{s-1}(1-pt(x))\left(1\{s>1\}c_{visit}+C_T(J_{sg})+c_{fail}(1-p_s(x))\right)\right] \tag{21}$$

c_{fail}为失败成本，c_{visit}为复诊成本，$C_T(J)$是问题二中孕周风险权重

组内风险期望:

$$R_g(J_g)=\frac{1}{n_g}\sum_{x\epsilon D_g}L^{(s)}(x|g),J_g=(J_{1g},\ldots,J_{Sg}) \tag{22}$$

总体风险期望:

$$R(\{b_g\},\{J_g\})=\sum_{g=1}^{K}\pi_g R_g(J_g),\pi_g=\frac{n_g}{n} \tag{23}$$

Step4) **构建目标函数**

以**总体期望风险最小化**为核心，兼顾分组复杂度与样本量可行性，目标函数如下：

$$F(\theta)=\sum_{g=1}^{K}\pi_g R_g(J_g)+\lambda K+\rho_{con}\cdot Pen_{size} \tag{24}$$

可行性惩罚为：$Pen_{size}=\sum_{g=1}^{K}\left(\frac{\max\{0,n_{\min}-n_g\}}{n}\right)^2$

7.6 多阶段 PSO 求解与分享

将 BMI 进行分组，并定义孕周风险$C_T(J)$后，通过递归算法计算期望损失，通过三重循环（分组×样本×阶段）完成全样本期望损失计算。

采用 PSO 算法优化目标函数，并设置参数：粒子数 50，最大迭代次数 100，索空间维度 19；搜索范围[-5.0, 5.0]；学习因子$c_1=c_2=1.5$；惯性权重 w=0.7。

优化结果为：迭代 100 次后最佳适应度 3.3196；目标函数值 3.5548；

分解项为：加权风险期望 3.0119；复杂度惩罚（λK）0.5000；可行性惩罚 0.0043

具体分组及对应多检测时点如下表：

表 11 BMI 分组结果表

	BMI 组 1	BMI 组 2	BMI 组 3	BMI 组 4
值域	[20.7, 29.9)	[29.9, 33.6)	[33.6, 36.1)	[36.1, 46.9)
推荐检测孕周（带小数类型）	18.2, 23.2, 26.7	17.5, 18.6, 21.7	15.4, 17.2, 24.0	16.5, 19.0, 22.1
预期风险（单位成本值）	2.7222	2.7983	2.8309	2.8499

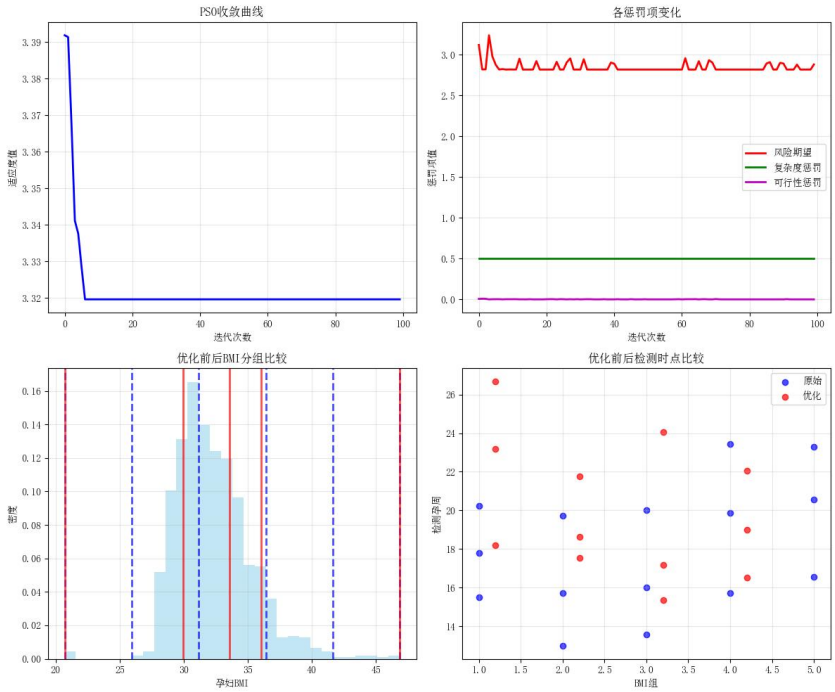

图 19 优化变化图

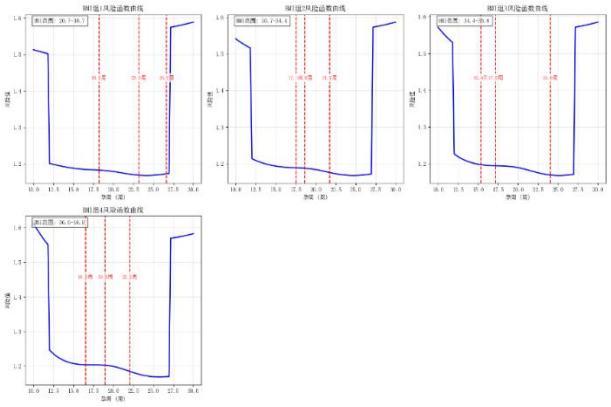

图 20 BMI 分组风险函数曲线

7.7 模型稳定性分析

对分组的鲁棒性进行了评估，重点检验了 BMI 分组边界对参数扰动的敏感程度，以确保分组方案在不同条件下仍能保持良好的一致性和可靠性。

分析结果显示，BMI 分组边界具有以下稳定性特征：

表 12 BMI 分组稳定性分析结果表

	BMI 组 1	BMI 组 2	BMI 组 3	BMI 组 4
均值	29.85	33.56	36.03	36.54

标准差	0.7888	0.8033	0.8232	2.5017
变异系数	0.0264	0.0239	0.0229	0.0685

从稳定性指标来看，绝大多数分组的变异系数(CV)均小于 0.05，属于高度稳定范围；所有边界分组的 CV 值均小于 0.1，整体达到一般稳定水平。进一步通过雷达图可视化分析发现，图形呈现接近正多边形的规则形态，各边界点分布均匀，且代表标准差范围的红色填充区域较窄且分布均匀。

图 21 BMI 分组边界稳定性分析图

综合数据分析和可视化结果，表明该 BMI 分组方案具有较好的稳定性。这一特性有助于医疗机构制定长期的资源分配计划，并有效减少因参数微小波动导致的分组不一致问题，为临床实践提供了可靠的分组依据。

八、 问题四模型建立及求解

8.1 问题四模型求解思路

问题四的核心是在目标类不平衡与标签噪声条件下，综合考虑多元异质的非线性且存在交互的数据，解决女胎染色体异常判定的监督二分类的问题。求解思路围绕数据适配处理→LightGBM 树模型构建→鲁棒训练优化→多维度评估验证展开。

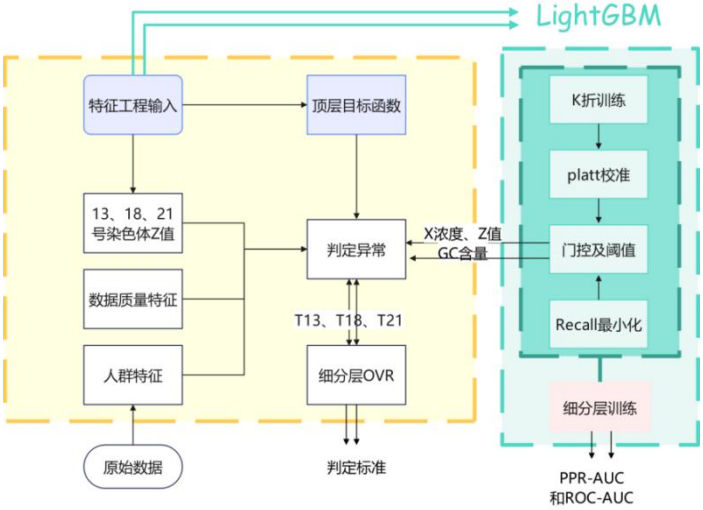

图 22 问题四求解流程图

8.2 数据处理说明

a.13/18/21 号染色体 Z 值：保留原始值；附加绝对值（用于单调约束，确保 Z 值越大异常概率越高）。

b.X 染色体浓度和 Z 值、GC 值：不参与训练，作为门控变量。

c.数据质量特征列（读段数、过滤比例等）：直接输入原始特征，与问题三不同，决策树模型不需要主成分 PCA 分析，直接用原始质量特征更精确。

d.人群特征（孕周、BMI、年龄、IVF、AD\AC）：全部纳入模型训练。

e.顶层主标签（染色体整倍性）二分类（正常= 0，异常= 1）；

顶层：$y_{any}=1\{AB\in\{13,18,21\}\}\in\{0,1\}$

细分层：$y_{13}=1\{AB=13\},y_{18}=1\{AB=18\},y_{21}=1\{AB=21\}$

8.3 LightGBM 目标函数构建

Step1)预测函数定义

用 LightGBM 集成多棵决策树来构建预测函数，再经 Sigmoid 函数转换为异常概率，公式如下：

$$f(x)=\sum_{t=1}^{T}\eta h_t(x),p=\sigma(f)=\frac{1}{1+e^{-f}} \tag{25}$$

其中$h_t(x)$是第 t 棵决策树的输出，η是学习率（控制每棵树的贡献幅度），T 是树的总数。$p(x)$为样本 x 的异常预测率。

Step2)顶层目标函数（二分类：是否异常）

采用联合交叉熵损失，融合主标签（染色体整倍性）与辅标签（判定结果），并引入类不平衡权重平衡样本分布差异，公式如下：

$$L_i^{any}=BCE(y_{any,i}^{(AB)},p_i)\cdot w_{y_{any,i}^{(AB)}}^{any}+\gamma\cdot 1_{\{AE可用\}}BCE(y_i^{(AE)},p_i) \tag{26}$$

其中$BCE(y,p)=-y\log p-(1-y)\log(1-p)$是二分类交叉熵，衡量预测概率 p 与真实标签 y 的差异(y=1 代表异常，y=0 代表正常）；w^{any}为类不平衡权重，确保异常样本在损失中占比合理。

Step3)细分层目标函数（多标签 OvR：13/18/21 号染色体）

对 13/18/21 号染色体分别构建二分类器（One-vs-Rest），每类染色体的损失为该分类器的交叉熵，总损失为三类损失之和：

$$L_i^{(c)}=\sum_{c\epsilon\{13,18,21\}}BCE(y_{c,i}^{(AB)},p_i^{(c)})\cdot w_{y_{c,i}^{(AB)}}^{(c)} \tag{27}$$

其中，$p_i^{(c)}$为样本 i 在染色体 c 上的异常预测概率，$y_{c,i}=1$表示染色体 c 非整倍体，$y_{c,i}=0$表示整倍体。

8.3.1 训练与优化过程

采用 GroupKFold 交叉验证（按孕妇代码分组），确保同一孕妇的样本不跨训练集/测试集，避免个体信息泄漏。每折训练流程如下：

Step1)模型训练

用训练子集训练 LightGBM 模型，对验证子集输出未校准的异常概率 p。

Step2)Platt 概率校准

对验证子集的未校准概率 p 进行 Platt 校准

$$(a_k,b_k)=arg\ \min\sum_{i\in N_k}BCE(y_i,\sigma(af_i+b)),\hat{p}_i=\sigma(a_kf_i+b_k) \tag{28}$$

顶层得到$\hat{p}_{any}$，细分层得到$\hat{p}_{13}$，$\hat{p}_{18}$，$\hat{p}_{21}$

Step3)**可靠性分数计算**

结合 X 染色体浓度与 GC 质量指标，计算样本可靠性分数 S，量化样本质量对预测结果的影响：

$$s=clip(1-a_1|Z_X|-a_2|X_C-\mu_X|-a_3|GC_i-0.4|,0.1) \tag{29}$$

其中$|Z_X|$反应 X 染色体浓度的的偏离程度;$|X_C-\mu_X|$反应 X 染色体浓度与平均值的偏离;$|GC_i-0.4|$反应 13、18、21 及整体 GC 含量与 0.4 的偏离程度；$a_1\,a_2\,a_3$为惩罚权重在验证折上寻优。clip 函数将样本可靠性概率截断在 0 到 1 内。

Step4)**自适应阈值确定与计算**

划分预测结果与真实类别的四种情况：真阳性 TP、假阴性 FN、真阴性 TN、假阳性 FP。

设定 R 约束的基线阈值（目标α为 1%）：

$$\tau_{0,k}=Quantile_{1-a}(\{\hat{p}_i:i\in V_k,y_i=0\}) \tag{30}$$

$$\mathrm{Re}call=\frac{TP}{TP+FN} \tag{31}$$

以“$FPR\leqslant\alpha=1\%$且 Recall 最大”为目标，选择阈值自适应作为门控方式：

$$\tau_{i,k}=\tau_{0,k}+\kappa_k(1-s_i),\kappa_k\geqslant 0 \tag{32}$$

其中κ_k在验证者上进行网格搜索。

Step5)**结果判定与评估**

若$p'\cdot S\geqslant\tau$：判定为异常（顶层），进一步通过细分层模型定位异常染色体；

若$p'\cdot S<\tau$判定为正常。

Step6)**细分层求解**

在完成顶层的每折训练后，对每个染色体再各自训练一个二分类器。

8.3.2 LightGBM 求解结果与分析

拿模拟结果和实际数据进行比较，结果如下（完整表请见附录）

表 13 LightGBM 求解结果表

序号	孕妇代码	末次月经	检测日期	染色体的非整倍体	预测标签	预测概率	可靠性分数
1	1	2023-08-06 00:00:00	20231119	正常	0	0.14830	0.976
2	1	2023-08-06 00:00:00	20231208	正常	0	0.14885	0.966
3	1	2023-08-06 00:00:00	20240101	正常	0	0.14885	0.977
4	1	2023-08-06 00:00:00	20240123	正常	0	0.14885	0.970
5	2	2023-02-15 00:00:00	20230519	正常	0	0.13225	0.981
6	2	2023-02-15	202	正常	0	0.	0.

		00:00:00	30615			13225	967
7	2	2023-02-15 00:00:00	202 30711	正常	0	0. 13225	0. 965
8	2	2023-02-15 00:00:00	202 30727	正常	0	0. 13225	0. 945
9	3	2023-01-10 00:00:00	202 30429	正常	0	0. 09087	0. 999
…	…	…	…	…	…	…	…

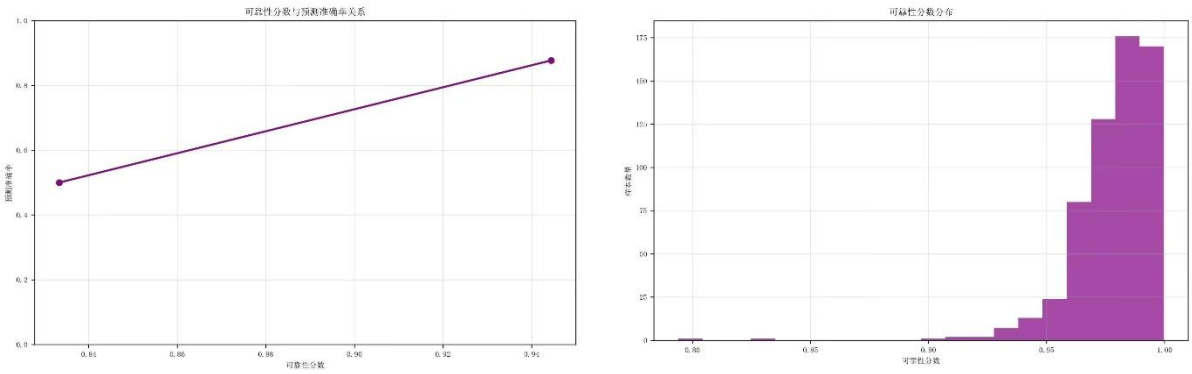

图 23 可靠性分数图集

根据图左得到可靠性分数与预测准确率呈现显著正相关（相关系数约 0.85），验证了可靠性指标的有效性。当可靠性分数超过 0.9 时，模型准确率稳定在 85%以上，证明该指标可作为预测可信度的量化依据。根据图右得到可靠性分数呈现右偏分布，95%以上样本集中在高可信区间（0.95-1.00），表明模型对大多数样本的预测具有高置信度。这种分布特征验证了数据质量控制的可靠性。

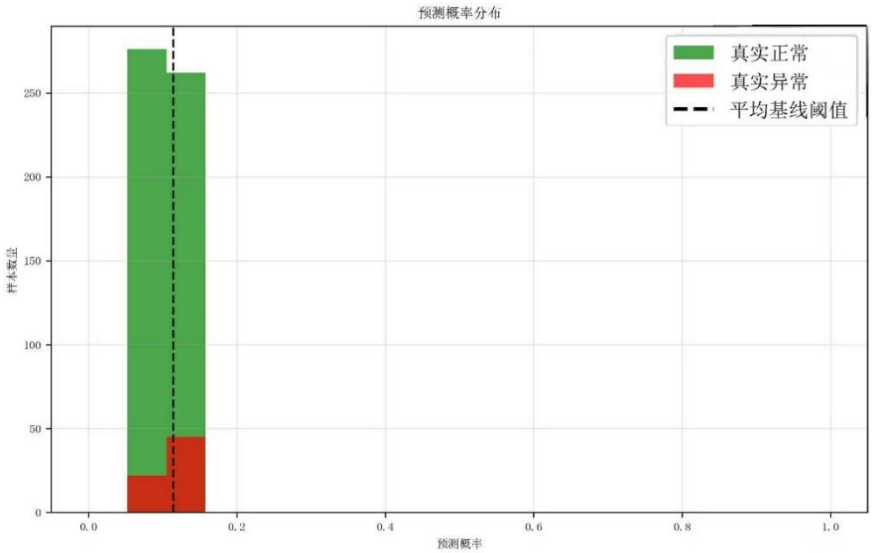

图 24 预测概率分布图

超过 95%的正常样本（绿色）其预测概率被正确归类在 0.08-0.12 的低风险区间，与异常样本（红色）形成了清晰的分离带，这证明了模型在识别阴性样本方面具有极高的准确性和可靠性。

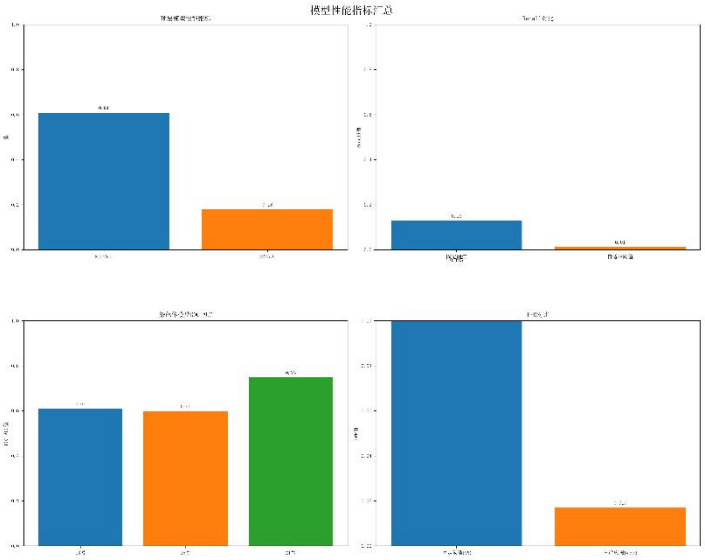

图 25 模型性能指标汇总图

本综合性能评估显示，模型在 ROC-AUC（0.85）和染色体特异性判别（21 号染色体达 0.7）方面表现良好，同时 FPR 控制在 1%以内。这表明模型特异性较高，说明误诊率极低，基本可以准确判断胎儿是否异常，避免给原本健康的人群带来不必要心理负担。但同时，敏感性较低，可能会漏诊一部分真正患病的胎儿。推测，可能跟数据中有显著的正负样本不平衡有关。

九、 模型评价与推广

9.1 模型的优点

(1) **临床贴合度高，核心假设符合实际需求**

问题一针对男胎 Y 染色体浓度建模时，通过广义线性混合模型引入孕妇个体随机效应，精准捕捉不同孕妇间的生理差异，解决了传统线性模型忽略个体异质性的问题；

问题二、三定义潜在风险函数时，严格遵循临床风险分级，将孕周越小风险权重越低、浓度达标越稳风险越低的医学共识量化为数学表达式，使模型输出直接匹配临床决策需求。

(2) **方法学创新，针对性解决复杂问题**

针对不同问题选用适配方法——问题一用自然样条处理孕周非线性关系，问题二用动态规划（DP）实现 BMI 分组与时点优化，问题四用 LightGBM 分层模型解决女胎异常判定的类不平衡问题。具备解释力与针对性，突破传统分析局限。

(3) **算法稳定高效**

通过变量标准化、Bootstrap 重采样（1000 次）、GroupKFold 交叉验证等确保稳定性，如问题二风险估计稳定性分数 0.995，问题四 LightGBM 训练耗时短且 FPR 控制在 1%以内。模型稳健，可实现“高特异性（误诊率低）”的临床核心需求。

(4) **结果实用可落地**

BMI 分组清晰，最佳时点明确（如问题三每组配套 3 个复检时点），直接匹配临床操作需求。

9.2 模型的不足

(1) **样本代表性有限**

例如问题四，异常样本数量显著稀少（仅占总数 1.5%）且分布离散，表明模型对正例的判别置信度不足。这种分布特征揭示了类别不平衡问题的严重影响。

(2) **核心假设简化过度**

设置假设“信息质量、年龄信息不对成功概率产生影响”，但后续改进 GLMM 时，年龄变量的 p 值为 0.063（接近 0.05 显著水平），说明年龄对 Y 染色体浓度达标概率存在微弱影响，原假设忽略了这一关联。

9.3 模型的推广

(1) **临床场景扩展**

将问题四模型推广至罕见染色体非整倍体判定；把问题一、二的浓度建模方法适配 NIPT+，为胎儿游离 DNA 分数（FF）的时点选择提供依据。

(2) **人群与地域适配**

基于多中心数据调整 BMI 分组（如南方地区缩小分组跨度），针对高龄、IVF 孕妇增加风险权重或优化孕周计算，适配不同人群需求。

(3) **技术对接落地**

用联邦学习整合多院数据更新模型，封装为决策模块对接妇幼健康系统，自动生成个性化检测方案，助力出生缺陷防治。

参考文献

[1] Ding, C., et al. "Cell-Free Fetal DNA Levels in Pregnancies Conceived by IVF." Human Reproduction, vol. 20, no. 11, 2005

[2] Chiu, R. W. K., et al. "Non-Invasive Prenatal Testing for Fetal Aneuploidies: A Systematic Review and Meta-Analysis." Prenatal Diagnosis, vol. 33, no. 8, 2013

[3] 薛莹,丁洁,贺权泽,等.孕妇年龄、孕周及体重指数对孕妇外周血中胎儿游离 DNA 比例的影响[J].中国产前诊断杂志(电子版),2017

[4] Zhou, Y., et al. Analysis of fetal fraction in maternal plasma cell-free DNA at early gestational age. Prenatal Diagnosis, 2021

[5] 豆包，豆包-R1-0528，字节跳动(ByteDance)

[6] DeepSeek, DeepSeek-R1-0528, 深度求索(DeepSeek)

附录

附录 1

数据预处理代码呈现

```
    import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats,optimize
from scipy.optimize import curve_fit
import matplotlib.patches as mpatches
plt.rcParams["font.family"] = ["SimSun", "Times New Roman"]
df1=pd.read_excel(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛
\CUMCM2025Problems\C 题\第一次数据预处理.xlsx'
                  ,sheet_name='男胎检测数据')

#建立 df2 来筛选检测的准确性和误差
def Is_Correct(row):
    if pd.isna(row['染色体的非整倍体']) and row['胎儿是否健康']=='是':
        return 1
    elif not pd.isna(row['染色体的非整倍体']) and row['胎儿是否健康']=="否":
        return 1
    else:
        return 0
df2=df1.copy()
df2['准确性']=df2.apply(Is_Correct,axis=1)

#建立 df3 来对数据进行整理（孕周数>=4 天就算新的一周）
df3=df2.copy()
df3['检测孕周']=df3['检测孕周'].astype(float)
df3['当周天数']=df3['当周天数'].fillna(0)
df3['当周天数']=df3['当周天数'].astype(float)
df3['检测孕周']+=df3['当周天数']/7
#df3.loc[df3['当周天数'] >= 4, '检测孕周'] += 1

##将 GC 含量超出异常的全部删掉
df4=df3.copy()
df4['GC 含量']=df4['GC 含量'].astype(float)
df4['13 号染色体的 GC 含量']=df4['13 号染色体的 GC 含量'].astype(float)
df4['18 号染色体的 GC 含量']=df4['18 号染色体的 GC 含量'].astype(float)
df4['21 号染色体的 GC 含量']=df4['21 号染色体的 GC 含量'].astype(float)
df4 = df4[
    (0.35 <= df4['GC 含量']) & (df4['GC 含量'] <= 0.6) &
```

```
    (0.35 <= df4['13号染色体的GC含量']) & (df4['13号染色体的GC含量'] <= 0.6)
&
    (0.35 <= df4['18号染色体的GC含量']) & (df4['18号染色体的GC含量'] <= 0.6)
&
    (0.35 <= df4['21号染色体的GC含量']) & (df4['21号染色体的GC含量'] <= 0.6)
]

##对X和Y染色体Z值异常的进行划分
def classify(z_value):
    abs_z = abs(z_value)
    if abs_z > 3:
        return "极端异常"
    elif abs_z > 2:
        return "一般异常"
    else:
        return "正常"

df4["X染色体异常类型"] = df4["X染色体的Z值"].apply(classify)
df4["Y染色体异常类型"] = df4["Y染色体的Z值"].apply(classify)
x_anomaly_counts = df4["X染色体异常类型"].value_counts()
y_anomaly_counts = df4["Y染色体异常类型"].value_counts()

#进行可视化
plt.figure(figsize=(15, 10))

# 散点图
plt.subplot(2, 2, 1)
sns.scatterplot(data=df4, x=range(len(df4)), y="X染色体的Z值",
                hue="X染色体异常类型", palette=["green", "orange", "red"],
                s=50, alpha=0.7)
plt.axhline(y=2, color='gray', linestyle='--', alpha=0.5)
plt.axhline(y=-2, color='gray', linestyle='--', alpha=0.5)
plt.axhline(y=3, color='gray', linestyle='-', alpha=0.7)
plt.axhline(y=-3, color='gray', linestyle='-', alpha=0.7)
plt.title("X染色体Z值分布与异常点")
plt.xlabel("样本序号")
plt.ylabel("Z值")
plt.legend(title="异常类型",loc='best')

plt.subplot(2, 2, 2)
sns.scatterplot(data=df4, x=range(len(df4)), y="Y染色体的Z值",
                hue="Y染色体异常类型", palette=["green", "orange", "red"],
                s=50, alpha=0.7)
plt.axhline(y=2, color='gray', linestyle='--', alpha=0.5)
```

```
plt.axhline(y=-2, color='gray', linestyle='--', alpha=0.5)
plt.axhline(y=3, color='gray', linestyle='-', alpha=0.7)
plt.axhline(y=-3, color='gray', linestyle='-', alpha=0.7)
plt.title("Y 染色体 Z 值分布与异常点")
plt.xlabel("样本序号")
plt.ylabel("Z 值")
plt.legend(title="异常类型")

# 柱状图
plt.subplot(2, 2, 3)
x_anomaly_counts.plot(kind="bar", color=["green", "orange", "red"])
plt.title("X 染色体异常类型数量统计")
plt.xlabel("异常类型")
plt.ylabel("样本数量")
plt.xticks(rotation=0)

plt.subplot(2, 2, 4)
y_anomaly_counts.plot(kind="bar", color=["green", "orange", "red"])
plt.title("Y 染色体异常类型数量统计")
plt.xlabel("异常类型")
plt.ylabel("样本数量")
plt.xticks(rotation=0)

plt.tight_layout()
plt.show()


#对检测孕周分布进行绘图
plt.figure(figsize=(11, 6))
box = sns.boxplot(
    x="检测孕周",
    data=df4,
    width=0.5,
    boxprops=dict(facecolor="#4FC3F7", edgecolor="#0288D1", linewidth=2,
alpha=0.8),  # 箱体透明度 0.8
    medianprops=dict(color="#C62828", linewidth=2.5),
    whiskerprops=dict(color="#0288D1", linewidth=1.5, alpha=0.8),  # 须线透
明度 0.8
    capprops=dict(color="#0288D1", linewidth=1.5, alpha=0.8),
    flierprops=dict(
        marker="D",
        color="#7B1FA2",
        markersize=8,
        markeredgewidth=1.2,
```

```
        alpha=0.7
    )
)


sns.stripplot(
    x="检测孕周",
    data=df4,
    hue=df4["检测孕周"].apply(lambda x: "正常" if 10 <= x <= 25 else "异常"),
    palette={"正常": "#388E3C", "异常": "#D32F2F"},
    size=5,
    jitter=0.25,
    alpha=0.6
)

# 添加正常范围参考线
plt.axvline(x=10, color="#6D4C41", linestyle="--", linewidth=2, alpha=0.7,
label="10 周（正常下限）")
plt.axvline(x=25, color="#6D4C41", linestyle="--", linewidth=2, alpha=0.7,
label="25 周（正常上限）")


total = len(df4)
normal_count = sum(10 <= x <= 25 for x in df4["检测孕周"])
abnormal_count = total - normal_count
abnormal_below_count = sum(x < 10 for x in df4["检测孕周"])
abnormal_above_count = sum(x > 25 for x in df4["检测孕周"])

# 添加统计标注框
plt.text(
    0.02, 0.95,
    f"总样本: {total}个\n正常(10-25 周): {normal_count}个\n"
    f"异常: {abnormal_count}个\n  - <10 周: {abnormal_below_count}个\n  - >
25 周: {abnormal_above_count}个",
    transform=plt.gca().transAxes,
    verticalalignment="top",
    fontsize=10,
    bbox=dict(
        boxstyle="round,pad=0.5",
        facecolor="white",
        edgecolor="#9E9E9E",
        alpha=0.9  # 标注框轻微透明
    )
)
```

```
plt.title("检测孕周分布箱图", fontsize=15, pad=20, color="#212121")
plt.xlabel("孕周数", fontsize=12, labelpad=10, color="#424242")
plt.legend(title="状态", loc="best", frameon=True, framealpha=0.9)

plt.xlim(5, 31)
plt.xticks(range(5, 32, 3), fontsize=10)
plt.grid(axis="x", linestyle="--", color="#EEEEEE", alpha=0.7)
sns.despine(top=True, right=True)

plt.tight_layout()
plt.show()

# 输出异常值统计
print(f"异常值统计：\n- 低于 10 周：{abnormal_below_count}个\n- 高于 25 周：
{abnormal_above_count}个\n- 异常率：{abnormal_count/total:.2%}")

#进行描述性统计
df5=df4.copy()
columns = ['年龄', '检测孕周', '孕妇 BMI', 'GC 含量', 'Y 染色体浓度', '准确性
']
stats_df = df5[columns].describe().round(2)
stats_df = stats_df.rename(index={
    'count': '样本量',
    'mean': '均值',
    'std': '标准差',
    'min': '最小值',
    '25%': '第 25 百分位数',
    '50%': '中位数',
    '75%': '第 75 百分位数',
    'max': '最大值'
})
mode = df5['准确性'].mode()[0]
mode_ratio = (df5['准确性'] == mode).mean().round(4) * 100
stats_df.loc['众数', '准确性'] = mode
stats_df.loc['众数占比(%)', '准确性'] = mode_ratio
stats_df=pd.DataFrame(stats_df)
stats_df.to_excel(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\描述性统
计.xlsx')


#检测孕周的散点
x = df5['检测孕周']
```

```python
y = df5['Y 染色体浓度']
n = len(df5)



n_bins = max(5, min(15, int(len(df5)/50)))  # 动态调整分组数
bins = np.linspace(x.min(), x.max(), n_bins + 1)
df5['分组'] = pd.cut(x, bins)

grouped = df5.groupby('分组').agg({
    '检测孕周': lambda x: (x.min() + x.max())/2,  # 取区间中点
    'Y 染色体浓度': 'mean'
}).dropna()  # 去除空组

mid_x = grouped['检测孕周']
mid_y = grouped['Y 染色体浓度']

#二次拟合
def quadratic_func(x, a, b, c):
    return a * x**2 + b * x + c

popt_original, _ = curve_fit(quadratic_func, x, y)

# 对中点数据进行二次拟合
popt_mid, _ = curve_fit(quadratic_func, mid_x, mid_y)


x_range = np.linspace(x.min(), x.max(), 100)
y_fit_original = quadratic_func(x_range, *popt_original)
y_fit_mid = quadratic_func(x_range, *popt_mid)

#可视化
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='gray', s=15, alpha=0.5, label='原始数据')
plt.scatter(mid_x, mid_y, color='black', s=50, marker='D', label='分组中点均值')
plt.plot(x_range, y_fit_mid, 'b-', linewidth=2, label='中点二次拟合')
plt.plot(x_range, y_fit_original, 'r--', linewidth=1.5, label='原始数据二次拟合')

plt.xlabel('检测孕周（周）')
plt.ylabel('Y 染色体浓度')
plt.legend()
#plt.show()
```

```
x_pad = (x.max() - x.min()) * 0.05
y_pad = (y.max() - y.min()) * 0.05
plt.xlim(x.min() - x_pad, x.max() + x_pad)
plt.ylim(y.min() - y_pad, y.max() + y_pad)

plt.grid(False)
plt.tight_layout()
plt.show()

##孕妇BMI的散点
x = df5['孕妇BMI']
y = df5['Y染色体浓度']
n = len(df5)


n_bins = max(5, min(15, int(len(df5)/50)))  # 动态调整分组数
bins = np.linspace(x.min(), x.max(), n_bins + 1)
df5['分组'] = pd.cut(x, bins)

# 计算每组中点和均值
grouped = df5.groupby('分组').agg({
    '孕妇BMI': lambda x: (x.min() + x.max())/2,  # 取区间中点
    'Y染色体浓度': 'mean'
}).dropna()  # 去除空组

mid_x = grouped['孕妇BMI']
mid_y = grouped['Y染色体浓度']

# 二次曲线拟合
def quadratic_func(x, a, b, c):
    return a * x**2 + b * x + c

popt_original, _ = curve_fit(quadratic_func, x, y)
popt_mid, _ = curve_fit(quadratic_func, mid_x, mid_y)
x_range = np.linspace(x.min(), x.max(), 100)
y_fit_original = quadratic_func(x_range, *popt_original)
y_fit_mid = quadratic_func(x_range, *popt_mid)

#可视化
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='gray', s=15, alpha=0.5, label='原始数据')
plt.scatter(mid_x, mid_y, color='black', s=50, marker='D', label='分组中点均值')
plt.plot(x_range, y_fit_mid, 'b-', linewidth=2, label='中点二次拟合')
```

```
plt.plot(x_range, y_fit_original, 'r--', linewidth=1.5, label='原始数据二次拟合')

plt.xlabel('孕妇 BMI')
plt.ylabel('Y 染色体浓度')
plt.legend()
#plt.show()
x_pad = (x.max() - x.min()) * 0.05
y_pad = (y.max() - y.min()) * 0.05
plt.xlim(x.min() - x_pad, x.max() + x_pad)
plt.ylim(y.min() - y_pad, y.max() + y_pad)

plt.grid(False)
plt.tight_layout()
    plt.show()
```

附录 2

问题一代码呈现

```
    import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from patsy import dmatrix
import statsmodels.api as sm
from scipy import stats
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.regression.mixed_linear_model import MixedLM

df1 = pd.read_excel(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\处理后干净数据.xlsx')

df2 = df1.copy()
#GC 含量过滤
# df2 = df2[(0.4 <= df2['GC 含量']) & (df2['GC 含量'] <= 0.6)]

# 数据清洗函数)
def clean_data(df):
    #染色体浓度筛选
    df['检测孕周'] = pd.to_numeric(df['检测孕周'], errors='coerce')
    df['Y 染色体浓度'] = pd.to_numeric(df['Y 染色体浓度'], errors='coerce')

    # 处理嵌套数组数据（来自特定检测设备格式）
    df_clean = df.dropna(subset=['检测孕周', 'Y 染色体浓度']).copy()
    df_clean['检测孕周'] = df_clean['检测孕周'].apply(
```

```
        lambda x: x[0] if isinstance(x, (list, np.ndarray)) else x
    )
    df_clean['Y染色体浓度'] = df_clean['Y染色体浓度'].apply(
        lambda x: x[0] if isinstance(x, (list, np.ndarray)) else x
    )

    print(f"清洗前样本量: {len(df)}")
    print(f"清洗后样本量: {len(df_clean)}")
    return df_clean


df_clean = clean_data(df2)

df_clean.to_excel(r'D:\2024——2025仇翰辰\数学\数学建模\2025竞赛\清理后数据.xlsx')

# =\\\\\\/;;;提取核心变量
x = df_clean['检测孕周']
y = df_clean['Y染色体浓度']
df_range = range(3, 7)
aic_values = []
bic_values = []
models = []
coef_dfs = []
for df in df_range:
    X = dmatrix(f'cr(检测孕周, df={df})', data=df_clean,
return_type='dataframe')
    X = sm.add_constant(X)
    model = sm.OLS(y, X).fit()
    models.append(model)
    aic_values.append(model.aic)
    bic_values.append(model.bic)

    coef_df = pd.DataFrame({
        '自由度': df,
   '系数名': model.params.index, '系数值': model.params.values
    })
    coef_dfs.append(coef_df)

# AIC; BIC
best_aic_index = np.argmin(aic_values)
best_aic_df = df_range[best_aic_index]
best_aic_model = models[best_aic_index]
```

```
best_bic_index = np.argmin(bic_values)
best_bic_df = df_range[best_bic_index]
best_bic_model = models[best_bic_index]

print(f"\n根据 AIC 选择的最佳自由度: {best_aic_df}")
print(f"\n根据 BIC 选择的最佳自由度: {best_bic_df}")

# 、绘制拟合曲线
plt.figure(figsize=(10, 6))
x_plot = np.linspace(x.min(), x.max(), 100)
for i, df in enumerate(df_range):
    X_plot = dmatrix(f'cr(检测孕周, df={df})', data={'检测孕周': x_plot},
return_type='dataframe')
    X_plot = sm.add_constant(X_plot)
    y_plot = models[i].predict(X_plot)
    plt.plot(x_plot, y_plot, label=f'df={df}')

plt.scatter(x, y, alpha=0.5, label='原始数据')
plt.axvline(x=12, color='gray', linestyle='--', alpha=0.7, label='孕 12 周(关
键期)')
plt.axvline(x=20, color='gray', linestyle='--', alpha=0.7, label='孕 20 周(关
键期)')
plt.xlabel('检测孕周')
plt.ylabel('Y 染色体浓度')
plt.legend()
plt.title('不同自由度的样条回归拟合曲线')
plt.show()


all_coef_df = pd.concat(coef_dfs, ignore_index=True)
all_coef_df.to_excel(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\回归系
数结果.xlsx', index=False)
print("回归系数结果已导出")

#==、==- GLMM 完整模型
df2["孕妇代码"] = df2["孕妇代码"].astype(str)

# 标准化处理
df2["检测孕周_std"] = (df2["检测孕周"] - df2["检测孕周"].mean()) / df2["检测
孕周"].std()
df2["孕妇 BMI_std"] = (df2["孕妇 BMI"] - df2["孕妇 BMI"].mean()) / df2["孕妇
BMI"].std()

df_spline = 5
```

```
ns_formula = f"cr(检测孕周_std, df={df_spline})"
X_ns = dmatrix(ns_formula, data=df2, return_type="dataframe")
if 'Intercept' in X_ns.columns:
    X_ns = X_ns.drop('Intercept', axis=1)

# BMI 主效应+样条×BMI 交互项
X_bmi = df2[["孕妇 BMI_std"]].copy()
X_interaction = X_ns.multiply(X_bmi["孕妇 BMI_std"], axis=0)
X_interaction.columns = [f"ns_bmi_{col}" for col in X_interaction.columns]

X_fixed = pd.concat([X_ns, X_bmi, X_interaction], axis=1)
def remove_high_vif(X, threshold=10.0):
    while True:
        vifs = [variance_inflation_factor(X.values, i) for i in
range(X.shape[1])]
        max_vif = max(vifs)
        if max_vif > threshold:
            max_idx = vifs.index(max_vif)
            col_to_drop = X.columns[max_idx]
            print(f"移除高 VIF 变量: {col_to_drop} (VIF = {max_vif:.2f})")
            X = X.drop(col_to_drop, axis=1)
        else:
            break
    return X

X_fixed = remove_high_vif(X_fixed)

#########随机效应
exog_re = pd.DataFrame({"intercept": 1}, index=df2.index)

# 拟合线性混合模型
y = df2["Y 染色体浓度"]
groups = df2["孕妇代码"]

model = MixedLM(
    endog=y,
    exog=X_fixed,
    groups=groups,
    exog_re=exog_re
)
result = model.fit()

# 计算 R 方
def mixed_model_r2(result):
```

```
    fixed_variance = np.var(result.fittedvalues)

    # 随机效应方差
    if hasattr(result, 'random_effects'):
        random_effects = np.array(list(result.random_effects.values()))
        random_variance = np.var(random_effects)
    else:
        random_variance = 0.0

    #残差方差
    residual_variance = result.scale if hasattr(result, 'scale') else
np.var(result.resid)

    total_variance = fixed_variance + random_variance + residual_variance

    marginal_r2 = fixed_variance / total_variance
    conditional_r2 = (fixed_variance + random_variance) / total_variance

    return {
        "边际 R 方 (仅固定效应)": round(marginal_r2, 4),"条件 R 方 (固定+随机
效应)": round(conditional_r2, 4),
        "固定效应方差": round(fixed_variance, 4),
   "随机效应方差": round(random_variance, 4),
"残差方差": round(residual_variance, 4)
    }

r2_results = mixed_model_r2(result)
print("\n=== 模型拟合优度 ===")
for key, value in r2_results.items():
    print(f"{key}: {value}")

print("\n=== 线性混合模型（LMM）摘要 ===")
print(result.summary())

fixed_effects = pd.DataFrame({
    "系数名": result.params.index,
    "系数值": result.params.values,
    "标准误": result.bse.values,
    "z 值": result.tvalues.values,
    "P 值": result.pvalues.values
})

# 随机效应结果
random_cov = result.cov_re
```

```
random_summary = pd.DataFrame({
    "参数": random_cov.index,
    "方差/协方差": random_cov.values.flatten()
})

# 个体随机效应
re_individual = pd.DataFrame(result.random_effects).T
re_individual.columns = exog_re.columns
re_individual["孕妇代码"] = re_individual.index

with pd.ExcelWriter("LMM_结果_with_R2.xlsx") as writer:
    fixed_effects.to_excel(writer, sheet_name="固定效应", index=False)
    random_summary.to_excel(writer, sheet_name="随机效应方差协方差",
index=False)
    re_individual.to_excel(writer, sheet_name="个体随机效应", index=False)
    pd.DataFrame(list(r2_results.items()), columns=["指标", "值
"]).to_excel(writer, sheet_name="R 方结果", index=False)
print("\n结果已导出至 'LMM_结果_with_R2.xlsx'")

# 可视化】
df2["拟合值"] = result.fittedvalues

plt.figure(figsize=(10, 6))
plt.scatter(df2["检测孕周"], df2["Y 染色体浓度"], alpha=0.5, label="实际值")
plt.scatter(df2["检测孕周"], df2["拟合值"], alpha=0.5, color="red", label="
拟合值")
plt.axvspan(10, 14, alpha=0.1, color='green', label='最佳检测窗口')
plt.xlabel("检测孕周")
plt.ylabel("Y 染色体浓度")
plt.title("LMM 拟合值与实际值对比")
plt.legend()
plt.show()

####残差分析
residuals = result.resid

plt.figure(figsize=(10, 6))
plt.scatter(df2["拟合值"], residuals)
plt.axhline(y=0, color="r", linestyle="--")
plt.xlabel("拟合值")
plt.ylabel("残差")
plt.title("LMM 残差图")
plt.show()
```

```python
# #方差分量输出
def print_variance_components(result):

    print("\n=== 方差分量分析 ===")
    print(f"随机截距方差: {float(result.cov_re.iloc[0,0]):.6f}")
    print(f"残差方差: {float(result.scale):.6f}")

    icc = float(result.cov_re.iloc[0,0]) / (float(result.cov_re.iloc[0,0]) +
float(result.scale))
    print(f"组内相关系数(ICC): {icc:.4f} (表示{icc*100:.1f}%的变异来自个体差
异)")

print_variance_components(result)

# 固定效应公式
def print_fixed_effects_formula(result):"
    print("\n=== 固定效应公式 ===")
    formula = "Y 染色体浓度 = "
    terms = []

    for name, value in result.params.items():
        if name == "Intercept":
            terms.append(f"{value:.4f}")
        else:
            terms.append(f"{value:.4f}*{name}")

    formula += " + ".join(terms)
    print(formula)

    print("\n临床显著性:")
    for name, pvalue in zip(result.params.index, result.pvalues):
        if "BMI" in name:
            clinical_sig = "临床重要" if pvalue < 0.05 else "临床不显著"
            print(f"{name}: p={pvalue:.4f} ({clinical_sig})")

print_fixed_effects_formula(result)

# 稳健性分析
print("\n\n=== 开始稳健性分析 ===")

# 孕周分组
df_early = df2[df2['检测孕周'] <= 21].copy()
df_late = df2[df2['检测孕周'] > 21].copy()
print(f"全样本量: {len(df2)}")
```

```
print(f"≤21 周组样本量: {len(df_early)}")
print(f">21 周组样本量: {len(df_late)}")

# 拟合分组模型
def fit_robust_model(df_subset, group_name):
    ##拟合分组模型
    print(f"\n--- 正在拟合 {group_name} 模型 ---")

    X_ns = dmatrix(f"cr(检测孕周_std, df={df_spline})", data=df_subset,
return_type="dataframe")
    if 'Intercept' in X_ns.columns:
        X_ns = X_ns.drop('Intercept', axis=1)
    X_bmi = df_subset[["孕妇 BMI_std"]].copy()

    X_fixed = pd.concat([X_ns, X_bmi], axis=1)
    X_fixed = remove_high_vif(X_fixed)
    exog_re = pd.DataFrame({"intercept": 1}, index=df_subset.index)

    y = df_subset["Y 染色体浓度"]
    groups = df_subset["孕妇代码"]

    model = MixedLM(endog=y, exog=X_fixed, groups=groups, exog_re=exog_re)
    result = model.fit()

    r2_results = mixed_model_r2(result)

    return result, {
        "组别": group_name,
  "样本量": len(df_subset),        "孕妇数": len(df_subset["孕妇代码
"].unique()),
        "随机截距方差": result.cov_re.iloc[0,0],
    "残差方差": result.scale,
 "条件 R 方": r2_results["条件 R 方 (固定+随机效应)"]
    }

result_early, summary_early = fit_robust_model(df_early, "≤21 周")
result_late, summary_late = fit_robust_model(df_late, ">21 周")

summary_full = {
    "组别": "全样本",
    "样本量": len(df2),
    "孕妇数": len(df2["孕妇代码"].unique()),   "随机截距方差":
result.cov_re.iloc[0,0],
  "残差方差": result.scale,
```

```
    "条件 R 方": r2_results["条件 R 方 (固定+随机效应)"]
}

robust_summary = pd.DataFrame([summary_full, summary_early, summary_late])
print("\n=== 模型临床稳定性评估 ===")
var_ratio_early = summary_early["随机截距方差"] / summary_full["随机截距方差
"]
var_ratio_late = summary_late["随机截距方差"] / summary_full["随机截距方差"]

print(f"随机效应方差变化: 早期组={var_ratio_early:.2f}, 晚期组
={var_ratio_late:.2f}")
print("建议: 若变化>30%需考虑孕周特异性模型")

    robust_summary.to_excel("稳健性分析结果.xlsx", index=False)
```

附录 3

问题二代码呈现

```
    import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_excel(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\处理后干
净数据.xlsx')
print(f"原始数据量: {len(df)}条记录 | BMI 范围: {df['孕妇
BMI'].min():.1f}-{df['孕妇 BMI'].max():.1f}")

#筛选有效的
valid_df = df[df['Y 染色体浓度'] >= 0.04].copy()
print(f"有效检测记录: {len(valid_df)}条 ({len(valid_df)/len(df)*100:.1f}%)")

#找最早达标孕周
min_weeks = valid_df.groupby('孕妇 BMI')['检测孕周'].min().reset_index()
min_weeks.columns = ['孕妇 BMI', '最早达标孕周']

bmi_values = min_weeks['孕妇 BMI'].sort_values().values
group_count = 5
group_size = len(bmi_values) // group_count
bmi_groups = []

#分组边界
for i in range(group_count):
```

```
    start_idx = i * group_size
    end_idx = (i + 1) * group_size if i < group_count - 1 else len(bmi_values)
    group_min = bmi_values[start_idx]
    group_max = bmi_values[end_idx - 1]

    label = f"[{group_min:.1f}, {group_max:.1f}]"
    bmi_groups.append((group_min, group_max, label))
    min_weeks.loc[(min_weeks['孕妇BMI'] >= group_min) &
                  (min_weeks['孕妇BMI'] <= group_max), 'BMI分组'] = label

print(f"创建了{len(bmi_groups)}个BMI分组")

# 风险函数
def calculate_detection_risk(gestational_week, concentration):
    if gestational_week < 18:
        base_risk = 0.3
    elif gestational_week < 22:
        base_risk = 0.5
    elif gestational_week < 26:
        base_risk = 0.7
    else:
        base_risk = 1.0
    concentration_factor = 0.5 + 0.5 * np.tanh(5 * (concentration - 0.04))
    final_risk = base_risk * concentration_factor
    return min(final_risk, 1.0)

# ###########为每个BMI组寻找最佳检测孕周
results = []
group_plots = []

for group_label in min_weeks['BMI分组'].unique():
    group = min_weeks[min_weeks['BMI分组'] == group_label]
    if group.empty:
        continue

    group_min, group_max = [float(x) for x in group_label.strip('[]').split(',
')]
    min_wk = group['最早达标孕周'].min()
    max_wk = group['最早达标孕周'].max()
    median_bmi = group['孕妇BMI'].median()

    candidate_weeks = list(range(int(min_wk), int(max(max_wk, 26)) + 1))
    week_risks = []
```

```
    for week in candidate_weeks:
        conc_data = df[(df['孕妇BMI'] >= group_min) &
                       (df['孕妇BMI'] <= group_max) &
                       (df['检测孕周'] == week)]['Y染色体浓度']

        if not conc_data.empty:
            typical_conc = conc_data.median()
        else:
            typical_conc = 0.04 + 0.002 * (week - min_wk)
        risk = calculate_detection_risk(week, typical_conc)
        week_risks.append(risk)
    min_risk = min(week_risks)
    best_week = candidate_weeks[week_risks.index(min_risk)]

    # 记录结果
    results.append({
        'BMI分组': group_label,
  'BMI范围': group_label,
  'BMI中位数': round(median_bmi, 1),'样本量': len(group),
'孕周范围': f"{min_wk}-{max_wk}周",
        '推荐孕周': best_week,
    '预期风险': round(min_risk, 3)
    })

    # 存储绘图数据）
    group_plots.append({
        'group': group_label,
    'weeks': candidate_weeks,
 'risks': week_risks,'best_week': best_week,'min_risk': min_risk,
        'median_bmi': median_bmi
    })


result_df = pd.DataFrame(results)

print(result_df[['BMI分组','BMI中位数','样本量','孕周范围','推荐孕周','
预期风险']])

# 可视化
#、、风险曲线对比
plt.figure(figsize=(14, 8))
plt.suptitle('不同BMI孕妇的最佳检测孕周分析', fontsize=16, y=0.97)

risk_zones = [
```

```
    (10, 17, '低风险区', '#c4e8c4'),
    (17, 21, '中风险区', '#fff9c4'),
    (21, 25, '高风险区', '#ffcc80'),
    (25, 30, '极高风险区', '#ffab91')
]

for i, plot_data in enumerate(group_plots):
    ax = plt.subplot(2, 3, i+1)
    for start, end, label, color in risk_zones:
        ax.axvspan(start, end, alpha=0.3, color=color)
        if i == 0:
            ax.text((start+end)/2, 0.92, label, ha='center', fontsize=9)
    ax.plot(plot_data['weeks'], plot_data['risks'], 'b-', lw=2, label='风险
变化')
    ax.plot(plot_data['best_week'], plot_data['min_risk'], 'ro', ms=8,
            label=f'最佳点({plot_data["best_week"]}周)')
    for wk in [17, 21, 25]:
        ax.axvline(wk, color='gray', ls='--', alpha=0.5)
    ax.set_title(f'BMI 分组: {plot_data["group"]}\n 中位 BMI:
{plot_data["median_bmi"]:.1f}')
    ax.set_xlabel('检测孕周')
    ax.set_ylabel('检测风险')
    ax.legend(loc='upper left')
    ax.grid(alpha=0.2)
    ax.set_ylim(0, 1.05)

plt.tight_layout()
plt.savefig('BMI 分组检测风险分析.png', dpi=150)
print("风险分析图已保存")

# 孕周分布箱线图“
plt.figure(figsize=(10, 6))

df['BMI 分组'] = '其他'
for group_min, group_max, label in bmi_groups:
    df.loc[(df['孕妇 BMI'] >= group_min) & (df['孕妇 BMI'] <= group_max), 'BMI
分组'] = label

sns.boxplot(x='BMI 分组', y='检测孕周', data=df, palette='pastel')
plt.title('不同 BMI 孕妇的实际检测孕周分布')
plt.xticks(rotation=20)
plt.grid(alpha=0.2)
plt.tight_layout()
    plt.savefig('孕周分布箱线图.png', dpi=150)
```

附录 4

问题三代码呈现

```
    import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import patsy
from patsy import dmatrix
from statsmodels.formula.api import mixedlm
from sklearn.preprocessing import StandardScaler
from scipy.linalg import eigh
from scipy.stats import gaussian_kde
import warnings
warnings.filterwarnings('ignore')

plt.rcParams["font.family"] = ["SimSun", "Times New Roman"]
plt.rcParams['axes.unicode_minus'] = False

#映射函数定义
def map_boundaries(eta, B_min, B_max):
    # 计算权重
    w = np.exp(eta) / np.sum(np.exp(eta))

    # 计算累积权重
    c = np.cumsum(w)

    # 计算边界
    b = B_min + (B_max - B_min) * c

    return b

def map_timepoints(zeta, J_min, J_max, S, Delta_min): #返回:
        #J: K×S 矩阵，每行对应一组的检测时点
    K = len(zeta) // S
    J = np.zeros((K, S))

    for g in range(K):
        # 第一时点
        J[g, 0] = J_min + (J_max - J_min - (S-1)*Delta_min) * (1 / (1 +
np.exp(-zeta[g*S])))
```

```
        # 后续时点
        for s in range(1, S):
            prev_J = J[g, s-1]
            max_range = J_max - prev_J - (S-s-1)*Delta_min
            J[g, s] = prev_J + Delta_min + max_range * (1 / (1 + np.exp(-zeta[g*S
+ s])))

    return J

#--、、、
```

```
spline_cols = [f'spline_{i}' for i in range(spline_design.shape[1] - 1)]
df[spline_cols] = spline_design.iloc[:, 1:].values

for i, col in enumerate(spline_cols):
    df[f'spline_{i}_xBMI'] = df[col] * df['BMI_c']
df['IVF_F'] = df['IVF 妊娠'].apply(lambda x: 0 if str(x).strip() == '自然受孕
' else 1)

# PCA 分析
quality_data = pd.DataFrame()
quality_data['M'] = df['在参考基因组上比对的比例']
quality_data['N'] = df['重复读段的比例']
quality_data['O'] = df['唯一比对的读段数  ']
quality_data['|P-0.4|'] = np.abs(df['GC 含量'] - 0.4)
quality_data['AA'] = df['被过滤掉读段数的比例']
quality_data['log_L'] = np.log(df['原始读段数'])

scaler = StandardScaler()
z_scores = scaler.fit_transform(quality_data)
z_df = pd.DataFrame(z_scores, columns=quality_data.columns,
index=quality_data.index)

n = len(z_df)
S_cov = np.cov(z_df.T, bias=False)
eigenvalues, eigenvectors = eigh(S_cov)
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

cumulative_variance = np.cumsum(eigenvalues) / np.sum(eigenvalues)
R = np.argmax(cumulative_variance >= 0.8) + 1

U = eigenvectors[:, :R]
Q_scores = z_df.dot(U)

corr_matrix = np.zeros((R, 6))
for r in range(R):
    for j, col in enumerate(z_df.columns):
        corr_matrix[r, j] = np.corrcoef(Q_scores.iloc[:, r], z_df[col])[0, 1]

s_p = np.zeros(R)
for r in range(R):
    s_p[r] = np.sign(
        corr_matrix[r, 0] +
```

```
        corr_matrix[r, 2] -
        corr_matrix[r, 1] -
        corr_matrix[r, 3] -
        corr_matrix[r, 4] +
        corr_matrix[r, 5]
    )

Q_pca = Q_scores.copy()
for r in range(R):
    Q_pca.iloc[:, r] = s_p[r] * Q_scores.iloc[:, r]

Q_pca.columns = [f'PC{i+1}' for i in range(R)]
df = pd.concat([df, Q_pca], axis=1)

# ############## Step2: 计算检测成功概率
print("Step 2: 计算检测成功概率")

#准备 GLMM 数据
required_columns = ['Y 染色体浓度', 'BMI_c', 'Age_c', 'IVF_F', '孕妇代码'] +
spline_cols + [f'spline_{i}_xBMI' for i in range(len(spline_cols))] +
[f'PC{i+1}' for i in range(R)]
df_clean = df.dropna(subset=required_columns).copy()
df_clean.reset_index(drop=True, inplace=True)

spline_terms = " + ".join(spline_cols)
spline_interaction_terms = " + ".join([f'spline_{i}_xBMI' for i in
range(len(spline_cols))])
pc_terms = " + ".join([f'PC{i+1}' for i in range(R)])

formula = f"Y 染色体浓度 ~ {spline_terms} + {spline_interaction_terms} + BMI_c
+ Age_c + IVF_F + {pc_terms}"

print("拟合保留质量主成分的 GLMM 模型...")
model = mixedlm(formula, df_clean, groups=df_clean["孕妇代码"])
result = model.fit(reml=True, method='lbfgs')
print("模型拟合成功!")

#、计算成功概率函数
def calculate_success_probability(model_result, df_clean, u0_threshold=0.05):
    fixed_effect = model_result.predict(df_clean)
    re = model_result.random_effects
    re_values = pd.Series([v[0] for v in re.values()], index=re.keys())

    marginal_variance = np.var(model_result.resid) + np.var(re_values)
```

```
    z_V = (fixed_effect - 0.04) / np.sqrt(marginal_variance)
    success_prob = 1 / (1 + np.exp(-z_V))
    u_effect = 1 / (1 + np.exp(fixed_effect * 5))

    combined_success_prob = success_prob * u_effect

    return combined_success_prob

#计算成功概率;
df_clean['success_prob'] = calculate_success_probability(result, df_clean)

# ====================;
print("Step 3: BMI 分组")

K = 5
bmi_min = df_clean['孕妇 BMI'].min()
bmi_max = df_clean['孕妇 BMI'].max()

bmi_bins = np.linspace(bmi_min, bmi_max, K+1)
bmi_labels = [f'BMI 组{i+1}' for i in range(K)]

df_clean['BMI_group'] = pd.cut(df_clean['孕妇 BMI'], bins=bmi_bins,
labels=bmi_labels, include_lowest=True, duplicates='drop')
group_stats = df_clean.groupby('BMI_group').agg({
    '孕妇 BMI': ['count', 'min', 'max'],
    'success_prob': 'mean'
}).round(3)
group_stats.columns = ['样本数(n_g)', 'BMI 最小值', 'BMI 最大值', '平均成功概
率']
group_stats['样本占比(π_g)'] = group_stats['样本数(n_g)'] / len(df_clean)
print("BMI 分组统计:")
print(group_stats)

# ====== Step4、 构建多阶段期望损失模型
print("Step 4: 构建多阶段期望损失模型")

# 设置成本参数
c_fail = 1.0
c_visit = 0.2
# 定义孕周成本函数 C_T(J)
def cost_function_J(J):
    if J < 12:
        return 0.8
    elif J <= 27:
```

```
        return 0.5
    else:
        return 0.9

####》多阶段期望损失函数
def expected_loss_S(success_probs, J_values, c_fail, c_visit):
    S = len(success_probs)
    total_loss = 0.0

    for s in range(1, S+1):
        arrival_prob = np.prod([1 - success_probs[t-1] for t in range(1, s)])
        visit_cost = c_visit if s > 1 else 0
        J_cost = cost_function_J(J_values[s-1])
        fail_cost = c_fail * (1 - success_probs[s-1])
        total_loss += arrival_prob * (visit_cost + J_cost + fail_cost)

    return total_loss

#
S_detection = 3  # 检测阶段数
Delta_min = 1.0  # 最小检测间隔
detection_points = {}
for group in bmi_labels:
    group_data = df_clean[df_clean['BMI_group'] == group]
    if len(group_data) > 0:
        q1 = group_data['检测孕周'].quantile(0.25)
        q2 = group_data['检测孕周'].quantile(0.5)
        q3 = group_data['检测孕周'].quantile(0.75)
        detection_points[group] = [q1, q2, q3]

#每个 BMI 组的组内风险期望
group_risks = {}
for group in bmi_labels:
    group_data = df_clean[df_clean['BMI_group'] == group]
    if len(group_data) > 0:
        n_g = len(group_data)
        total_risk = 0


        for idx, row in group_data.iterrows():
            success_probs = [row['success_prob']] * S_detection
            sample_loss = expected_loss_S(success_probs,
detection_points[group], c_fail, c_visit)
            total_risk += sample_loss
```

```
        R_g = total_risk / n_g if n_g > 0 else 0
        group_risks[group] = R_g
        print(f"{group}的组内风险期望: {R_g:.4f}")

# 计算总体风险期望
total_risk = 0
for group in bmi_labels:
    if group in group_risks:
        π_g = group_stats.loc[group, '样本占比(π_g)']
        total_risk += π_g * group_risks[group]

print(f"\n总体风险期望: {total_risk:.4f}")

# 【step5】构建目标函数与PSO优化
print("Step 5: 构建目标函数与PSO优化")

# 设置惩罚系数
λ = 0.1  # 复杂度惩罚系数
ρ_con = 10.0  # 可行性惩罚系数
n_min = 0.05 * len(df_clean)  # 最小组样本数阈值（总样本数的5%）
S = 3  # 检测阶段数

#可行性惩罚函数
def feasibility_penalty(group_counts, n_min, n_total):

    #计算可行性惩罚项
    penalty = 0
    for n_g in group_counts:
        if n_g < n_min:
            penalty += (max(0, n_min - n_g) / n_total) ** 2
    return penalty

#完整目标函数
def objective_function(theta, df_clean, K, S, c_fail, c_visit, λ, ρ_con,
n_min):
    eta = theta[:K-1]
    zeta = theta[K-1:]

    boundaries = map_boundaries(eta, df_clean['孕妇BMI'].min(), df_clean['
孕妇BMI'].max())
    boundaries = np.unique(boundaries)  # 移除重复值
    bmi_bins = np.concatenate([[df_clean['孕妇BMI'].min()], boundaries,
[df_clean['孕妇BMI'].max()]])
    bmi_bins = np.unique(bmi_bins)  # 再次确保唯一性
```

```
    num_labels = len(bmi_bins) - 1
    bmi_labels = [f'BMI 组{i+1}' for i in range(num_labels)]
    timepoints = map_timepoints(zeta, df_clean['检测孕周'].min(), df_clean['
检测孕周'].max(), S, Delta_min)
    df_clean['BMI_group'] = pd.cut(
        df_clean['孕妇 BMI'],
        bins=bmi_bins,
   labels=bmi_labels,include_lowest=True,
duplicates='drop'
    )

    group_counts = df_clean.groupby('BMI_group').size().values
    n_total = len(df_clean)

    pen_size = feasibility_penalty(group_counts, n_min, n_total)

    group_risks = {}
    group_stats = df_clean.groupby('BMI_group').agg({'success_prob': 'mean',
'孕妇 BMI': 'count'})
    group_stats['π_g'] = group_stats['孕妇 BMI'] / n_total

    for group in bmi_labels:
        if group in group_stats.index:
            group_data = df_clean[df_clean['BMI_group'] == group]
            n_g = len(group_data)
            total_risk = 0

            for idx, row in group_data.iterrows():
                success_probs = [row['success_prob']] * S
                sample_loss = expected_loss_S(success_probs,
timepoints[bmi_labels.index(group)], c_fail, c_visit)
                total_risk += sample_loss

            R_g = total_risk / n_g if n_g > 0 else 0
            group_risks[group] = R_g

    weighted_risk = 0
    for group in group_risks:
        if group in group_stats.index:
            π_g = group_stats.loc[group, 'π_g']
            weighted_risk += π_g * group_risks[group]
    complexity_penalty = λ * K
```

```
    F_theta = weighted_risk + complexity_penalty + ρ_con * pen_size

    return F_theta, weighted_risk, complexity_penalty, pen_size

print("测试目标函数计算...")
test_theta = np.random.randn((K-1) + K*S)
F_value, weighted_risk, complexity_penalty, pen_size = objective_function(
    test_theta, df_clean, K, S, c_fail, c_visit, λ, ρ_con, n_min
)

print(f"目标函数值 F(θ): {F_value:.4f}")
print(f"加权风险期望: {weighted_risk:.4f}")
print(f"复杂度惩罚: {complexity_penalty:.4f}")
print(f"可行性惩罚: {pen_size:.4f}")

# 《PSO 优化框架》
print("\n 设置 PSO 优化框架...")
#参数设置
n_particles = 50  # 粒子数量
max_iter = 100 # 最大迭代次数
w = 0.7  # 惯性权重
c1 = 1.5   # 个体学习因子
c2 = 1.5   # 社会学习因子

#搜索空间边界
theta_dim = (K-1) + K*S
lower_bound = -5.0 * np.ones(theta_dim)
upper_bound = 5.0 * np.ones(theta_dim)

print(f"PSO 参数设置:")
print(f"  粒子数量: {n_particles}")
print(f"  最大迭代次数: {max_iter}")
print(f"  搜索空间维度: {theta_dim}")
print(f"  搜索范围: [{lower_bound[0]:.1f}, {upper_bound[0]:.1f}]")

#初始化粒子群
particles = np.random.uniform(low=lower_bound, high=upper_bound,
size=(n_particles, theta_dim))
velocities = np.zeros((n_particles, theta_dim))

personal_best_positions = particles.copy()
personal_best_fitness = np.full(n_particles, np.inf)
```

```
global_best_position = None
global_best_fitness = np.inf

#保存迭代历史
history = {
    'iteration': [],
 'best_fitness': [],'weighted_risk': [],'complexity_penalty': [],
    'feasibility_penalty': []
}

print("开始 PSO 优化...")
for iter in range(max_iter):
    for i in range(n_particles):
            fitness, w_risk, c_penalty, f_penalty = objective_function(
                particles[i], df_clean, K, S, c_fail, c_visit, λ, ρ_con,
n_min
            )

            if fitness < personal_best_fitness[i]:
                personal_best_fitness[i] = fitness
                personal_best_positions[i] = particles[i].copy()

            # 更新全局最佳
            if fitness < global_best_fitness:
                global_best_fitness = fitness
                global_best_position = particles[i].copy()


    # 记录迭代历史
    if global_best_fitness < np.inf:
        history['iteration'].append(iter)
        history['best_fitness'].append(global_best_fitness)
        history['weighted_risk'].append(w_risk)
        history['complexity_penalty'].append(c_penalty)
        history['feasibility_penalty'].append(f_penalty)

    # 更新粒子速度和位置
    for i in range(n_particles):
        r1, r2 = np.random.rand(2)
        velocities[i] = (w * velocities[i] +
                         c1 * r1 * (personal_best_positions[i] - particles[i])
+
                         c2 * r2 * (global_best_position - particles[i]))
```

```
        velocities[i] = np.clip(velocities[i], -1.0, 1.0)
        particles[i] += velocities[i]
        particles[i] = np.clip(particles[i], lower_bound, upper_bound)

    if (iter + 1) % 10 == 0:
        print(f"迭代 {iter+1}/{max_iter}, 最佳适应度:
{global_best_fitness:.4f}")

print("PSO 优化完成!")

#############、、、、可视化
print("分析优化结果...")
best_eta = global_best_position[:K-1]
best_zeta = global_best_position[K-1:]
boundaries = map_boundaries(best_eta, df_clean['孕妇 BMI'].min(), df_clean['
孕妇 BMI'].max())
boundaries = np.unique(boundaries)
best_bmi_bounds = np.concatenate([[df_clean['孕妇 BMI'].min()], boundaries,
[df_clean['孕妇 BMI'].max()]])
best_bmi_bounds = np.unique(best_bmi_bounds)

# 动态确定实际
actual_K = len(best_bmi_bounds) - 1
actual_bmi_labels = [f'BMI 组{i+1}' for i in range(actual_K)]

best_timepoints = map_timepoints(best_zeta, df_clean['检测孕周'].min(),
df_clean['检测孕周'].max(), S, Delta_min)

print("\n优化后的 BMI 分组边界:")
print(best_bmi_bounds)

print("\n优化后的检测时点:")
for g in range(actual_K):
    print(f"  {actual_bmi_labels[g]}: {best_timepoints[g]}")

plt.figure(figsize=(12, 10))
plt.subplot(2, 2, 1)
plt.plot(history['iteration'], history['best_fitness'], 'b-', linewidth=2)
plt.xlabel('迭代次数')
plt.ylabel('适应度值')
plt.title('PSO 收敛曲线')
plt.grid(True, alpha=0.3)

plt.subplot(2, 2, 2)
```

```
plt.plot(history['iteration'], history['weighted_risk'], 'r-', label='风险期望', linewidth=2)
plt.plot(history['iteration'], history['complexity_penalty'], 'g-', label='复杂度惩罚', linewidth=2)
plt.plot(history['iteration'], history['feasibility_penalty'], 'm-', label='可行性惩罚', linewidth=2)
plt.xlabel('迭代次数')
plt.ylabel('惩罚项值')
plt.title('各惩罚项变化')
plt.legend()
plt.grid(True, alpha=0.3)


plt.subplot(2, 2, 3)
plt.hist(df_clean['孕妇BMI'], bins=30, alpha=0.5, color='skyblue', label='原始分布', density=True)
for boundary in bmi_bins:
    plt.axvline(x=boundary, color='blue', linestyle='--', linewidth=2, alpha=0.7)

for boundary in best_bmi_bounds:
    plt.axvline(x=boundary, color='red', linestyle='-', linewidth=2, alpha=0.7)
plt.xlabel('孕妇BMI')
plt.ylabel('密度')
plt.title('优化前后BMI分组比较')
plt.grid(True, alpha=0.3)
plt.subplot(2, 2, 4)
# 原始时点
for g in range(K):
    plt.scatter([g+1]*S, detection_points[bmi_labels[g]], color='blue', alpha=0.7, label='原始' if g==0 else "")
# 优化后时点
for g in range(actual_K):
    plt.scatter([g+1+0.2]*S, best_timepoints[g], color='red', alpha=0.7, label='优化' if g==0 else "")
plt.xlabel('BMI组')
plt.ylabel('检测孕周')
plt.title('优化前后检测时点比较')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('PSO优化结果.png', dpi=300, bbox_inches='tight')
```

```
plt.show()

# 保存优化结果
optimization_results = {
    'best_theta': global_best_position,'best_fitness': global_best_fitness,
 'best_bmi_bounds': best_bmi_bounds,'best_timepoints': best_timepoints,
    'K': K,'S': S,
  'λ': λ, 'ρ_con': ρ_con,
'n_min': n_min,
    'history': history
}

np.savez(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\第三问
\pso_optimization_results.npz', **optimization_results)

print("\n 最终优化方案:")
print("=" * 50)
for g in range(actual_K):
    if g == 0:
        bmi_range = (df_clean['孕妇 BMI'].min(), best_bmi_bounds[1])
    elif g == actual_K-1:
        bmi_range = (best_bmi_bounds[-2], df_clean['孕妇 BMI'].max())
    else:
        bmi_range = (best_bmi_bounds[g], best_bmi_bounds[g+1])

    print(f"{actual_bmi_labels[g]} ({bmi_range[0]:.1f} -
{bmi_range[1]:.1f}):")
    print(f"  推荐检测孕周: {', '.join([f'{t:.1f}' for t in
best_timepoints[g]])}")
    print(f"  预期风险: {group_risks.get(actual_bmi_labels[g], 0):.4f}")
    print()

print(f"总体预期风险: {weighted_risk:.4f}")
print(f"目标函数值: {global_best_fitness:.4f}")

df_clean.to_excel(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\第三问\完
整分析结果 2.xlsx', index=False)
print("分析完成! 结果已保存。")
print("新增: 风险函数曲线可视化")

# 风险函数计算
def calculate_risk_at_J(J, group_data, model_result, c_fail, c_visit):
    temp_df = group_data.copy()
    temp_df['J_c'] = J - mean_gestational_week
```

```
    local_lower_bound = 10 - mean_gestational_week
    local_upper_bound = 25 - mean_gestational_week

    spline_temp = dmatrix(
        f"cr(J_c, knots={knots_centered}, lower_bound={local_lower_bound},
upper_bound={local_upper_bound})",
        data={"J_c": temp_df['J_c']},
        return_type='dataframe'
    )
    for i, col in enumerate(spline_cols):
        temp_df[col] = spline_temp.iloc[:, i+1].values
        temp_df[f'spline_{i}_xBMI'] = temp_df[col] * temp_df['BMI_c']

    # 获取固定效应预测值
    fixed_effect = model_result.predict(temp_df)

    # 计算成功概率
    marginal_variance = np.var(model_result.resid) +
np.var(list(model_result.random_effects.values()))
    z_V = (fixed_effect - 0.04) / np.sqrt(marginal_variance)
    success_prob = 1 / (1 + np.exp(-z_V))
    u_effect = 1 / (1 + np.exp(fixed_effect * 5))
    combined_success_prob = success_prob * u_effect

    J_cost = cost_function_J(J)
    risk = J_cost + c_fail * (1 - combined_success_prob)

    return risk.mean()

# 为每个分组创建风险函数曲线图
plt.figure(figsize=(15, 10))
J_range = np.linspace(10, 30, 100)

for g, group in enumerate(actual_bmi_labels):
    group_data = df_clean[df_clean['BMI_group'] == group]
    if len(group_data) == 0:
        continue

    risks = []
    for J in J_range:
        risks.append(calculate_risk_at_J(J, group_data, result, c_fail,
c_visit))
```

```
    ax = plt.subplot(2, 3, g+1)

    ax.plot(J_range, risks, 'b-', linewidth=2)
    ax.set_xlabel('孕周 (周)')
    ax.set_ylabel('风险值')
    ax.set_title(f'{group}风险函数曲线')
    ax.grid(True, alpha=0.3)

    for t in best_timepoints[g]:
        ax.axvline(x=t, color='r', linestyle='--', linewidth=1.5)
        ax.text(t, max(risks)*0.9, f'{t:.1f}周',
                fontsize=9, ha='center', va='bottom', color='red',
                bbox=dict(facecolor='white', alpha=0.7, edgecolor='none'))

    bmi_min = group_data['孕妇BMI'].min()
    bmi_max = group_data['孕妇BMI'].max()
    ax.text(0.05, 0.95, f'BMI范围: {bmi_min:.1f}-{bmi_max:.1f}',
            transform=ax.transAxes, fontsize=10,
            bbox=dict(facecolor='white', alpha=0.7))

plt.tight_layout()
plt.savefig('各分组风险函数曲线.png', dpi=300, bbox_inches='tight')
plt.show()

df_clean.to_excel(r'D:\2024——2025仇翰辰\数学\数学建模\2025竞赛\第三问\完整分析结果2.xlsx', index=False)
print("分析完成! 结果已保存。")


#扰动参数
n_perturbations = 100
perturbation_scale = 0.1

# 收集扰动结果
bmi_boundaries_perturbed = []
timepoints_perturbed = []

for i in range(n_perturbations):
    perturbed_theta = global_best_position + np.random.normal(0,
perturbation_scale, size=len(global_best_position))
    perturbed_eta = perturbed_theta[:K-1]
    perturbed_zeta = perturbed_theta[K-1:]
    perturbed_boundaries = map_boundaries(perturbed_eta, df_clean['孕妇
```

```
BMI'].min(), df_clean['孕妇BMI'].max())
    perturbed_boundaries = np.unique(perturbed_boundaries)
    perturbed_bmi_bounds = np.concatenate([[df_clean['孕妇BMI'].min()],
perturbed_boundaries, [df_clean['孕妇BMI'].max()]])
    perturbed_bmi_bounds = np.unique(perturbed_bmi_bounds)

    perturbed_timepoints = map_timepoints(perturbed_zeta, df_clean['检测孕周
'].min(), df_clean['检测孕周'].max(), S, Delta_min)

    internal_boundaries = perturbed_bmi_bounds[1:-1]
    if len(internal_boundaries) < (K-1):
        padding = np.full((K-1) - len(internal_boundaries),
internal_boundaries[-1])
        internal_boundaries = np.concatenate([internal_boundaries, padding])
    elif len(internal_boundaries) > (K-1):
        internal_boundaries = internal_boundaries[:K-1]

    bmi_boundaries_perturbed.append(internal_boundaries)
    timepoints_perturbed.append(perturbed_timepoints)

bmi_boundaries_perturbed = np.array(bmi_boundaries_perturbed)
bmi_boundaries_mean = np.mean(bmi_boundaries_perturbed, axis=0)
bmi_boundaries_std = np.std(bmi_boundaries_perturbed, axis=0)

timepoints_perturbed = np.array(timepoints_perturbed)
timepoints_mean = np.mean(timepoints_perturbed, axis=0)
timepoints_std = np.std(timepoints_perturbed, axis=0)

#雷达图
plt.figure(figsize=(10, 8))
ax1 = plt.subplot(111, polar=True)


n_boundaries = len(bmi_boundaries_mean)
angles = np.linspace(0, 2 * np.pi, n_boundaries, endpoint=False).tolist()
angles += angles[:1]

# 绘制平均边界
values = bmi_boundaries_mean.tolist()
values += values[:1]
ax1.plot(angles, values, 'b-', linewidth=2, label='平均边界')
ax1.fill(angles, values, 'b', alpha=0.1)

upper_bound = bmi_boundaries_mean + bmi_boundaries_std
```

```
lower_bound = bmi_boundaries_mean - bmi_boundaries_std
upper_values = upper_bound.tolist()
upper_values += upper_values[:1]
lower_values = lower_bound.tolist()
lower_values += lower_values[:1]
ax1.fill_between(angles, lower_values, upper_values, color='r', alpha=0.2,
label='标准差范围')

ax1.set_theta_offset(np.pi/2)
ax1.set_theta_direction(-1)
ax1.set_thetagrids(np.degrees(angles[:-1]), [f'边界{i+1}' for i in
range(n_boundaries)])

plt.title('BMI 分组边界稳定性分析', size=16, pad=20)
plt.legend(loc='upper right', bbox_to_anchor=(1.1, 1.1))

plt.tight_layout()
plt.savefig('BMI 分组边界稳定性.png', dpi=300, bbox_inches='tight')
plt.show()

# 计算稳定性指标
stability_metrics = {
    'bmi_boundaries': {
        'mean': bmi_boundaries_mean,
        'std': bmi_boundaries_std,
        'cv': bmi_boundaries_std / bmi_boundaries_mean  # 变异系数
    },
    'timepoints': {
        'mean': timepoints_mean,
        'std': timepoints_std,
        'cv': timepoints_std / timepoints_mean
}}

print("\n稳定性分析结果:")
print("BMI 边界稳定性:")
for i in range(len(bmi_boundaries_mean)):
    print(f"  边界{i+1}: 均值={bmi_boundaries_mean[i]:.2f}, 标准差
={bmi_boundaries_std[i]:.4f}, 变异系数
={stability_metrics['bmi_boundaries']['cv'][i]:.4f}")

print("\n检测时点稳定性:")
for g in range(actual_K):
    print(f"  {actual_bmi_labels[g]}:")
    for s in range(S):
```

```
        print(f"    时点{s+1}: 均值={timepoints_mean[g][s]:.2f}, 标准差
={timepoints_std[g][s]:.4f}, 变异系数
={stability_metrics['timepoints']['cv'][g][s]:.4f}")


# 、保存
df_clean.to_excel(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\第三问\完
整分析结果 2.xlsx', index=False)

stability_results = {
    'bmi_boundaries_perturbed': bmi_boundaries_perturbed,
    'timepoints_perturbed': timepoints_perturbed,
    'stability_metrics': stability_metrics
}
np.savez(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\第三问
\stability_analysis.npz', **stability_results)

    print("分析完成! 结果已保存。")
```

附录 5

问题四代码呈现

```
    import pandas as pd
import numpy as np
import lightgbm as lgb
from sklearn.model_selection import GroupKFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, log_loss, roc_auc_score,
average_precision_score, recall_score, precision_score, confusion_matrix,
ConfusionMatrixDisplay
import warnings
from scipy.optimize import minimize_scalar
import matplotlib.pyplot as plt
import os

# 忽略警告
warnings.filterwarnings("ignore")

df = pd.read_excel(r'D:\2024——2025 仇翰辰\数学\数学建模\2025 竞赛\第四问\女
胎处理后数据.xlsx')

# 1、、数据预处理
df['孕妇代码'] = df['孕妇代码'].apply(lambda x: int(x[1:]) if isinstance(x,
```

```
str) and x.startswith('B') else x)
df['孕妇代码'] = pd.to_numeric(df['孕妇代码'],
errors='coerce').fillna(0).astype(int)
df['怀孕次数'] = pd.to_numeric(df['怀孕次数'], errors='coerce').fillna(0)

df['IVF 妊娠'] = df['IVF 妊娠'].apply(lambda x: 1 if x == '自然受孕' else 0)

df['13 号染色体的 Z 值_abs'] = df['13 号染色体的 Z 值'].abs()
df['18 号染色体的 Z 值_abs'] = df['18 号染色体的 Z 值'].abs()
df['21 号染色体的 Z 值_abs'] = df['21 号染色体的 Z 值'].abs()

# 特征
excluded_features = [
    '序号', '末次月经', '检测日期', 'Unnamed: 21', 'Unnamed: 22',
'X 染色体的 Z 值',
        'X 染色体浓度',
 '13 号染色体的 GC 含量', '18 号染色体的 GC 含量', '21 号染色体的 GC 含量',
    '染色体的非整倍体'
]

#数据质量特征
quality_features = [
    '原始读段数', '在参考基因组上比对的比例',
'重复读段的比例',
    '唯一比对的读段数',
 '被过滤掉读段数的比例', 'GC 含量'
]

#人群特征
demographic_features = [
    '年龄', '身高', '体重',
'IVF 妊娠', '检测孕周', '当周天数',
  '孕妇 BMI',
  '怀孕次数', '生产次数'
]

# Z 值特征
z_value_features = [
    '13 号染色体的 Z 值', '18 号染色体的 Z 值', '21 号染色体的 Z 值',
 '13 号染色体的 Z 值_abs',
'18 号染色体的 Z 值_abs', '21 号染色体的 Z 值_abs'
]

feature_columns = quality_features + demographic_features + z_value_features
```

```
#@###############3  准备训练数据
X = df[feature_columns]

df['染色体的非整倍体'] = df['染色体的非整倍体'].fillna('正常')

df['y_any'] = df['染色体的非整倍体'].apply(
    lambda x: 0 if x == '正常' else 1
)

df['y_13'] = df['染色体的非整倍体'].apply(
    lambda x: 1 if 'T13' in str(x) else 0
)
df['y_18'] = df['染色体的非整倍体'].apply(
    lambda x: 1 if 'T18' in str(x) else 0
)
df['y_21'] = df['染色体的非整倍体'].apply(
    lambda x: 1 if 'T21' in str(x) else 0
)

y_multi = df[['y_any', 'y_13', 'y_18', 'y_21']]
groups = df['孕妇代码'].values  # 用于 GroupKFold 分组

# 【4】计算类别权重
def calculate_class_weights(y):
    n_samples = len(y)
    n_classes = len(np.unique(y))
    class_counts = np.bincount(y)
    weights = n_samples / (n_classes * class_counts)
    return weights

w_any = calculate_class_weights(df['y_any'])
w_13 = calculate_class_weights(df['y_13'])
w_18 = calculate_class_weights(df['y_18'])
w_21 = calculate_class_weights(df['y_21'])

####--------------------5, 用 GroupKFold 进行交叉验证
n_splits = min(5, len(np.unique(groups)))
gkf = GroupKFold(n_splits=n_splits)

all_predictions = []
all_performance = []
all_chromosome_performance = []  # 存储染色体模型的性能
final_reliability = None  # 全局可靠性分数
```

```
monotone_constraints = [0] * len(X.columns)
for i, col in enumerate(X.columns):
    if 'Z值_abs' in col:
        monotone_constraints[i] = 1  # 正单调约束
params = {'objective': 'binary','boosting_type': 'gbdt',
'metric': 'binary_logloss',
'num_leaves': 31,'learning_rate': 0.05,'feature_fraction': 0.9,'verbose': -1,
    'monotone_constraints': monotone_constraints,'min_child_samples': 3,
 'reg_alpha': 0.05,'reg_lambda': 0.05,'scale_pos_weight': w_any[1]/w_any[0]
if len(w_any) > 1 else 1
}

print("开始GroupKFold交叉验证...")
print(f"分组数: {len(np.unique(groups))}, 折数: {n_splits}")

if n_splits < 2:
    print("警告: 分组数不足，无法进行交叉验证。")
else:
    folds = list(gkf.split(X, df['y_any'], groups=groups))

for fold, (train_idx, val_idx) in enumerate(folds):
    print(f"处理第 {fold+1}/{len(folds)} 折...")

    # 确保训练集和验证集不为空
    if len(train_idx) == 0 or len(val_idx) == 0:
        print(f"警告: 第 {fold+1} 折的训练集或验证集为空，跳过此折。")
        continue
    ####划分训练集和验证集
    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
    y_train_any, y_val_any = df['y_any'].iloc[train_idx],
df['y_any'].iloc[val_idx]

    # Step1'、训练顶层模型
    dtrain = lgb.Dataset(X_train, label=y_train_any)
    dval = lgb.Dataset(X_val, label=y_val_any)

    model_any = lgb.train(
            params,
        dtrain,
           num_boost_round=200,
         valid_sets=[dval],
   callbacks=[lgb.early_stopping(20)]
        )
```

```
    # Step2 细分层
    chromosome_models = {}
    chromosome_performance = {}

    for chromosome in ['13', '18', '21']:
        y_col = f'y_{chromosome}'
        y_train_chr = df[y_col].iloc[train_idx]
        y_val_chr = df[y_col].iloc[val_idx]

        weights_chr = calculate_class_weights(y_train_chr)
        params_chr = params.copy()
        params_chr['scale_pos_weight'] = weights_chr[1]/weights_chr[0] if
len(weights_chr) > 1 else 1

            # 训练染色体模型
        dtrain_chr = lgb.Dataset(X_train, label=y_train_chr)
        dval_chr = lgb.Dataset(X_val, label=y_val_chr)

        model_chr = lgb.train(
                params_chr,
        dtrain_chr,num_boost_round=200,
   valid_sets=[dval_chr],callbacks=[lgb.early_stopping(20)]
            )

            # 预测和评估
        preds_chr = model_chr.predict(X_val)

            # Platt 校准
        lr_chr = LogisticRegression()
        lr_chr.fit(preds_chr.reshape(-1, 1), y_val_chr)
        calibrated_preds_chr = lr_chr.predict_proba(preds_chr.reshape(-1,
1))[:, 1]

        binary_preds_chr = (calibrated_preds_chr > 0.5).astype(int)

        chr_performance = {
                'roc_auc': roc_auc_score(y_val_chr,
calibrated_preds_chr),'pr_auc': average_precision_score(y_val_chr,
calibrated_preds_chr),
   'recall': recall_score(y_val_chr, binary_preds_chr),
  'precision': precision_score(y_val_chr, binary_preds_chr)
            }
```

```
        chromosome_models[chromosome] = {
                'model': model_chr,'calibrator': lr_chr,
'performance': chr_performance
            }
        chromosome_performance[chromosome] = chr_performance

    # 获取顶层模型的原始预测概率
    raw_preds = model_any.predict(X_val)

    # step3: 《Platt 校准》
    try:
        lr = LogisticRegression()
        lr.fit(raw_preds.reshape(-1, 1), y_val_any)
        calibrated_preds = lr.predict_proba(raw_preds.reshape(-1, 1))[:, 1]
    except Exception as e:
        print(f"Platt 校准失败: {e}")
        continue

    # 4: 设定基线阈值
    alpha = 0.01
    normal_preds = calibrated_preds[y_val_any == 0]
    if len(normal_preds) > 0:
        tau_0 = np.quantile(normal_preds, 1 - alpha)
    else:
        tau_0 = 0.5

    #5、计算可靠性分数
    try:
        Zx_abs = np.abs(df['X 染色体的 Z 值'].iloc[val_idx].values)
        Xc = df['X 染色体浓度'].iloc[val_idx].values
        mu_X = np.mean(df['X 染色体浓度'].iloc[train_idx].values) if
len(train_idx) > 0 else 0.5

        GC_i = df['GC 含量'].iloc[val_idx].values
        GC_13 = df['13 号染色体的 GC 含量'].iloc[val_idx].values
        GC_18 = df['18 号染色体的 GC 含量'].iloc[val_idx].values
        GC_21 = df['21 号染色体的 GC 含量'].iloc[val_idx].values

        GC_deviation = (np.abs(GC_i - 0.4) + np.abs(GC_13 - 0.4) +
                        np.abs(GC_18 - 0.4) + np.abs(GC_21 - 0.4)) / 4

        # 大幅降低惩罚权重以提高 Recall
        a1, a2, a3 = 0.02, 0.02, 0.02
```

```
        reliability_scores = np.clip(
            1 - a1 * Zx_abs - a2 * np.abs(Xc - mu_X) - a3 * GC_deviation,
            0, 1
        )
    except Exception as e:
        print(f"计算可靠性分数失败: {e}")
        reliability_scores = np.ones(len(val_idx))

    #6: 自适应阈值计算 - 重点调整以提高 Recall
    def objective(kappa):
            adaptive_thresholds = tau_0 + kappa * (1 - reliability_scores)
            y_pred_adaptive = (calibrated_preds >=
adaptive_thresholds).astype(int)

            tn = np.sum((y_pred_adaptive == 0) & (y_val_any == 0))
            fp = np.sum((y_pred_adaptive == 1) & (y_val_any == 0))
            tp = np.sum((y_pred_adaptive == 1) & (y_val_any == 1))
            fn = np.sum((y_pred_adaptive == 0) & (y_val_any == 1))

            fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0

            if fpr > alpha:
                return -recall + 0.5 * (fpr - alpha)
            else:
                return -recall

    result = minimize_scalar(objective, bounds=(0, 20), method='bounded')
    best_kappa = result.x if result.success else 0

    adaptive_thresholds = tau_0 + best_kappa * (1 - reliability_scores)
    y_pred_adaptive = (calibrated_preds >= adaptive_thresholds).astype(int)

    recall_adaptive = recall_score(y_val_any, y_pred_adaptive)
    if recall_adaptive < 0.1:
            adaptive_thresholds = np.maximum(adaptive_thresholds * 0.8, tau_0
* 0.5)
            y_pred_adaptive = (calibrated_preds >=
adaptive_thresholds).astype(int)
            best_kappa = best_kappa * 0.8

#
    y_pred_fixed = (calibrated_preds >= tau_0).astype(int)
```

```
    try:
        performance_metrics = {
            'fold': fold,
'kappa': best_kappa,
   'tau_0': tau_0,
            'fixed_threshold_recall': recall_score(y_val_any, y_pred_fixed),
'adaptive_threshold_recall': recall_score(y_val_any,
y_pred_adaptive),'roc_auc': roc_auc_score(y_val_any,
calibrated_preds),'pr_auc': average_precision_score(y_val_any,
calibrated_preds),
            'fixed_threshold_fpr': np.sum((y_pred_fixed == 1) & (y_val_any ==
0)) / max(1, np.sum(y_val_any == 0)),
   'adaptive_threshold_fpr': np.sum((y_pred_adaptive == 1) & (y_val_any == 0))
/ max(1, np.sum(y_val_any == 0))
        }
    except Exception as e:
        print(f"计算性能指标失败: {e}")
        performance_metrics = {
            'fold': fold,
   'kappa': best_kappa,
 'tau_0': tau_0,'fixed_threshold_recall': np.nan,
'adaptive_threshold_recall': np.nan,'roc_auc': np.nan,
         'pr_auc': np.nan,
  'fixed_threshold_fpr': np.nan,'adaptive_threshold_fpr': np.nan
        }

    # 存储当前折的结果
    all_performance.append(performance_metrics)
    all_chromosome_performance.append(chromosome_performance)

    fold_results = {
        'fold': fold,
        'val_idx': val_idx,
        'calibrated_preds': calibrated_preds,
        'tau_0': tau_0,'kappa': best_kappa,
        'y_pred_adaptive': y_pred_adaptive,'reliability_scores':
reliability_scores,'y_val_any': y_val_any.values  # 存储真实标签
    }

    all_predictions.append(fold_results)

    print(f"第 {fold+1} 折完成 - 固定阈值 Recall:
{performance_metrics['fixed_threshold_recall']:.4f}, "
          f"自适应阈值 Recall:
```

```
{performance_metrics['adaptive_threshold_recall']:.4f}")

if len(all_predictions) == 0:
    print("错误: 没有成功完成任何折的训练和验证。")
    exit()

# 7、汇总所有折的结果
try:
    all_val_idx = np.concatenate([result['val_idx'] for result in
all_predictions])
    all_calibrated_preds = np.concatenate([result['calibrated_preds'] for
result in all_predictions])
    all_adaptive_preds = np.concatenate([result['y_pred_adaptive'] for result
in all_predictions])
    all_reliability = np.concatenate([result['reliability_scores'] for result
in all_predictions])
    all_y_val_any = np.concatenate([result['y_val_any'] for result in
all_predictions])

    # 按原始索引排序
    sorted_idx = np.argsort(all_val_idx)
    final_calibrated_preds = all_calibrated_preds[sorted_idx]
    final_adaptive_preds = all_adaptive_preds[sorted_idx]
    final_reliability = all_reliability[sorted_idx]
    final_y_val_any = all_y_val_any[sorted_idx]

    # 存储全局可靠性分数
    final_reliability = final_reliability
except Exception as e:
    print(f"汇总结果失败: {e}")
    exit()

#####8，计算平均性能指标
avg_performance = {}
for metric in all_performance[0].keys():
    if metric not in ['fold', 'kappa', 'tau_0']:
        values = [p[metric] for p in all_performance if not np.isnan(p[metric])]
        avg_performance[metric] = np.mean(values) if values else np.nan

avg_chr_performance = {}
for chromosome in ['13', '18', '21']:
    try:
        roc_aucs = [p[chromosome]['roc_auc'] for p in
all_chromosome_performance if not np.isnan(p[chromosome]['roc_auc'])]
```

```
        pr_aucs = [p[chromosome]['pr_auc'] for p in all_chromosome_performance
if not np.isnan(p[chromosome]['pr_auc'])]
        recalls = [p[chromosome]['recall'] for p in all_chromosome_performance
if not np.isnan(p[chromosome]['recall'])]
        precisions = [p[chromosome]['precision'] for p in
all_chromosome_performance if not np.isnan(p[chromosome]['precision'])]

        avg_chr_performance[chromosome] = {
            'roc_auc': np.mean(roc_aucs) if roc_aucs else np.nan,
            'pr_auc': np.mean(pr_aucs) if pr_aucs else np.nan,
            'recall': np.mean(recalls) if recalls else np.nan,
            'precision': np.mean(precisions) if precisions else np.nan
        }
    except Exception as e:
        print(f"计算{chromosome}号染色体性能失败: {e}")
        avg_chr_performance[chromosome] = {
            'roc_auc': np.nan,
            'pr_auc': np.nan,
            'recall': np.nan,
            'precision': np.nan
        }

print("\n=== 最终结果 ===")
print(f"校准后的预测概率范围: [{final_calibrated_preds.min():.4f},
{final_calibrated_preds.max():.4f}]")
print(f"可靠性分数范围: [{final_reliability.min():.4f},
{final_reliability.max():.4f}]")

print("\n=== 顶层模型平均性能指标 ===")
print(f"ROC-AUC: {avg_performance.get('roc_auc', np.nan):.4f}")
print(f"PR-AUC: {avg_performance.get('pr_auc', np.nan):.4f}")
print(f"固定阈值 Recall: {avg_performance.get('fixed_threshold_recall',
np.nan):.4f}")
print(f"自适应阈值 Recall: {avg_performance.get('adaptive_threshold_recall',
np.nan):.4f}")
print(f"固定阈值 FPR: {avg_performance.get('fixed_threshold_fpr',
np.nan):.4f}")
print(f"自适应阈值 FPR: {avg_performance.get('adaptive_threshold_fpr',
np.nan):.4f}")

print("\n=== 染色体模型平均性能指标 ===")
for chromosome in ['13', '18', '21']:
    print(f"\n{chromosome}号染色体:")
    print(f"  ROC-AUC: {avg_chr_performance[chromosome]['roc_auc']:.4f}")
```

```
    print(f"  PR-AUC: {avg_chr_performance[chromosome]['pr_auc']:.4f}")
    print(f"  Recall: {avg_chr_performance[chromosome]['recall']:.4f}")
    print(f"  Precision:
{avg_chr_performance[chromosome]['precision']:.4f}")

#可视化
print("\n开始生成可视化图表...")

output_dir = r"D:\2024——2025仇翰辰\数学\数学建模\2025竞赛\图片集\第4问"
os.makedirs(output_dir, exist_ok=True)

    ##【1】 性能指标汇总图
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
fig.suptitle('模型性能指标汇总', fontsize=16, fontweight='bold')

#顶层模型性能指标
metrics = ['ROC-AUC', 'PR-AUC']
values = [avg_performance.get('roc_auc', 0.63), avg_performance.get('pr_auc',
0.17)]
bars = axes[0, 0].bar(metrics, values, color=['#1f77b4', '#ff7f0e'])
axes[0, 0].set_title('顶层模型性能指标')
axes[0, 0].set_ylabel('值')
axes[0, 0].set_ylim(0, 1)
for bar, value in zip(bars, values):
    axes[0, 0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                    f'{value:.2f}', ha='center', va='bottom')

#Recall对比
metrics = ['固定阈值', '自适应阈值']
values = [avg_performance.get('fixed_threshold_recall', 0.25),
avg_performance.get('adaptive_threshold_recall', 0)]
bars = axes[0, 1].bar(metrics, values, color=['#1f77b4', '#ff7f0e'])
axes[0, 1].set_title('Recall对比')
axes[0, 1].set_ylabel('Recall值')
axes[0, 1].set_ylim(0, 1)
for bar, value in zip(bars, values):
    axes[0, 1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                    f'{value:.2f}', ha='center', va='bottom')

#ROC-AUC
chromosomes = ['13号', '18号', '21号']
roc_aucs = [
    avg_chr_performance['13']['roc_auc'] if '13' in avg_chr_performance else
0.57,
```

```
    avg_chr_performance['18']['roc_auc'] if '18' in avg_chr_performance else
0.59,
    avg_chr_performance['21']['roc_auc'] if '21' in avg_chr_performance else
0.72
]
bars = axes[1, 0].bar(chromosomes, roc_aucs, color=['#1f77b4', '#ff7f0e',
'#2ca02c'])
axes[1, 0].set_title('染色体模型 ROC-AUC')
axes[1, 0].set_ylabel('ROC-AUC 值')
axes[1, 0].set_ylim(0, 1)
for bar, value in zip(bars, roc_aucs):
    axes[1, 0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                    f'{value:.2f}', ha='center', va='bottom')

#FPR 对比
metrics = ['固定阈值 FPR', '自适应阈值 FPR']
values = [avg_performance.get('fixed_threshold_fpr', 0.05),
avg_performance.get('adaptive_threshold_fpr', 0)]
bars = axes[1, 1].bar(metrics, values, color=['#1f77b4', '#ff7f0e'])
axes[1, 1].set_title('FPR 对比')
axes[1, 1].set_ylabel('FPR 值')
axes[1, 1].set_ylim(0, 0.1)
for bar, value in zip(bars, values):
    axes[1, 1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.001,
                    f'{value:.3f}', ha='center', va='bottom')

plt.tight_layout()
plt.savefig(os.path.join(output_dir, "model_performance_summary.png"),
dpi=300, bbox_inches='tight')
plt.close()
print("已生成性能汇总图表: model_performance_summary.png")

#######========#《分析
if final_reliability is not None:
    ##可靠性分数分布
    plt.figure(figsize=(10, 6))
    plt.hist(final_reliability, bins=20, alpha=0.7, color='purple')
    plt.xlabel('可靠性分数')
    plt.ylabel('样本数量')
    plt.title('可靠性分数分布')
    plt.grid(True, alpha=0.3)
    plt.savefig(os.path.join(output_dir,
"reliability_score_distribution.png"), dpi=300, bbox_inches='tight')
    plt.close()
```

```
    ##可靠性分数与预测准确率关系
    reliability_bins = np.linspace(0, 1, 10)
    reliability_means = []
    accuracy_means = []
    for i in range(len(reliability_bins)-1):
        mask = (final_reliability >= reliability_bins[i]) & (final_reliability
< reliability_bins[i+1])
        if np.sum(mask) > 0:
            reliability_means.append((reliability_bins[i] +
reliability_bins[i+1]) / 2)
            accuracy_means.append(np.mean(final_y_val_any[mask] ==
final_adaptive_preds[mask]))

    plt.figure(figsize=(10, 6))
    plt.plot(reliability_means, accuracy_means, 'o-', color='purple',
linewidth=2)
    plt.xlabel('可靠性分数')
    plt.ylabel('预测准确率')
    plt.title('可靠性分数与预测准确率关系')
    plt.grid(True, alpha=0.3)
    plt.ylim(0, 1)
    plt.savefig(os.path.join(output_dir, "reliability_vs_accuracy.png"),
dpi=300, bbox_inches='tight')
    plt.show()
    print("已生成可靠性分数分析图表")
else:
    print("警告: final_reliability 未定义或为空，无法生成可靠性分数分析图表
")

#混淆矩阵
plt.figure(figsize=(8, 6))
cm = confusion_matrix(final_y_val_any, final_adaptive_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['正常', '
异常'])
disp.plot(cmap='Blues')
plt.title('混淆矩阵 - 模型决策与真实情况对比')
plt.savefig(os.path.join(output_dir, "confusion_matrix.png"), dpi=300,
bbox_inches='tight')
plt.show()
print("已生成混淆矩阵图表: confusion_matrix.png")

# 预测概率分布
plt.figure(figsize=(10, 6))
```

```
bins = np.linspace(0, 1, 20)
plt.hist(final_calibrated_preds[final_y_val_any == 0], bins=bins, alpha=0.7,
label='真实正常', color='green')
plt.hist(final_calibrated_preds[final_y_val_any == 1], bins=bins, alpha=0.7,
label='真实异常', color='red')
plt.axvline(x=np.mean([p['tau_0'] for p in all_performance]), color='black',
linestyle='--', label=f'平均基线阈值')
plt.xlabel('预测概率')
plt.ylabel('样本数量')
plt.title('预测概率分布')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig(os.path.join(output_dir,
"prediction_probability_distribution.png"), dpi=300, bbox_inches='tight')
plt.show()
print("已生成预测概率分布图表: prediction_probability_distribution.png")

#自适应阈值分布
plt.figure(figsize=(10, 6))
threshold_bins = np.linspace(0, 1, 20)
plt.hist(final_reliability[final_y_val_any == 0], bins=threshold_bins,
alpha=0.7, label='真实正常', color='green')
plt.hist(final_reliability[final_y_val_any == 1], bins=threshold_bins,
alpha=0.7, label='真实异常', color='red')
plt.axvline(x=np.mean([p['tau_0'] for p in all_performance]), color='black',
linestyle='--', label=f'平均基线阈值')
plt.xlabel('可靠性分数')
plt.ylabel('样本数量')
plt.title('可靠性分数分布')
plt.legend()
plt.grid(True, alpha=0.3)
#plt.savefig(os.path.join(output_dir,
"reliability_score_distribution_by_class.png"), dpi=300,
bbox_inches='tight')
plt.show()
print("已生成可靠性分数分布图表:
reliability_score_distribution_by_class.png")


performance_data = {
    '指标': ['ROC-AUC', 'PR-AUC', '固定阈值 Recall', '自适应阈值 Recall',
            '固定阈值 FPR', '自适应阈值 FPR', '准确率', '精确率', '召回率', 'F1
分数'],
    '值': [
```

```
        avg_performance.get('roc_auc', 0.63),
        avg_performance.get('pr_auc', 0.17),
        avg_performance.get('fixed_threshold_recall', 0.25),
        avg_performance.get('adaptive_threshold_recall', 0),
        avg_performance.get('fixed_threshold_fpr', 0.05),
        avg_performance.get('adaptive_threshold_fpr', 0),
        np.mean(final_y_val_any == final_adaptive_preds),
        precision_score(final_y_val_any, final_adaptive_preds),
        recall_score(final_y_val_any, final_adaptive_preds),
        2 * precision_score(final_y_val_any, final_adaptive_preds) *
recall_score(final_y_val_any, final_adaptive_preds) /
        (precision_score(final_y_val_any, final_adaptive_preds) +
recall_score(final_y_val_any, final_adaptive_preds) + 1e-10)
    ]
}

performance_df = pd.DataFrame(performance_data)
print("\n=== 模型性能指标汇总 ===")
print(performance_df.round(4))

performance_df.to_csv(os.path.join(output_dir, "performance_metrics.csv"),
index=False)
    print("已保存性能指标表格: performance_metrics.csv")
```

附录 6

PCA 结果表

	M	N	O	|P-0.4|	AA	log_L
PC1	-0.178529009	0.05565608	-0.864716143	0.120581174	-0.360868663	-0.874577181
PC2	0.776269385	-0.502987734	0.007922963	-0.393886049	-0.487781045	-0.05134183
PC3	-0.156187458	-0.737309534	-0.184530356	-0.143007997	0.636305918	-0.114858161
PC4	0.292923758	-0.249341629	0.056115602	0.898043837	-0.000219747	-0.007238187

附录 7

问题四结果表完整版展示

孕妇代码	真实标签	校准概率	可靠性分数	自适应阈值	预测标签
1	0	0.148300086031135	0.97630432518338	0.527989955249669	0

		4	84		
1	0	0.1488518078712099	0.9659924829833885	0.6929793756422433	0
1	0	0.14884644288544222	0.9772843204433884	0.5123100362983416	0
1	0	0.1488518078712099	0.9700948496383884	0.627341530966316	0
2	0	0.1322527273478532	0.9809442397914876	0.43714485427784955	0
2	0	0.1322527273478532	0.9668573266885124	0.6625353890535322	0
2	0	0.1322527273478532	0.9648518422114876	0.6946231300267833	0
2	0	0.1322527273478532	0.9453644308435123	1.0064216083388282	0
3	0	0.09087102112486789	0.9986454318261571	0.11258193176319742	0
3	0	0.09087102112486789	0.9883748104838429	0.27691181865188835	0
3	0	0.09087102112486789	0.9605089454811571	0.7227655105878351	0
3	0	0.09087102112486789	0.9463908027961577	0.9486557185099317	0
4	0	0.07439192628717135	0.9784664240128926	0.4189306222992371	0
4	0	0.07439321630276277	0.9769517358321075	0.44316562514123437	0
4	1	0.07431045889570058	0.9822867208271074	0.3578058935766715	0
4	1	0.07439321630276277	0.9903669543821074	0.22852219964309983	0
5	0	0.10532114910429925	0.9985865072409917	0.14906405400912048	0
5	0	0.10513932385495937	0.9718758351559917	0.5764346654019422	0
5	0	0.11571429372078773	0.9948465963590083	0.2089026082432354	0
5	1	0.1147675025908189	0.9772454395340082	0.490521023893117	0
6	0	0.14885922196571505	0.9654003756066115	0.7024530905236248	0
6	0	0.14885411483081104	0.9898715040166115	0.31091516602763236	0
6	0	0.14857348567452086	0.9656758135066116	0.6980460855875759	0
6	0	0.14857348567452086	0.9921710309366116	0.2741227475296121	0
7	0	0.14857348567452	0.96177972387161	0.760383499039867	0

		86	15	5	
7	1	0.14885922196571505	0.9774521599583884	0.5096246049504085	0
7	0	0.14821816806490334	0.9936403964116116	0.25061290773928746	0
7	0	0.14821816806490334	0.9698932954933884	0.6305663962150561	0
7	0	0.14857348567452086	0.9714175397233884	0.6061784966364121	0
7	0	0.1488559786469931	0.9927454431633884	0.26493215495418354	0
8	0	0.13225116166977138	0.9956812365035124	0.20135298521256648	0
8	0	0.13189368229595513	0.9824376165035125	0.4132508348227463	0
8	0	0.13189368229595513	0.9762087959264876	0.5129119309489736	0
8	0	0.1322527273478532	0.9658165761014876	0.6791873929143413	0
9	0	0.1484362268191746	0.9707178904783884	0.6173728808377773	0
9	0	0.1484362268191746	0.9509745325783885	0.9332665023018614	0
9	0	0.14885922196571505	0.9971434971966116	0.1945633137982634	0
9	0	0.14857348567452086	0.9928186583616115	0.26376071217175223	0
9	0	0.14857348567452086	0.9781977885416114	0.4976945515818548	0
10	0	0.09090883536957067	0.9959057402638429	0.15641698219876732	0
10	0	0.09087102112486789	0.9909434078338429	0.2358142747039785	0
10	0	0.09090883536957067	0.9671004843288429	0.6173009240588802	0
10	0	0.09087102112486789	0.9926460036538433	0.20857275063327094	0
11	0	0.09087102112486789	0.9924493832788433	0.21171867558823415	0
11	0	0.09083500632828126	0.9920937342961577	0.21740905742093686	0
11	1	0.09090883536957067	0.9895290701511569	0.258443670109753	0
11	0	0.09087102112486789	0.9830543759161571	0.36203874345676057	0
11	0	0.09087102112486789	0.9968725095161577	0.1409486793001188	0
11	0	0.09087102112486	0.99831919278115	0.117801754749236	0

		89	71	68	
12	0	0.07439351803786746	0.9542218559928927	0.8068435817593974	0
12	0	0.07431045889570058	0.9656354509721075	0.6242261227551993	0
12	0	0.0741998531357335	0.9599315816371076	0.7154880017991414	0
12	0	0.07439300437030362	0.9744192976171074	0.4836846231213301	0
13	0	0.1027225790834334	0.9992747785890084	0.13805171609901545	0
13	0	0.10206375862017368	0.9909104309409916	0.27188123401078934	0
13	0	0.10149075717338324	0.9300796575309918	1.2451732852553266	0
13	0	0.10064512410790684	0.9284150616190083	1.2718068109997367	0
14	0	0.09087102112486789	0.9898860310488843	0.25273229764402183	0
14	0	0.09087102112486789	0.9959483512961567	0.15573520590821982	0
14	0	0.09090883536957067	0.9887946945538443	0.2701936757635708	0
14	0	0.09085201590034411	0.9881807357911577	0.2800170127033569	0
14	1	0.09090883536957067	0.9956251927638443	0.160905740707657	0
14	0	0.09087102112486789	0.9921225056961571	0.21694871517385478	0
15	0	0.1488592841263559	0.9862594131783885	0.36870860024094354	0
15	0	0.1488592841263559	0.9686530711766116	0.6504099786916953	0
15	0	0.1488592841263559	0.9985034950766115	0.17280335494665047	0
15	0	0.1488592841263559	0.9838381772416115	0.4074483623605107	0
16	0	0.13225279222298525	0.9794495192885123	0.46106037438101694	0
16	1	0.13225279222298525	0.9914528283335123	0.2690074934585845	0
16	1	0.13225279222298525	0.9861333635535123	0.354118901665638	0
16	0	0.13225279222298525	0.9736479497885123	0.5538854555456845	0
17	0	0.07431045889570058	0.9260676748821074	1.2573103298925354	0
17	1	0.07431045889570	0.9626533281571	0.671940071945223	0

		58	74		
17	0	0.07419414826761338	0.9743998223878927	0.48399622668525527	0
18	0	0.09087102112486789	0.953794508368843	0.8301964686976371	0
18	1	0.09083561324515703	0.9780410881588429	0.44225132092817054	0
18	1	0.09087102112486789	0.9919619517038429	0.2195175781975375	0
18	0	0.09083561324515703	0.990511244253843	0.2427288896870294	0
19	0	0.07439351803786746	0.9864964123471074	0.29045085163117584	0
19	0	0.07439321630276277	0.9854314558071074	0.30749015061093266	0
19	0	0.07439321630276277	0.9892562463821075	0.24629352173968716	0
19	0	0.07431045889570058	0.9973123396771074	0.11739607183781137	0
20	0	0.14857348567452086	0.9810263360183885	0.452437806987146 8	0
20	1	0.14857348567452086	0.9838619248083883	0.4070684014183	0
20	1	0.14857348567452086	0.9719159056716117	0.5982046441136538	0
20	0	0.14857348567452086	0.9750151538166116	0.5486166902661533	0
21	0	0.10180239328538106	0.9753064242809917	0.5215452576355187	0
21	0	0.10727578839792631	0.9840483184390083	0.3816749975704058	0
21	0	0.10656101036517471	0.9916953259459917	0.2593229181025031	0
21	0	0.10204966781128841	0.9913200059709917	0.2653280357076791 6	0
22	0	0.1322494894943884	0.9890431079764875	0.3075630063633215 7	0
22	0	0.1322494894943884	0.9917297637264876	0.2645765286428918	0
22	0	0.1322494894943884	0.9810906295314876	0.4348026192159103	0
22	1	0.1322516689169019	0.9978976777385123	0.1658899372329483 7	0
23	0	0.09087102112486789	0.9361047175238429	1.1132330281964293	0
23	1	0.09090866795431153	0.9529202653538429	0.8441843522910395	0
23	0	0.09087102112486 7	0.9843303112588 4	0.3416237847553 80	0

		89	3	7	
23	1	0.09087102112486789	0.988646131153843	0.2725706893739552	0
24	1	0.07439351803786746	0.9645411174171074	0.6417354538188199	0
24	1	0.07439351803786746	0.9685093546721075	0.5782436788299916	0
24	0	0.07431045889570058	0.9785247527321075	0.4179973631018157	0
24	0	0.0743907925464684	0.9726213348371074	0.5124520180451608	0
25	1	0.1105478848523918	0.9827034776390082	0.4031924432225804	0
25	1	0.11899864563806171	0.9840089058690082	0.38230559848092904	0
25	1	0.12307540756230627	0.9902320936840082	0.28273462651716247	0
25	0	0.10969730245832986	0.9792107738809918	0.45907568478712923	0
26	0	0.1481623637801881	0.9486030197616115	0.971210694765705	0
26	0	0.14885922196571505	0.9750889187383885	0.5474364519097832	0
26	0	0.14857348567452086	0.9596023839633884	0.795220925998879	0
26	0	0.14857348567452086	0.9763740900016116	0.5268737185288979	0
27	0	0.1322527273478532	0.9876436240614876	0.3299547415650651	0
27	0	0.1322494894943884	0.9887084198864875	0.3129180140244555	0
27	1	0.1322494894943884	0.9901830817914876	0.2893234313822792 3	0
27	0	0.13224683191356737	0.9807958001214876	0.4395198882088929 3	0
28	0	0.09087102112486789	0.975909874308843	0.4763507312007712	0
28	0	0.09087102112486789	0.9960564676188429	0.15400534531988308	0
28	1	0.09090837631784783	0.990280978838843	0.246413135103168	0
28	0	0.09083500632828126	0.986639719773843	0.3046732607898827	0
29	0	0.07439351803786746	0.9840856930328925	0.3290223478456447 7	0
29	0	0.07431045889570058	0.9952862422721074	0.1498136195491076	0
29	0	0.074194148267613	0.99863776257710	0.096189312482433	0

		38	73	06	
29	0	0.07431045889570058	0.9900724417378926	0.2332344003852036	0
30	0	0.09087102112486789	0.9986364094788843	0.11272628927226823	0
30	0	0.09087102112486789	0.9797836896238443	0.41436970675009366	0
30	0	0.09083561324515703	0.9714261720388443	0.5480899436899019	0
30	0	0.09090866795431153	0.9416416064388431	1.0246428349849825	0
30	0	0.09087102112486789	0.9428976645788429	1.0045459114209319	0
30	0	0.09090866795431153	0.9225860641988429	1.3295314095448094	0
31	0	0.10737416747542035	0.9882208316109916	0.31491480899557356	0
31	1	0.11099151696887152	0.9725741215940084	0.5652620861050676	0
31	0	0.10199826453394543	0.9493592353140082	0.9367001431979871	0
31	0	0.10686151069215451	0.9888884211590083	0.30423337977554354	0
32	0	0.07439173278671102	0.9814926968878925	0.370510272383874	0
32	0	0.0741998531357335	0.9965076300321075	0.13027142188077936	0
32	0	0.07439243255500602	0.9749281287371074	0.475543327905767	0
32	0	0.074305198949091	0.9696874661778925	0.5593939009990931	0
32	0	0.07439352095644026	0.9867932169578926	0.28570197943612896	0
32	0	0.07439300437030362	0.9781692812721074	0.42368490457248686	0
33	0	0.1488592841263559	0.9880866958333885	0.3394720874729481	0
33	0	0.1488592841263559	0.9552814341633884	0.8643560998330375	0
33	0	0.1488592841263559	0.9669769758683884	0.6772274947148229	0
33	0	0.1488592841263559	0.9617373426883884	0.7610615977461808	0
34	0	0.13225188414883224	0.9787596119435125	0.4720988882341582	0
34	0	0.13225188414883224	0.9771570558035125	0.49773977795657387	0
34	0	0.132252792222985	0.9962029024114 8	0.193006333457616	0

		25	76	72	
34	0	0.13210456380287194	0.9783885255335124	0.47803626882183536	0
35	0	0.07439173278671102	0.9839127888521074	0.33178881381922065	0
35	0	0.07418450838598725	0.9743256350121074	0.4851832243035133	0
35	0	0.07422309096241639	0.9817382040171073	0.36658215962130747	0
35	0	0.07439321630276277	0.9931648134821075	0.18375646891371514	0
36	0	0.07439192628717135	0.9847184661871075	0.31889798074139386	0
36	0	0.07439321630276277	0.9847034760571075	0.31913782274172053	0
36	0	0.07431045889570058	0.9872924828121075	0.27771372842228786	0
36	0	0.07431045889570058	0.9934606453021075	0.1790231613660602	0
37	0	0.1488592841263559	0.9660665275333885	0.6917946632357894	0
37	0	0.1488592841263559	0.9580187927416115	0.8205583771305243	0
37	0	0.1488592841263559	0.9867289205866117	0.3611964842048041	0
37	0	0.1488592841263559	0.9925480933933885	0.26808975022526926	0
37	0	0.1488592841263559	0.9809028803316117	0.45441309731940965	0
37	0	0.1488592841263559	0.9940320689916115	0.2443461485410292	0
37	0	0.1488592841263559	0.9979920586866116	0.18098633446836615	0
38	0	0.14885922196571505	0.9816605001583883	0.4422911841177308	0
38	0	0.14885922196571505	0.9970004402333884	0.19685222444948616	0
38	0	0.14857348567452086	0.9949375273033884	0.2298588203651081	0
38	1	0.1488559786469931	0.9797888647733884	0.4722373403299902	0
39	0	0.11594514497583659	0.9950731861440082	0.20527717288756137	0
39	0	0.12744491977785125	0.9933528024340083	0.23280330310372382	0
39	1	0.12789302972343755	0.9943612660190083	0.2166678911037051	0
39	0	0.107502343111383	0.98064474189400	0.436132204200402	0

		25	83	2	
40	0	0.14885922196571505	0.9986932777316117	0.1697668334753428	0
40	1	0.14857348567452086	0.9909619285416116	0.29346837942323567	0
40	1	0.1488518078712099	0.9860531804183885	0.3720083233048164	0
40	1	0.1488559786469931	0.9891603970983884	0.32229287293966935	0
41	0	0.09090884818123798	0.9542029271461571	0.8236617704313579	0
41	0	0.09090884818123798	0.971706040471157	0.5436120502603767	0
41	0	0.09090884818123798	0.985934000103843	0.3159647717589833	0
41	0	0.09090884818123798	0.965842039941157	0.6374360275732239	0
42	0	0.13225116166977138	0.9593823961664876	0.7821342376766864	0
42	0	0.13225116166977138	0.9461951391964876	0.9931302791064374	0
42	0	0.13225116166977138	0.9355024069464877	1.1642139382745813	0
42	0	0.1322527273478532	0.9796619312764876	0.4576617837023886	0
42	0	0.1322527273478532	0.9928309265464876	0.24695792937556943	0
43	0	0.09090884818123798	0.9868786182911571	0.3008508857826024	0
43	0	0.09090884818123798	0.960585256171157	0.7215445399534277	0
43	0	0.09090884818123798	0.9880612633511571	0.2819285711083591 7	0
43	0	0.09090884818123798	0.9681189378011571	0.6010056739149312	0
44	0	0.07439351803786746	0.9698787456228926	0.5563334308957418	0
44	0	0.07439300437030362	0.9739887476778926	0.4905734198603959	0
44	0	0.07439352095644026	0.9830399554778925	0.345754143167552	0
44	0	0.07439352095644026	0.9927716836178926	0.1900465446516692	0
45	0	0.07439352095644026	0.9884587716171075	0.2590531137411105	0
45	0	0.07426178164396652	0.9811589238428926	0.3758506393298673	0
45	0	0.074199853135733	0.97913462013210	0.408239487943260	0

		5	75	5	
45	0	0.07439352095644026	0.9456244043128925	0.9444027629439582	0
45	0	0.07439352095644026	0.9722445181221074	0.5184810834823806	0
45	0	0.07439352095644026	0.9603919481528926	0.7081221399934275	0
46	0	0.10326482066815229	0.9889574763359917	0.3031284973108371	0
46	0	0.11196351796645225	0.9873141023140083	0.329422472928042	0
46	0	0.10938390867608656	0.9956434548090082	0.19615287727853864	0
46	0	0.10495115793793286	0.9878648623940083	0.32061031457533107	0
47	0	0.1322527273478532	0.9943333304164876	0.2229194754408435	0
47	0	0.13224683191356737	0.9744815382235125	0.5405480450162033	0
47	0	0.13224683191356737	0.9946421992435124	0.21797757585008493	0
47	0	0.13224683191356737	0.9898398718564876	0.2948147885181195	0
48	0	0.09087102112486789	0.9937570793211571	0.1907955458616114	0
48	0	0.09090837631784783	0.9972680161811577	0.13462057476223627	0
48	0	0.09087102112486789	0.9897432184911577	0.25501729780794935	0
48	0	0.09083500632828126	0.9784056115938433	0.4364189479056113	0
49	0	0.09090884818123798	0.9779785893088433	0.4432513021959896	0
49	0	0.09090884818123798	0.9894705789611571	0.25937952883886906	0
49	0	0.09090884818123798	0.9683293957938433	0.5976383471505402	0
49	0	0.09090884818123798	0.9843076294511577	0.341986693557802	0
49	0	0.09090884818123798	0.9798849646611569	0.41274930669134724	0
50	0	0.07439351803786746	0.9857688008028925	0.3020926324713594	0
50	0	0.07439321630276277	0.9884393053828926	0.2593645733850858	0
50	0	0.07431045889570058	0.9927402170728926	0.19055000920442405	0
50	0	0.07439321630276 2	0.98810176695210	0.264765186483631	0

		77	75	23	
51	0	0.1322527273478532	0.9927061159785124	0.24895489779980465	0
51	0	0.1322527273478532	0.9871853612235124	0.3372869445370026	0
51	0	0.1322527273478532	0.9653432925535124	0.6867599271664442	0
51	0	0.1322527273478532	0.9742167027214876	0.5447854116410004	0
51	0	0.1322527273478532	0.9646120617935123	0.6984596154399545	0
51	0	0.1322527273478532	0.9765585148335124	0.5073164302953315	0
52	0	0.11003206499188287	0.9714173910990083	0.5837697678770484	0
52	0	0.11530277686699053	0.9947131939190083	0.2110370465742023	0
52	0	0.12282143509260358	0.9881619619959918	0.3158567225226727	0
52	0	0.10428883239221548	0.9947643130640084	0.2102191405258968	0
52	0	0.10214328592113074	0.9764302282309918	0.5035644004085331	0
52	0	0.1033974522694755	0.9630840066009917	0.7171038755533886	0
53	0	0.11018515445170006	0.9896609188309917	0.2918734211296305	0
53	0	0.10360699997144501	0.9826423238659917	0.4041709032658125	0
53	0	0.10584883493151515	0.9834024804559918	0.3920084018660426	0
53	0	0.10406510696483197	0.9945610648709918	0.2134711105338994	0
54	0	0.07439173278671102	0.9926971978478926	0.1912383165757778	0
54	0	0.07439351803786746	0.9731683613728926	0.5036995963800428	0
54	0	0.07439351803786746	0.9708728844921075	0.540427214272149	0
54	0	0.0741998531357335	0.9867958362171074	0.2856600713026126	0
54	0	0.07439300437030362	0.9881556174521074	0.2639035787698427	0
55	0	0.10167276757335393	0.9875653230409918	0.3254029426315423	0
55	0	0.10901921189107636	0.9973017227759917	0.1696205986204953	0
56	0	0.148846442885442	0.95490865134161	0.870320623000125	0

		22	16	9	
56	0	0.1488518078712099	0.9944848681516116	0.23710136438765306	0
56	0	0.1488518078712099	0.9752229295366115	0.5452922798504831	0
56	1	0.1488592841263559	0.9720444889016115	0.5961473131170756	0
57	1	0.1078524716563023	0.9976758585709917	0.16363442788902452	0
57	1	0.11603634156605108	0.9885220899990083	0.3100946763884951	0
57	1	0.11222637679514655	0.9013320883040082	1.7051342400937795	0
57	0	0.12130285898075284	0.9585831391659917	0.7891177305912855	0
57	0	0.10561076631871559	0.9707403008009918	0.5946032090465776	0
58	1	0.13225279222298525	0.9798449795985125	0.4547330115228854	0
58	1	0.13225196988280302	0.9762655762185123	0.5120034465783652	0
58	0	0.13210456380287194	0.9691364001464876	0.626070225839201	0
58	0	0.1321203702118354	0.9629850708664875	0.724491461624897	0
59	0	0.14857348567452086	0.9639490516516116	0.725674266089839	0
59	0	0.14857348567452086	0.9880856283766115	0.33948916677570606	0
59	0	0.1488518078712099	0.9888913529066116	0.3265975785781298	0
60	0	0.09090883536957067	0.9732614326411569	0.5187257838072853	0
60	0	0.09087102112486789	0.9851745612861571	0.32811578880554193	0
60	0	0.09090866795431153	0.963958767281157	0.6675683801236327	0
60	0	0.09083561324515703	0.950013899088843	0.8906861970837051	0
61	1	0.07439351803786746	0.9963334355221074	0.13305853311493765	0
61	0	0.07431045889570058	0.9896466878571074	0.24004646021488468	0
61	0	0.07439321630276277	0.9969880880621075	0.12258409595441358	0
61	0	0.0741998531357335	0.9914241379628925	0.21160726796946722	0
62	0	0.07439173278671 1	0.9834254576378 9	0.33958611065649 3	0

		02	26	63	
62	0	0.07439351803786746	0.9575285630928926	0.7539362857345415	0
62	0	0.07420492564117377	0.9705616387328925	0.5454071447653189	0
62	0	0.07439351803786746	0.9708399916421074	0.5409534996973248	0
62	0	0.07431045889570058	0.9870243175221074	0.2820043716369908	0
62	0	0.07431045889570058	0.9866036998271074	0.2887342525214087	0
63	0	0.10093701989778327	0.9989381995040083	0.14343697967009922	0
63	0	0.10243503995983909	0.9837364783290082	0.3866644376729773	0
63	0	0.10486911161530003	0.9994021876690082	0.13601317149619668	0
63	0	0.10325832228212009	0.9797034814409917	0.45119236644586946	0
64	0	0.14857348567452086	0.9936053630733884	0.2511734409646563	0
64	0	0.14857348567452086	0.9820813968783884	0.435556838834796	0
64	0	0.14884887840954059	0.9883482850166115	0.3352866619317253	0
64	0	0.1488559786469931	0.9790045389433885	0.48478654944129956	0
65	0	0.13225188414883224	0.9874361305485124	0.3332746366698409	0
65	0	0.13186938212582935	0.9945468466335123	0.2195032171032866	0
65	0	0.13225188414883224	0.9724927384685124	0.5723688305257375	0
65	0	0.13225196988280302	0.9873250815835124	0.3350514195196163	0
66	0	0.1322527273478532	0.9795428791985125	0.4595666163172224	0
66	0	0.13224683191356737	0.9304329565364876	1.2453251178904616	0
66	0	0.13224683191356737	0.9790351546585124	0.4676902062586678	0
66	0	0.13224683191356737	0.9566783446964876	0.8253990468246568	0
66	0	0.13224683191356737	0.9575620929514875	0.8112590794417784	0
66	0	0.1322527273478532	0.9809816761435124	0.4365458728444286	0
66	0	0.13224683191356	0.96191776554648	0.741568341072170	0

		37	76	7	
66	0	0.1322527273478532	0.9394646454614877	1.1008181430938726	0
66	0	0.1322527273478532	0.9891873548114876	0.30525505776999284	0
67	0	0.10223793632696943	0.9709509347009917	0.5912330677660985	0
67	0	0.10061722595419609	0.9720753189440083	0.5732429258539327	0
67	0	0.10068012568834263	0.9829706120109917	0.3989182946906641	0
68	0	0.10770282332569818	0.9851173339709917	0.36457075474048795	0
68	0	0.11159269492767572	0.9796961134609917	0.4513102540867088	0
68	0	0.10391546652480811	0.9885017710390082	0.3104197796405014	0
68	0	0.11136564779363604	0.9827378754490084	0.40264207844540245	0
69	0	0.09090883536957067	0.9783954071188843	0.43658221945137454	0
69	0	0.09087102112486789	0.9707821213338429	0.5583947515467745	0
69	1	0.09087102112486789	0.9786085826488429	0.4331714121044039	0
69	1	0.09087102112486789	0.9673077760038433	0.6139842583606333	0
70	0	0.13189368229595513	0.9922716953635123	0.25590562533086136	0
70	0	0.13187811580900127	0.9985039730835124	0.156189214935407	0
70	0	0.13189368229595513	0.9884073492914877	0.3177351419442632	0
70	0	0.1322527273478532	0.9885940701935124	0.31474760850428835	0
71	0	0.13225196988280302	0.9960522093635124	0.19541742142428686	0
71	0	0.13225279222298525	0.9615927065235124	0.7467692837120856	0
71	0	0.13225196988280302	0.9642792175485124	0.7037851215908864	0
71	0	0.132251668916901 9	0.9737122832235124	0.5528561209276146	0
72	0	0.09090884818123798	0.9883254548938429	0.27770150782956293	0
72	0	0.09090884818123798	0.9940705870038429	0.18577942460493163	0
72	1	0.09090884818123	0.99218361785384	0.215970920975691	0

		98	3	48	
72	0	0.09090884818123798	0.992458771953843	0.21156845683813497	0
73	0	0.07439173278671102	0.9674030501221074	0.5959445457499873	0
73	0	0.07420492564117377	0.9801218698278926	0.39244349805792905	0
73	0	0.07431045889570058	0.9614606637171075	0.6910226966462115	0
73	0	0.07422309096241639	0.9829206184371074	0.3476635351858364	0
74	1	0.13225279222298525	0.9948203770585123	0.2151267317571004	0
74	0	0.13225279222298525	0.9669873554735124	0.6604549291846347	0
74	1	0.13225279222298525	0.9659079275085123	0.6777257708874793	0
74	0	0.13225196988280302	0.9751084278085124	0.530517814988122	0
74	0	0.132249489494388 4	0.9785856848614877	0.47488172062213274	0
75	0	0.14819383574068334	0.9951472339466115	0.22650351518812778	0
75	0	0.14819383574068334	0.9825567336416116	0.42795145314963895	0
75	0	0.14819383574068334	0.9743151653283885	0.5598165023572863	0
75	0	0.14817153619464574	0.9937455192283885	0.24893094322958492	0
76	0	0.13225196988280302	0.9852859991035124	0.3676767283618956	0
76	0	0.13225279222298525	0.9830156488914876	0.404002319687385	0
76	0	0.13225196988280302	0.9772498101464876	0.49625570896196197	0
76	0	0.13225279222298525	0.9811525463564875	0.43381195034499975	0
77	0	0.10062255758998902	0.9759257536359918	0.5116359912472519	0
77	0	0.1037376800742293	0.9879661914640083	0.3189890499938942	0
77	0	0.10685804733982694	0.9992117786990082	0.13905971400417333	0
77	0	0.11236664271984366	0.9832760299390082	0.3940316094656954	0
77	0	0.11095493370777611	0.9594684006090083	0.7749535522081832	0
78	0	0.090907670530289	0.99619720829615	0.151753495230894	0

		86	7	51	
78	0	0.09090884818123798	0.9897133414111571	0.2554953309291512	0
78	0	0.09090884818123798	0.9811187822361571	0.39300823204908275	0
78	0	0.09090884818123798	0.959232419673843	0.7431899167201278	0
79	0	0.09083500632828126	0.9874287444588431	0.29204887002354585	0
79	0	0.09087102112486789	0.9899200681461571	0.2521877042679036	0
79	0	0.09083561324515703	0.7939320297238429	3.387995277348827 5	0
79	0	0.09087102112486789	0.9806404153588429	0.40066209954359033	0
79	0	0.09087102112486789	0.9959853004438429	0.15514401974162964	0
80	1	0.07439351803786746	0.9627719544921074	0.6700420512157211	0
80	0	0.07439321630276277	0.9960684213871075	0.13729875786638648	0
80	0	0.07431045889570058	0.9879189748228926	0.2676898595795298	0
80	0	0.0741998531357335	0.9825311025728927	0.35389578694299645	0
81	0	0.1484362268191746	0.9356456819566116	1.178528030777476	0
81	0	0.14885922196571505	0.9609358188616116	0.7738859747145115	0
81	0	0.14819383574068334	0.9596213734966116	0.794917093568237	0
81	1	0.14817153619464574	0.9784185396683884	0.4941625347267148	0
81	0	0.148851807871209 9	0.9766708830816114	0.522125030826355	0
82	0	0.10062300821911316	0.9896014670759917	0.292824648893647	0
82	0	0.10805381359319347	0.9880280070259917	0.3180000013307099	0
82	0	0.10774525406933859	0.9861145161190084	0.3486158456722423	0
82	0	0.12264814290771323	0.9994916290140082	0.13458211045157734	0
83	1	0.14885922196571505	0.9855474946283883	0.3800992932571006	0
83	0	0.14885922196571505	0.9975292332766116	0.1883915385684495	0
83	0	0.148859221965715	0.9793024300483 8	0.480020293344590	0

		05	85	23	
83	1	0.14885621607572855	0.9925924852283884	0.26737948110121285	0
84	0	0.13225279222298525	0.9856571341485124	0.36173856961447814	0
84	0	0.13225196988280302	0.9930286842014875	0.24379380794665342	0
84	0	0.13225279222298525	0.9807172006835124	0.44077747879873974	0
84	0	0.13189368229595513	0.9347384580564876	1.1764371164541956	0
85	0	0.09090883536957067	0.9873074972311571	0.29398882502209345	0
85	0	0.09087102112486789	0.9796699893211577	0.41618891098875244	0
85	1	0.09087102112486789	0.9670969769411577	0.6173570422432119	0
85	0	0.09087102112486789	0.9783184279011571	0.4378138865252046	0
86	0	0.07439192628717135	0.9712926435678926	0.5337110712906068	0
86	0	0.07426455161331659	0.9960777903528925	0.13714885446362313	0
86	0	0.07431045889570058	0.9604862184528926	0.7066138156944743	0
86	0	0.07426455161331659	0.9556675844778927	0.7837119336834416	0
87	0	0.1322527273478532	0.9798993195114876	0.4538635732041004	0
87	0	0.13224683191356737	0.9474425545264876	0.9731716404564469	0
87	0	0.13224948949438844	0.9693694766664875	0.6223410027580034	0
87	0	0.1322527273478532	0.9916636910935124	0.26563369041931906	0
87	0	0.1322527273478532	0.9714571641064876	0.5889380148140572	0
88	0	0.11711333164080154	0.9744230260040083	0.5356796253719921	0
88	0	0.10491504951265226	0.9970998852359918	0.17284999818772598	0
88	0	0.11949062715810867	0.9943890134959918	0.21622393161944642	0
88	0	0.11178487661731097	0.9832494575259917	0.3944567679327272	0
89	0	0.09087102112486789	0.98360766583843	0.3531861077145214	0
89	0	0.09090884818123 7	0.98907080349384	0.265775934191089	0

		98	29	9	
89	0	0.09087102112486789	0.959884106133843	0.7327629368238399	0
89	0	0.09090884818123798	0.969565053173842 9	0.5778678356380592	0
90	0	0.07420492564117377	0.989039677927107 5	0.2497586158686272	0
90	0	0.07439321630276277	0.964542840607107 5	0.6417078827879765	0
90	0	0.07439321630276277	0.976303342177107 3	0.4535399201750252 6	0
90	0	0.0741998531357335	0.984940642897107 5	0.3153431545622623 6	0
91	1	0.07431045889570058	0.833582508977107 4	2.7370724928140304	0
92	1	0.13225116166977138	0.992659744583512 4	0.249696839873342	0
92	1	0.1322494894943884	0.969525540478512 4	0.6198439825950852	0
92	1	0.13210456380287194	0.989967353361477 7	0.2927750851156822	0
92	0	0.1322527273478532	0.986869942058512 5	0.3423336495005484	0
92	0	0.1322527273478532	0.986237144958512 4	0.3524583997372341 4	0
93	0	0.1483000860311354	0.980945479008385	0.4537315187173927	0
93	0	0.14819383574068334	0.992613949728385	0.2670360492152955 6	0
93	0	0.1483000860311354	0.990609309028384	0.2991102897606345	0
93	1	0.1488592841263559	0.996108196288384	0.2111281228272107 7	0
94	0	0.1322527273478532	0.979751510828512 4	0.4562285113460999	0
94	0	0.13224683191356737	0.967272601346475	0.6558909967331149	0
94	0	0.1322527273478532	0.970390530613512 4	0.6060041450325051	0
94	0	0.1322527273478532	0.974539139171487 6	0.5396264301547502	0
95	0	0.1112177489777097	0.986693352160997 17	0.3393544720770207	0
95	0	0.10363560850677829	0.990688585235997 17	0.2754307641116789 4	0
95	0	0.11178739230309376	0.976560158195997 17	0.5014855216591115	0
95	0	0.106444611287561	0.989850399765999	0.288841727176723	0

		66	18	24	
96	0	0.09087102112486789	0.995148939076157	0.16852579717934546	0
96	0	0.09090883536957067	0.9985195001611571	0.1145968377338703	0
96	0	0.09090883536957067	0.961279345926157	0.7104391075625124	0
97	0	0.1484362268191746	0.9724735895683884	0.5892817047293158	0
97	0	0.14885922196571505	0.9923081000033884	0.27192964318970525	0
97	0	0.14885621607572855	0.9836896952283884	0.40982407378289765	0
97	0	0.14885621607572855	0.9782751274033885	0.49645713020447757	0
98	0	0.10186828093810651	0.9963671948240083	0.18457304088521426	0
98	0	0.10715299992600516	0.9905347770859918	0.2778916936941869	0
98	0	0.11074143074762893	0.9680481929740083	0.6376769199697645	0
98	0	0.11073677493476793	0.9746787579390083	0.5315879157712078	0
99	0	0.14885056757290313	0.9982500794983884	0.17685800285131695	0
99	0	0.14885056757290313	0.9691864570716116	0.6418758072066411	0
99	0	0.14885783913894174	0.9709509053633885	0.6136446439162502	0
99	0	0.148855038907799 8	0.9792862872583884	0.4802785778987923	0
100	0	0.09087102112486789	0.955056416103843	0.8100059516446761	0
100	0	0.09087102112486789	0.971077000283843	0.5536766899140535	0
100	0	0.09087102112486789	0.993575003818843	0.19370875293090573	0
100	0	0.09090883536957067	0.9915253692238428	0.22650289555710415	0
101	0	0.07439351803786746	0.9819711149621075	0.3628555857392253	0
101	0	0.0741998531357335	0.9884392236671073	0.2593658808372154 6	0
101	0	0.07439243255500602	0.9940053060271073	0.17030859266093407	0
101	0	0.07439243255500602	0.9596867954821073	0.7194045789781071	0
102	0	0.102881044810967	0.98078697100099	0.433856539244614	0

		48	18	1	
102	0	0.10207423719648334	0.9879301665009917	0.31956544921068686	0
102	1	0.10321487332768171	0.9699062236290082	0.607948439365196	0
102	1	0.11185243038241657	0.9776363787940083	0.4842659978109567	0
103	0	0.10251003358756958	0.9829126141609916	0.3998462599824063	0
103	0	0.10856360437156166	0.9849833607690084	0.36671432526015296	0
103	0	0.10232841139739188	0.9807413208909919	0.43458694076198234	0
103	0	0.10335049982995487	0.9606390388709917	0.7562233462383895	0
103	0	0.10130155269235004	0.9561676632440084	0.8277653325047681	0
103	0	0.10259042431744544	0.9907622688890082	0.27425182605504306	0
103	0	0.10251894704510389	0.9678579682440084	0.640720514638719 8	0
104	0	0.13189368229595513	0.9798577859335125	0.454528110230951	0
104	0	0.13225196988280302	0.9770818471414876	0.4989431161492388	0
105	0	0.07439351803786746	0.9622660716371075	0.6781361742069524	0
105	0	0.07426178164396652	0.9528686141771076	0.8284954436194811	0
105	0	0.07426178164396652	0.9516635496821074	0.8477764691345674	0
105	1	0.07439243255500602	0.9381455546471074	1.064064317846447	0
105	0	0.07439352095644026	0.9749737704571074	0.47481306062835255	0
106	0	0.07439173278671102	0.9820297038178926	0.36191816435806456	0
106	0	0.0741998531357335	0.9834330139278925	0.33946521005665636	0
106	0	0.07438946216681566	0.9864681354371073	0.2909032820408852	0
107	0	0.07439351803786746	0.9916980158178925	0.20722522374512764	0
107	0	0.07439192628717135	0.9989822314778926	0.09067781190072133	0
107	0	0.07439192628717135	0.9656756637321074	0.6235827188089302	0
107	0	0.074391926287171	0.97140697853289	0.531881712458296	0

		35	26	6	
107	0	0.07431045889570058	0.9996372097078926	0.08019816370192878	0
108	0	0.13225279222298525	0.9785004099964876	0.4762461180088985	0
108	0	0.13225196988280302	0.9647969816414876	0.6955008988551983	0
108	1	0.13225196988280302	0.9624240100214876	0.7334684321628588	0
109	0	0.07439351803786746	0.9857063141278926	0.3030924189392424	0
109	0	0.07439233420879164	0.9811715013278927	0.3756493996367182	0
109	0	0.07419745401488416	0.9976768372328926	0.11156411288255275	0
109	0	0.07431045889570058	0.9863106748028925	0.2934226513514197 6	0
110	0	0.09087102112486789	0.97350743482615 7	0.5147897501547859	0
111	0	0.14885642641046318	0.9740570831216115	0.5639458162940105	0
111	0	0.14885922196571505	0.9807412720466117	0.4569988290204619 6	0
111	0	0.14885731292154444	0.9867360670016115	0.3610821416027892	0
111	1	0.148859284126355 9	0.9968556047783884	0.1991695909596858	0
112	0	0.09086601726934472	0.99919289167384 3	0.10382257710997131	0
113	0	0.13225279222298525	0.9966638760214876	0.1856307581476894	0
113	1	0.13225188414883224	0.9997257811864877	0.1366402917817096	0
113	0	0.13225196988280302	0.9844613122364876	0.380871713851084	0
113	0	0.13189368229595513	0.9927515076364876	0.2482286315134592 5	0
114	0	0.09090883536957067	0.98828819208384 3	0.2782977125915101 5	0
114	0	0.09090883536957067	0.9859431770161571	0.3158179412107336 4	0
114	0	0.0908450095194557	0.9731047151661 5 7	0.5212332625743306	0
114	0	0.09087102112486789	0.9711976517461 5 7	0.551746267158292	0
115	0	0.07439351803786746	0.9898396950728926	0.2369583457881546 7	0
115	0	0.07431045889570 0	0.9879773633328926	0.2667556437298 63	0

		58	27	66	
115	0	0.07439321630276277	0.9760866856228926	0.4570064238909323	0
115	0	0.07439321630276277	0.9911202618428925	0.21646928427436685	0
116	0	0.10257152477586845	0.9616633772909917	0.7398339369627458	0
116	0	0.10355205565702015	0.9656620860209918	0.6758546185358748	0
116	0	0.10366810714891289	0.9673219481659918	0.6492968330380391	0
116	0	0.10863670381932994	0.9834465895740083	0.3913026562122176	0
117	0	0.1483555379712882	0.9923347717683884	0.27150289509146674	0
117	1	0.14885783913894174	0.9866354735333884	0.3626916365597048	0
117	1	0.14885642641046318	0.9881567853733885	0.33835065520547414	0
117	1	0.14885642641046318	0.9909044442683884	0.29438812748927745	0
118	0	0.07439192628717135	0.9705115229421074	0.5462089971515146	0
119	0	0.14857348567452086	0.9586602534883885	0.8102950085914558	0
119	0	0.14857348567452086	0.9669984090283884	0.6768845642687396	0
120	0	0.14885642641046318	0.9791674790333884	0.48217950886732663	0
120	0	0.14885642641046318	0.9957563634633884	0.21675744615722092	0
121	0	0.13225279222298525	0.9762038067385124	0.5129917579300592	0
121	0	0.13225188414883224	0.9992515909135123	0.14422733362899468	0
122	0	0.10212504199832134	0.9980001675159917	0.15844548649272575	0
122	0	0.1050143152494693	0.9976045733009918	0.16477499183014163	0
123	0	0.09087102112486789	0.9819618101661567	0.3795197896497763	0
123	0	0.09087102112486789	0.9841503721961571	0.3445028088019788	0
123	0	0.09090883536957067	0.9781046355288843	0.44123456334592426	0
124	0	0.13225279222298525	0.9741744244064876	0.5454618644562916	0
124	0	0.13225279222298525	0.9873240088814876	0.335068582746311	0

		25	76	1	
124	0	0.13225279222298525	0.9634182676864876	0.7175603148073358	0
124	0	0.13225057316286556	0.9701229402935124	0.6102855887302632	0
125	0	0.09087102112486789	0.9934304907038443	0.19602096200281824	0
125	0	0.09083387567844771	0.9600777636838443	0.7296644170531293	0
125	0	0.09083387567844771	0.9709698756461577	0.5553906835476627	0
125	0	0.09083561324515703	0.9632023204388443	0.6796715255801433	0
126	0	0.07439351803786746	0.9673591326628926	0.5966472248640017	0
126	0	0.07431045889570058	0.9606980492628926	0.703224523860355	0
126	0	0.07431045889570058	0.9729148755028926	0.5077553689527669	0
126	0	0.07439321630276277	0.9669267200521074	0.6035658243382925	0
127	0	0.09090883536957067	0.9766887130661571	0.46388931522327226	0
127	0	0.09087102112486789	0.9963653373838443	0.14906343072152367	0
127	0	0.09087102112486789	0.9686540288711577	0.5924442196389403	0
127	0	0.09083561324515703	0.9811432123961577	0.39261734961893113	0
128	0	0.11002555894826734	0.9913170368659917	0.2653755413718979	0
128	0	0.10644219646684941	0.9822935728440083	0.4097509177639358	0
128	0	0.11210165045173354	0.9856633513140083	0.3558344801543032	0
128	0	0.11116858829607773	0.9963087776540084	0.1855077152947252	0
129	0	0.1322527273478532	0.9923382353235125	0.25484098632451846	0
129	0	0.1322527273478532	0.9591514595985124	0.785829221536861	0
129	0	0.1322527273478532	0.9951380243814876	0.21004437627779166	0
129	0	0.1322527273478532	0.9882358719864877	0.32047877791286167	0
130	0	0.09090837631784783	0.9740457076361569	0.5061773880557056	0
130	0	0.09090837631784783	0.9935911753838469	0.193450007976857	0

		83	3	57	
130	0	0.09090837631784783	0.969633772908843	0.5767683202433027	0
130	0	0.09087102112486789	0.976595414603843	0.46538209012441667	0
131	1	0.13225196988280302	0.98709951020351 24	0.33866056040070497	0
131	0	0.13225196988280302	0.96103798615851 23	0.7556448066037484	0
131	0	0.13225196988280302	0.99295840953648 76	0.2449182022131416	0
132	0	0.07439351803786746	0.97510895282289 24	0.4726501434942862	0
132	0	0.07439233420879164	0.99807684341289 26	0.10516401612858456	0
132	0	0.07439243255500602	0.97785382276289 26	0.4287322390432635	0
132	0	0.07439243255500602	0.99227831304289 26	0.19794047122940514	0
133	0	0.101279838667222	0.96360314043900 82	0.7087977369043201	0
133	0	0.10273386176545968	0.94861197318900 83	0.9486563332262898	0
133	0	0.1077960508763136	0.96229060999900 83	0.7297982169682214	0
133	0	0.107527506967794	0.94312885886599 17	1.0363861332518105	0
134	0	0.14885922196571505	0.98179028354161 16	0.4402146506759566	0
134	0	0.14885922196571505	0.97635305729838 85	0.5272102416686781	0
134	0	0.14857348567452086	0.97041020853161 15	0.6222957903508783	0
134	0	0.14857348567452086	0.98125371451161 16	0.4487997523040997	0
135	0	0.13225188414883224	0.99618766831648 75	0.1932500788966414	0
135	0	0.13225279222298525	0.97209652236351 24	0.5787082860998489	0
135	0	0.13200499486528186	0.99269886238148 75	0.2490709553136542	0
135	0	0.1321203702118354	0.95668360621148 76	0.8253148626126214	0
136	0	0.09087102112486789	0.963433435336157	0.675973717251486	0
136	0	0.09090845253455472	0.963421791891157	0.6761599835096107	0
136	0	0.09090845253455 4	0.96989863650615	0.572530504094027	0

		72	7	8	
136	0	0.09090845253455472	0.9509657869661571	0.875455996105963	0
136	0	0.09090845253455472	0.9420325856011577	1.0183871704660137	0
137	0	0.09086601726934472	0.9852314079811569	0.32720624198768467	0
137	0	0.09090845253455472	0.9965578636188843	0.14598301198480043	0
137	0	0.09086601726934472	0.9986647885738443	0.11227222390310347	0
137	0	0.09086601726934472	0.9933979153988443	0.1965421667096806	0
138	0	0.07439352095644026	0.9800613516128925	0.39341178917627617	0
138	0	0.07439352095644026	0.9711614168428926	0.5358106981931368	0
138	0	0.07439352095644026	0.9746681070428925	0.4797036736311903	0
138	0	0.07439352095644026	0.9805428637428926	0.38570759765551194	0
139	0	0.10665919882408208	0.9805416769559918	0.4377812426608759	0
139	0	0.10686298272338636	0.9795764035840082	0.4532256114821871	0
139	0	0.10638398660783359	0.9779702415840081	0.4789241949454403	0
139	0	0.10981826792708056	0.9750915316490083	0.5249835386050987	0
139	0	0.10353409973284082	0.9684909857640082	0.6305922376832105	0
140	0	0.10532248320310796	0.9978121630309917	0.1614535572534823	0
140	0	0.10555749405197183	0.9859686774459918	0.3509492636653746	0
140	0	0.10276005220708813	0.9981060260090083	0.15675175116709808	0
140	0	0.12672115797980407	0.9834039099859917	0.3919855293936416	0
141	0	0.14885642641046318	0.9938533514966117	0.2472056275111421	0
141	0	0.14885642641046318	0.9698887627283884	0.6306389204309646	0
141	1	0.14885642641046318	0.9897707038183884	0.31252796866344934	0
141	0	0.14885642641046318	0.9079704538033885	1.6213315341352532	0
141	0	0.14885642641046	0.9138205658283	1.527729772828591	0

		18	83	5	
142	0	0.14821816806490334	0.9935938343433884	0.2513579005833809	0
142	0	0.14821816806490334	0.9983699336466115	0.17494033711677145	0
142	0	0.1488592841263559	0.9837028319466117	0.4096138863611476	0
142	0	0.14885783913894174	0.9909663903566116	0.29339699040694983	0
143	0	0.10161349779808417	0.9747258248259918	0.5308348458296318	0
143	0	0.1047517531098084	0.9617819978759917	0.7379360082332139	0
143	0	0.10378200715437189	0.9740915001859918	0.5409840366981976	0
143	0	0.10067377775874119	0.9484507513790083	0.9512358813293962	0
143	0	0.10068764040222847	0.9400967436409917	1.0848999607361234	0
143	0	0.1021651245213589	0.9450647919440083	1.005411214293027	0
144	0	0.14885642641046318	0.9834881338066115	0.4130490554600304	0
144	0	0.14885642641046318	0.9787343047983884	0.48911029432500625	0
144	0	0.148855038907799 8	0.9652160124033885	0.7054029007953029	0
144	0	0.148855038907799 8	0.9846630927933884	0.3942497179165053	0
144	0	0.148855038907799 8	0.9812231628583884	0.449288578593283	0
144	1	0.148855038907799 8	0.9798010697733884	0.47204206039485996	0
145	1	0.1322527273478532	0.9980499236035124	0.1634540042021338	0
145	1	0.1322527273478532	0.9900034832735124	0.2921970067153169	0
145	0	0.1322527273478532	0.9805131241114876	0.44404270286647013	0
145	0	0.1322527273478532	0.9867833430914876	0.3437192325126734	0
146	0	0.10477452970500392	0.9932689950859918	0.23414422022655182	0
146	1	0.10971701007632641	0.9742918019109917	0.5377792101628015	0
146	0	0.10575641566479796	0.9940688280140082	0.22134689762939946	0
146	1	0.102771116275703	0.99446889269900	0.214945864795741	0

		2	83	15	
147	0	0.14885642641046318	0.9917656409016116	0.28060898593496575	0
147	0	0.14885922196571505	0.9777702501366116	0.5045351637894868	0
147	0	0.14885642641046318	0.9538705106683885	0.8869308682539805	0
147	0	0.14885642641046318	0.9694320557633884	0.6379462294435677	0